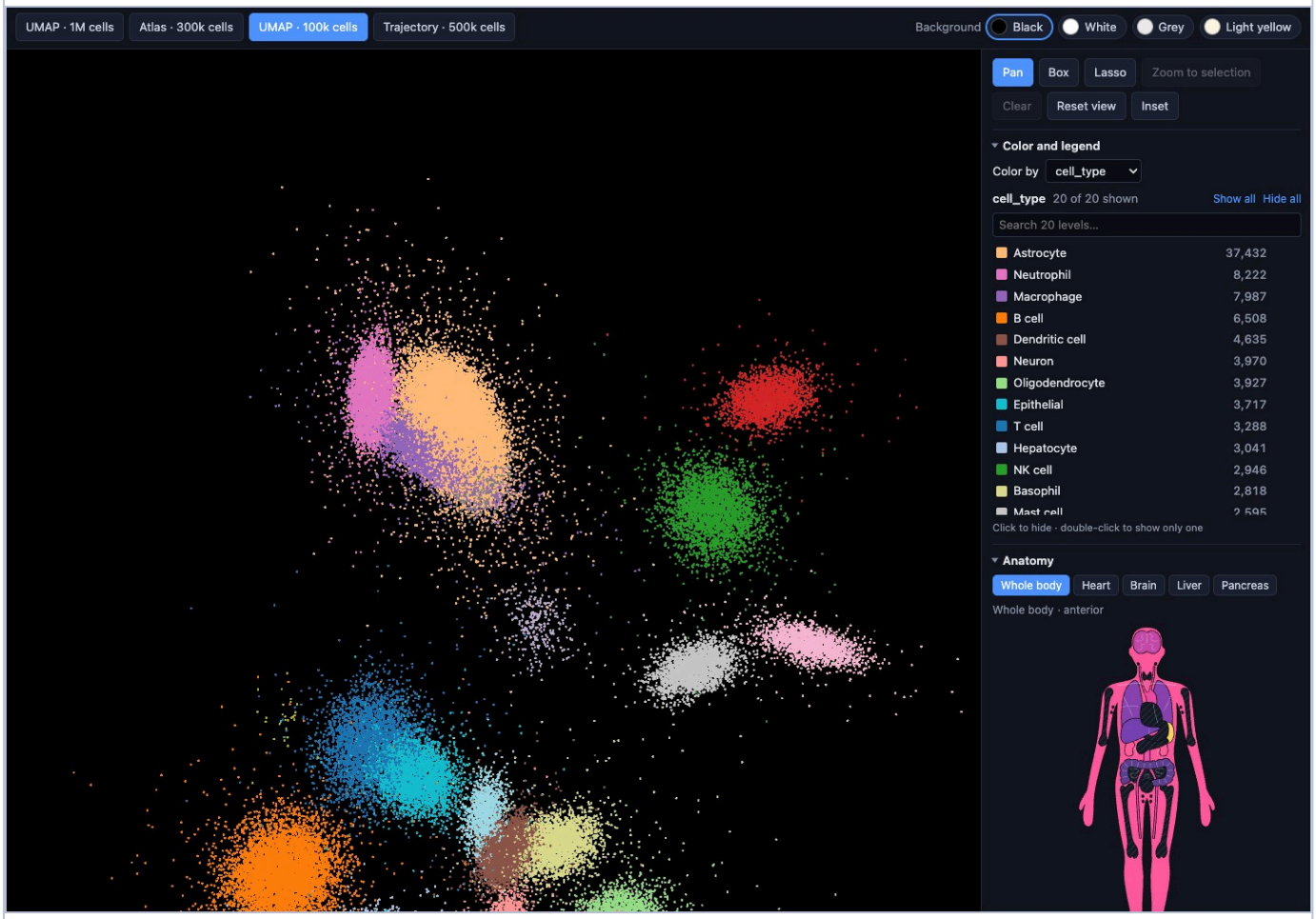
THE COMPLETE GUIDE

Cell Viewer

See, select and understand millions of cells in your browser. How to use it, how it works, and how to change it.



The Cell Viewer main screen.

For the code in this folder (apps/web, packages/core, server). Written in plain English for people who use the app and people who want to change it.
Screenshots were taken from the running app with the built-in demo data.

Contents

PART 1: START HERE

1 What is Cell Viewer?
2 A tour of the screen
3 Installing and running the app

PART 2: USING THE VIEWER

4 Moving around and looking at one cell
5 Selecting cells
6 Colors, the legend and backgrounds
7 Trajectories and the principal graph
8 The inset: a second, magnified view
9 Anatomy: link cells to the body
10 Chat: ask about the selected cells
11 Display options and speed

PART 3: YOUR OWN DATA

12 Preparing your own data
13 What is inside a dataset folder

PART 4: HOW THE CODE WORKS

14 The big picture
15 The folder map
16 Life of a dataset: from click to dots
17 Data in memory: the CellStore
18 Drawing the dots: ScatterCanvas
19 How selection works
20 Filtering and color
21 Themes
22 How the trajectory view works
23 How the inset works
24 How the anatomy view works
25 How the chat works
26 The server

PART 5: REFERENCE AND RECIPES

27 File-by-file reference
28 The HTTP API
29 Recipes: how to change things
30 Rules the code follows
31 When something goes wrong
32 Performance: how fast is it?
33 Cheat sheet and glossary

How to read this guide

You do not have to read it from start to finish. Pick the path that fits you.

You are...	Read
A user who wants to explore data	Part 1, then Part 2. Part 3 when you bring your own data.
A scientist who wants to load a file	Chapter 3 to run the app, then Chapter 12.
A developer new to the project	Chapters 1, 2, 14, 15 and 16 first. Then any chapter of Part 4 when you need it.
Someone with a change in mind	Chapter 15 ("how to find your way"), then Chapter 29 (recipes).
Fixing a problem	Chapter 31.

How the pictures work

Screenshots carry **numbered pink boxes**. The number in the box matches the number in the list under the picture. The list says what the boxed part is and what it does.

Colored boxes in diagrams always mean the same thing:
browser app core logic server
Amber boxes are data or files. Orange boxes are the GPU.

Kinds of notes

> **Note**
> Extra information that helps you understand.

> **Tip**
> A shortcut or a good habit.

> **Warning**
> Something that is easy to get wrong.

> **Idea**
> The reason behind a design choice.

A small promise

The names, numbers and file paths in this guide come from the code itself. Where the guide says "the app does X", the file names in the text tell you where to find the code that does it.

PART 1

Start here

What the app is, what every part of the screen means, and how to run it.

CHAPTER 1

What is Cell Viewer?

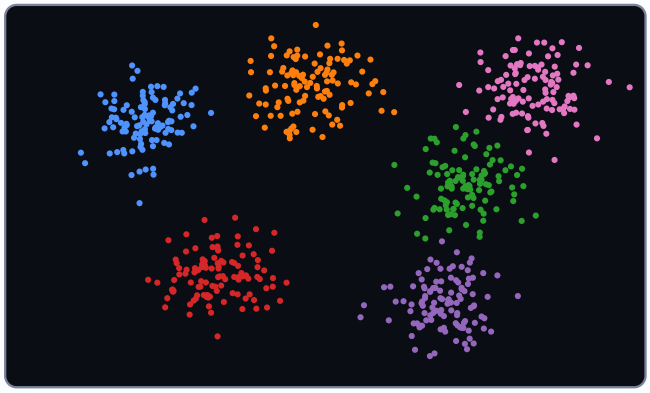
Cell Viewer is a web app that draws a picture of cells. Every dot on the screen is one real cell (or, in the demo data, one made-up cell). Cells that look alike are placed close together, so similar cells form islands. You can look at one million cells at once, zoom into an island, draw a line around some cells, and ask what those cells are.

The app was built for scientists who study **single-cell data**. Instead of measuring a whole piece of tissue, a lab measures thousands or millions of cells one by one. A computer program then squeezes all of those measurements into a flat map with two numbers per cell (an **x** and a **y**). Cell Viewer shows that map and lets you work with it.

> **You do not need to be a biologist or a programmer to start**
>
> Parts 1 and 2 of this guide are for everyone who wants to *use* the app. Parts 3 and 4 explain your data and the code, one idea at a time, for anyone who wants to *change* it. Part 5 is a reference with step-by-step recipes for common changes.

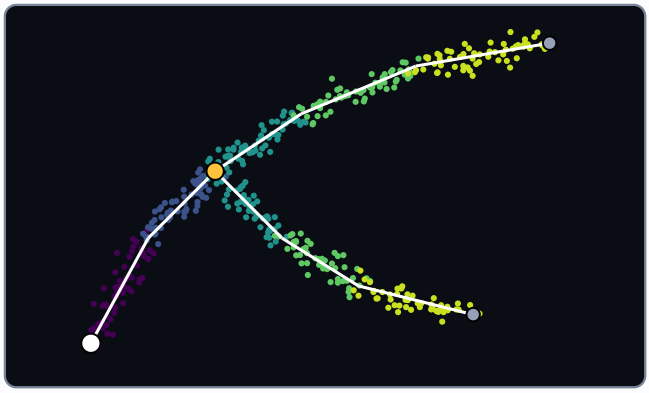
The two pictures it can draw

Embedding view



Islands of similar cells (each color = one cell type)

Trajectory view



Cells ordered along a process, with a graph on top

The same engine draws both pictures. In the trajectory view the color shows "pseudotime" (how far along the process a cell is) and a white line shows the main path.

What you can do with it

You want to...	Use this	Chapter
See a whole dataset at once	Pan and zoom the plot	4
Find out what one cell is	Click it to open its details card	4
Study a group of cells	Draw a box or a lasso around them	5
Hide or isolate a cell type	Click or double-click it in the legend	6
Compare a zoomed area with the whole plot	Open the Inset	8

You want to...	Use this	Chapter
See where cells come from in the body	Use the Anatomy panel	9
Ask questions about the selected cells	Use the chat box	10
Load your own data	Run the prep tool on a .h5ad or Parquet file	12
Change how the app works	Read Part 4, then the recipes	14 to 26, 29

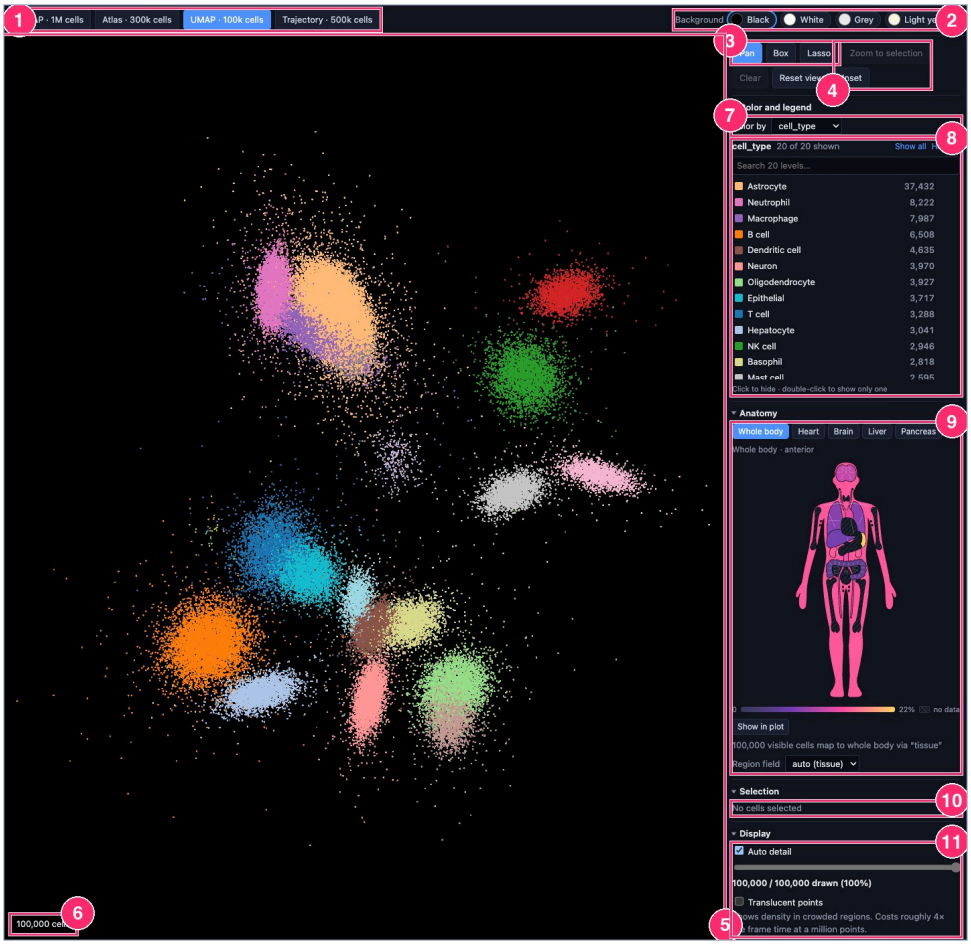
Words you will see in this guide

Word	Plain meaning
Cell	One dot on the plot. In the code it is just a number: its position (index) in a list.
Embedding	A flat map of cells. Common kinds are **UMAP** and **t-SNE**. Similar cells sit near each other.
Trajectory / pseudotime	A path that cells follow as they change, for example while they grow up. Pseudotime is a number from 0 to 1 saying how far along the path a cell is.
Principal graph	The thin tree drawn over a trajectory. It has a **root** (the start), **branch points** (where the path splits) and **leaves** (the ends).
Annotation / field	A label stored for every cell, such as cell type, tissue, donor or stage. The app calls each kind of label a *field*.
Level	One possible value of a field. The field `tissue` has levels such as lung, liver and brain.
Tile / chunk	A small binary file holding a slice of the data. A dataset is cut into chunks of 250,000 cells so the browser can load it piece by piece.
Manifest	A small JSON file that describes a dataset: how many cells, which fields, how many chunks.
Typed array	A fixed-size, very fast list of numbers (for example `Float32Array`). The app keeps all cell data in typed arrays, never in objects.
GPU / WebGL	The graphics chip in your computer. WebGL lets a web page use it. It can draw a million dots in a blink.
deck.gl	An open-source library that turns data into GPU drawings. Cell Viewer uses it to draw the dots.
Web Worker	A helper program that runs in the background of the page, so heavy work does not freeze the screen.
Selection	The group of cells you have outlined. Several parts of the screen react to it.
Level of detail (LOD)	Drawing only a sample of the cells when there are too many to draw smoothly.

CHAPTER 2

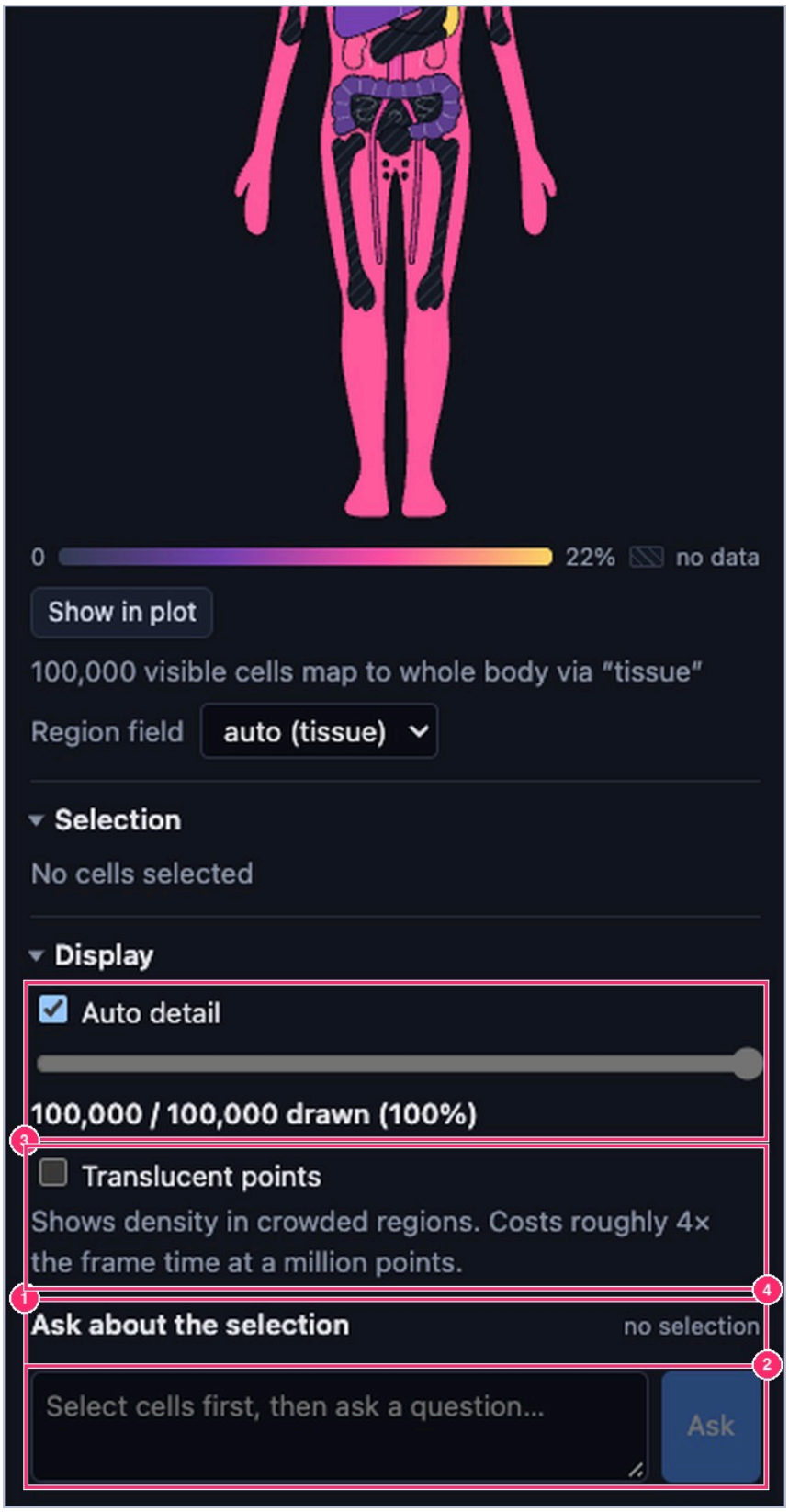
A tour of the screen

This chapter names every part of the screen. Each number in the picture matches a number in the list under it. The same names are used in the rest of the guide and in the code.



The main screen, with every area labeled.

1. **Dataset switcher.** One button per dataset on the server. The blue one is open now. Clicking another loads that dataset.
2. **Background picker.** Four backgrounds: black, white, grey and light yellow. The whole page changes, and the app remembers your choice.
3. **Mode buttons. Pan** moves the plot, **Box** and **Lasso** draw a selection.
4. **Action buttons. Zoom to selection**, **Clear**, **Reset view** and **Inset**.
5. **The plot.** The big black area. One dot is one cell. Drag and scroll here.
6. **Status line.** Shows how many cells are loaded, or a message such as "loading".
7. **Color by menu.** Chooses which field paints the dots.
8. **Legend.** One row per level of the field. Click rows to hide cells.
9. **Anatomy panel.** A body (or organ) drawing shaded by where the cells come from.
10. **Selection summary.** Counts and breakdowns for the selected cells. It says "No cells selected" until you select.
11. **Display options.** Level of detail slider and the Translucent points switch.



The bottom of the side panel. Scroll the side panel to reach it.

1. **Chat panel.** Ask a question about the selected cells here. It stays disabled until you select something.
2. **Question box and Ask button.** Type a question and press Ask or Enter.
3. **Level of detail.** Auto detail on or off, plus a slider.
4. **Translucent points.** Shows density, but it is slower. Off by default.

The side panel scrolls

The right-hand panel is taller than most screens. Each section (Color and legend, Anatomy, Selection, Display) has a small triangle you can click to fold it. The app remembers which sections you folded.

Where things live in the code

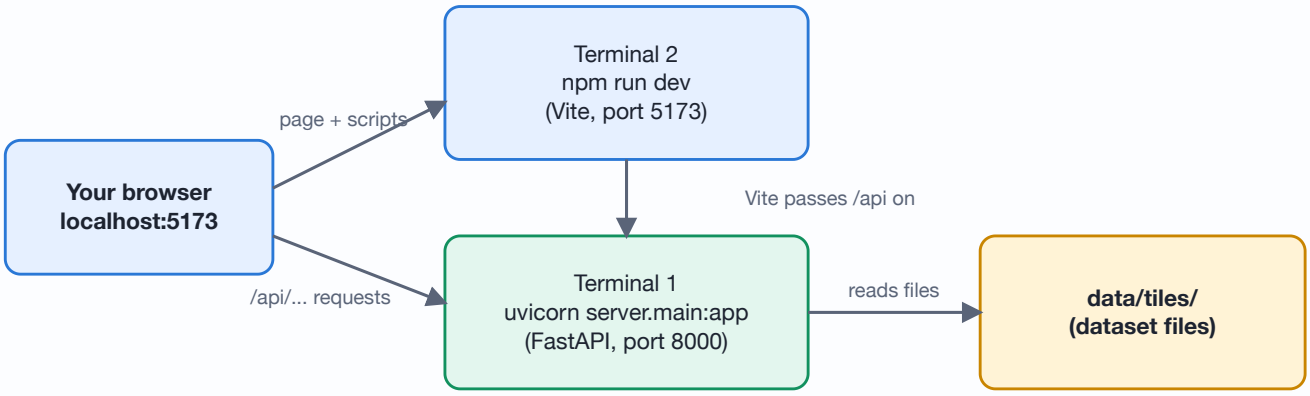
Each area of the screen is built by one small file. You will use this table again in Part 3.

Area on screen	Built by (file)	In plain words
Dataset switcher	`apps/web/src/ui/ViewSwitcher.ts`	Makes the row of dataset buttons
Background picker	`apps/web/src/ui/ThemePicker.ts`	Makes the four theme buttons
Mode and action buttons	`apps/web/src/ui/Toolbar.ts`	Makes Pan, Box, Lasso, Zoom, Clear, Reset, Inset
The plot	`apps/web/src/views/ScatterCanvas.ts`	Owns the GPU canvas and draws the dots
Color by and legend	`apps/web/src/ui/Legend.ts` and `ScatterView.ts`	The menu, the rows and the filter chips
Anatomy panel	`apps/web/src/anatomy/AnatomyPanel.ts`	The organ drawing, tabs and region selector
Selection summary	`apps/web/src/ui/SelectionSummary.ts`	Counts and breakdowns
Display options	`apps/web/src/ui/LodControl.ts`, `RenderControl.ts`	Detail slider and translucent switch
Chat	`apps/web/src/chat/ChatPanel.ts`	Messages, the question box and the Ask button
Details card	`apps/web/src/ui/InfoCard.ts`	The pinned card that opens when you click a cell
Inset	`apps/web/src/inset/InsetController.ts`	The floating panel with connector lines
Everything glued together	`apps/web/src/views/ScatterView.ts`	Creates all of the above and connects them

CHAPTER 3

Installing and running the app

You run Cell Viewer as **two programs** at the same time. One is the **data server** (written in Python). It hands out the cell data. The other is the **web app** (written in TypeScript). It draws the picture in your browser. You open the web app in your browser, and it talks to the data server in the background.



How the pieces connect while you work on the app (development mode).

What you need first

Tool	Version	How to check it
Node.js	18 or newer	`node --version`
Python	3.12 or newer	`python3 --version`
A browser	Any recent one with WebGL 2 (Chrome, Edge, Firefox, Safari)	Open any 3D web demo

Step by step

1. **Open a terminal** and go to the project folder: `cd cell-viewer-master`.

2. **Install the Python parts.** This makes a private Python environment in a folder called `.venv` and installs the server's libraries into it.

```
python3 -m venv .venv
.venv/bin/pip install -r server/requirements.txt
```

3. **Install the web parts.** This downloads the libraries listed in `package.json` into `node_modules`.

```
npm install
```

4. **Make some demo data.** This creates four fake datasets in `data/tiles/` (about 65 MB, takes a minute).

```
.venv/bin/python -m server.prep --sizes 100000 1000000 --traj-size 500000 --atlas-size 300000
```

5. **Start the data server** (Terminal 1). Leave it running.

```
.venv/bin/python -m uvicorn server.main:app --port 8000 --env-file .env
```

6. **Start the web app** (Terminal 2). Leave it running too.

```
npm run dev
```

7. **Open the app.** Go to `http://localhost:5173` in your browser. If that port is busy, Vite prints a different one (such as 5174). Use that one. It works the same.

> **Windows users**
> Replace `.venv/bin/python` with `.venv\Scripts\python` and `.venv/bin/pip` with `.venv\Scripts\pip`.

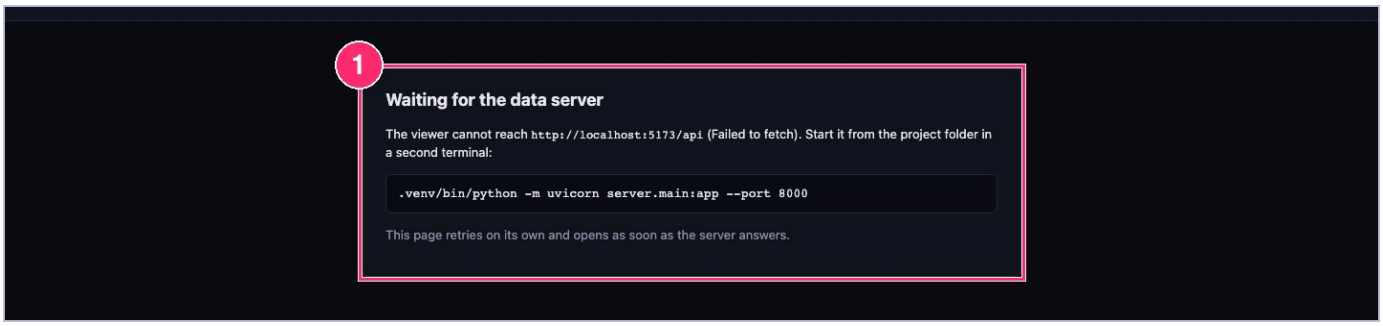
What the demo datasets are

Id (name)	Cells	Kind	What it is good for
`sim100k`	100,000	Embedding	A fast first look. Most screenshots in this guide use it.
`sim1m`	1,000,000	Embedding	Shows that a million cells stay smooth.
`traj500k`	500,000	Trajectory	A branching path with a principal graph on top.
`atlas300k`	300,000	Embedding with organs	Has an `anatomical_region` field so the Anatomy panel lights up.

All four are **made-up** data created by `server/simulate.py`. The cell types and tissues have real names, but the numbers are random.

If the data server is not running

If you open the page before the data server is up, the app does not show a blank screen. It explains what is missing, shows the command to run, and tries again by itself every few seconds. As soon as you start the server, the page opens.



The "Waiting for the data server" page. Start the server and this page turns into the app.

1 **Message box.** Says which address it cannot reach and why.

Choosing what opens first

Add this to the address	What it does
`?dataset=sim1m`	Opens a dataset by its id instead of the default.
`?theme=white`	Starts with a background: `black`, `white`, `grey` or `yellow`. The link wins over what the browser remembered.
`?dataset=sim100k&theme=grey`	Both together. Use `&` between them.
`/anatomy.html`	A separate page that shows every organ drawing, one color per region.

If you ask for a dataset that does not exist, the app opens another one and tells you so on the status line, so you notice the typo:



Asking for `?dataset=nope`.

1 **Notice.** Names the dataset you asked for and lists the ones that exist.

Connecting the chat to Claude (optional)

The chat panel works without any setup, using a simple offline helper. To get answers from a real AI model, put your Anthropic API key in the file `.env` (one line: `ANTHROPIC_API_KEY=...`) and restart the data server with `--env-file .env` as shown above. The key stays on the server. The browser never sees it.

Stopping and starting again

- Press `Ctrl` + `C` in each terminal to stop a program.
- Next time, you only need steps 5 and 6. The install steps and the demo data are one-time jobs.
- Your own datasets and the demo datasets are just folders in `data/tiles/`. Delete a folder to remove a dataset.

Making a production build

When you are ready to put the app on a real server, build the web app once and let the data server serve it too. Then only one program runs, on one address.

```
npm run build                          # makes the folder dist/
.venv/bin/python -m uvicorn server.main:app --host 0.0.0.0 --port 8000 --env-file .env
```

Open `http://your-server:8000`. The server notices the `dist/` folder and serves the page from it. See Chapter 26 for how that works.

PART 2

Using the viewer

Every feature, one at a time, with labeled screenshots.

CHAPTER 4

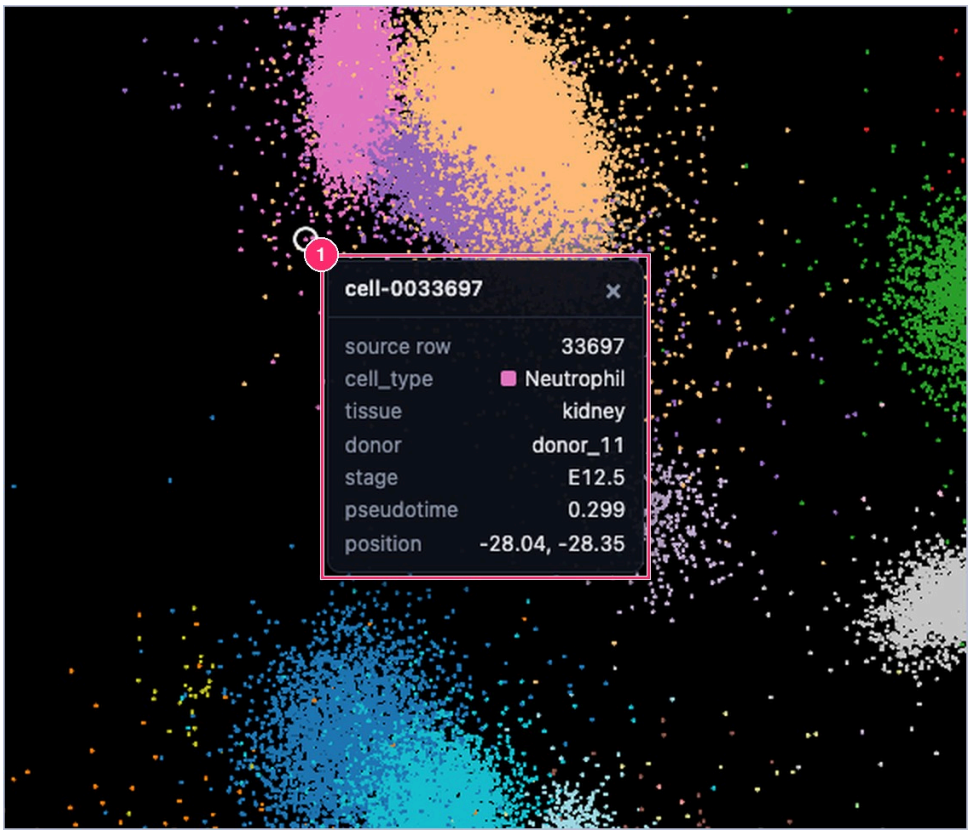
Moving around and looking at one cell

Pan, zoom and reset

What you do	What happens
Drag on the plot (Pan mode is on by default)	The picture slides with your pointer.
Scroll the mouse wheel or use a trackpad pinch	The picture zooms in or out around the pointer.
Click **Reset view**	The camera flies back so the whole dataset fits on the screen.
Move the pointer over the plot without clicking	**Nothing happens.** This is on purpose. There is no hover tooltip. Details appear only when you click.

Click a cell to see what it is

Click directly on a dot. A small card opens next to it with everything the app knows about that cell, and a white ring marks the dot so you can find it again.



A pinned details card. The ring around the clicked dot is drawn just under it.

1. **Details card.** The title is the cell's own name (for example a barcode) once the server has answered. Until then it says "Cell".

Row on the card	Meaning
source row	The row number of this cell in *your original file*. Use it to find the cell in your own notebook. Older datasets without this show "viewer index" instead.
cell_type, tissue, donor, stage	One row per categorical field. The row for the field you color by shows a small color square.
pseudotime	One row per numeric field.
position	The x and y of the cell on the map.

- The card **follows its cell** when you pan or zoom, and flips to the other side near the edge of the plot so it never leaves the screen.
- Close it with `Esc`, the × button, or by clicking empty space.
- If you hide the clicked cell's type in the legend, the card closes by itself.
- Only cells that are actually **drawn** can be clicked. When the app draws a sample of a huge dataset (Chapter 11), a click picks among the drawn cells.

> **How does a click find the right dot?**
> The dots are drawn by the GPU, which cannot easily say "which dot is under the mouse?" without a slow extra drawing pass. So the app does not ask the GPU. It turns your click into a position on the map and asks a background helper (a Web Worker) for the nearest cells to that position. Chapter 19 explains how that lookup stays fast.

CHAPTER 5

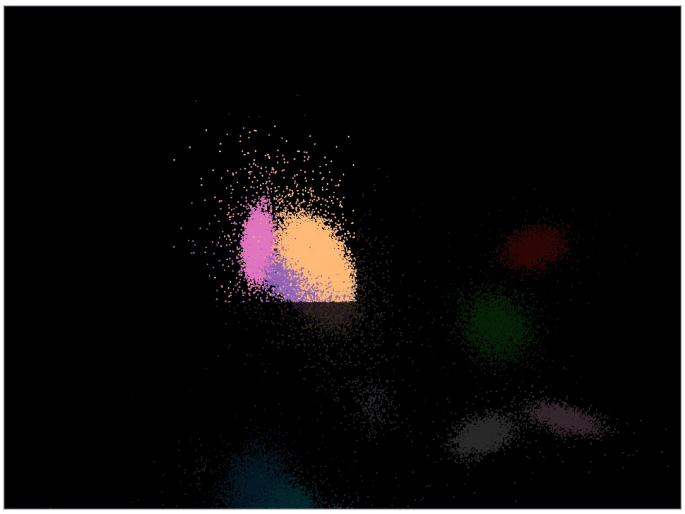
Selecting cells

A **selection** is a group of cells you have outlined. Once you have one, three things happen at the same time: the unselected cells fade, the Selection summary fills in, and the chat box wakes up.

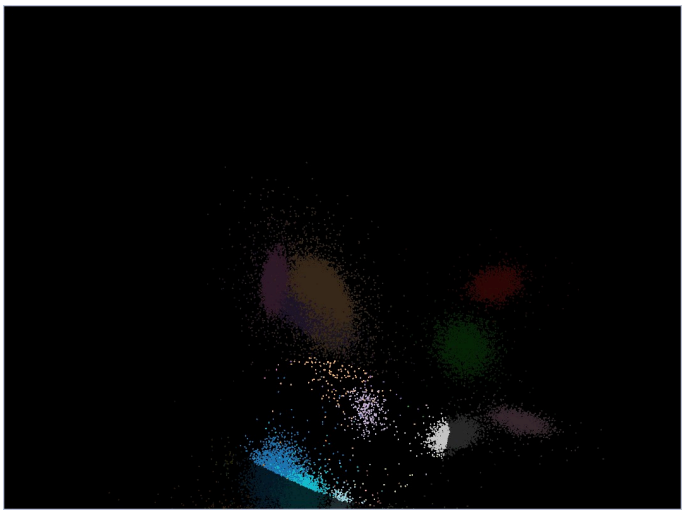
Box and lasso

1. Click **Box** (a rectangle) or **Lasso** (a freehand loop) in the toolbar. The button turns blue.
2. Drag on the plot. While you drag, a preview outline shows the area. Release to finish.
3. The cells inside become the selection. Press `Esc` *during* the drag to cancel.
4. Click **Pan** to go back to moving the plot.

A new selection replaces the old one. A drag shorter than 5 pixels is treated as a click, not a selection. Cells that the legend has hidden can never be selected.

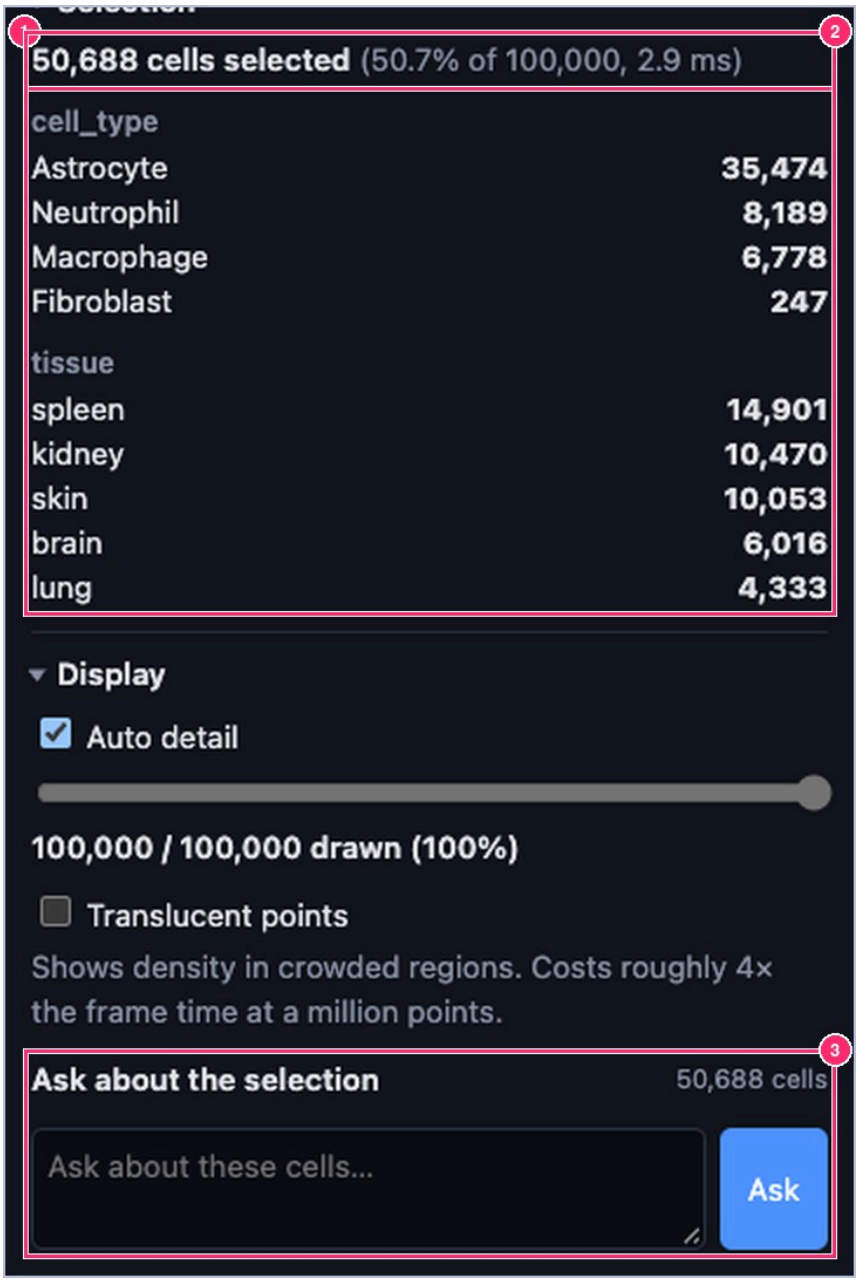


Box select: the selected cells keep their color. All others fade toward the background.



Lasso select: the same idea with a freehand shape.

The Selection summary

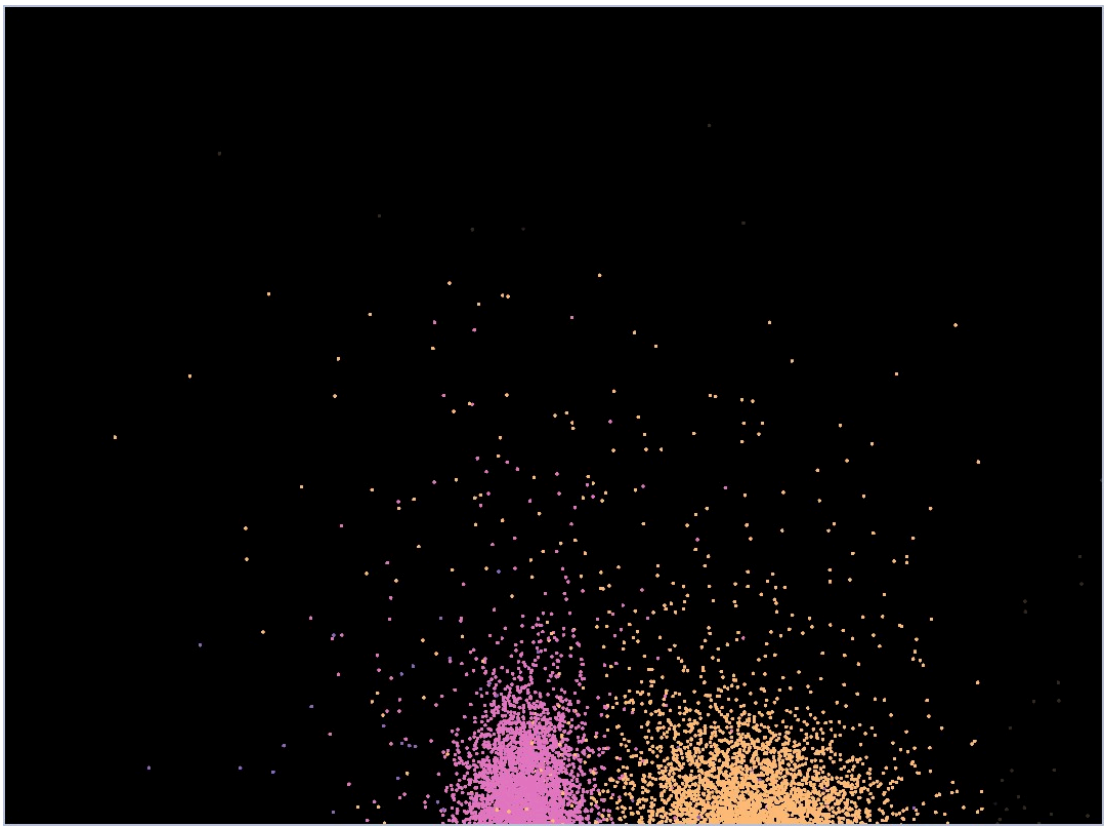


The side panel after a box selection. For every field, the five biggest groups are listed with their counts.

1. **Section title.** Click the little triangle to fold or open the section.
2. **Counts.** How many cells are selected, what share of the whole dataset that is, and how long the lookup took in milliseconds.
3. **Chat box.** Now enabled. The badge on the right counts the selected cells.

Zoom to selection, clear

- **Zoom to selection** flies the camera so the selected area fills the plot.
- **Clear** removes the selection. Everything returns to full color.
- Making a new selection also replaces the old one, and clicking the only selected anatomy region again clears it.



*After **Zoom to selection**.*

> **Filters and selections work together**
> If you hide a cell type in the legend, hidden cells are removed from the selection. If you show them again, they come back (as long as they were inside your outline). The app keeps your *raw* outline and recomputes the visible part whenever the filter changes.

CHAPTER 6

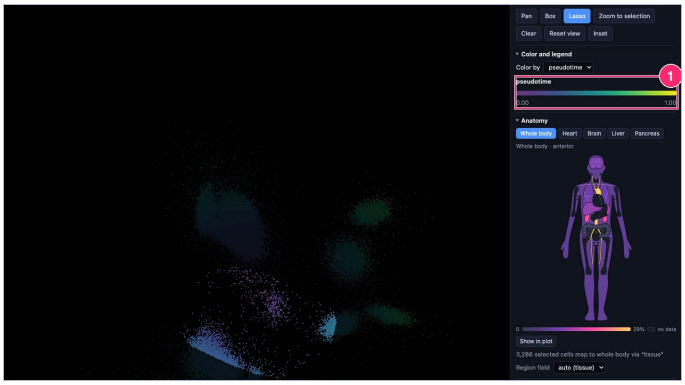
Colors, the legend and backgrounds

Color by

The **Color by** menu lists every field in the dataset. Categorical fields (cell type, tissue, donor, stage) give each level its own color. Numeric fields (such as pseudotime) give a smooth color ramp from low to high.



Categorical: one color per tissue.

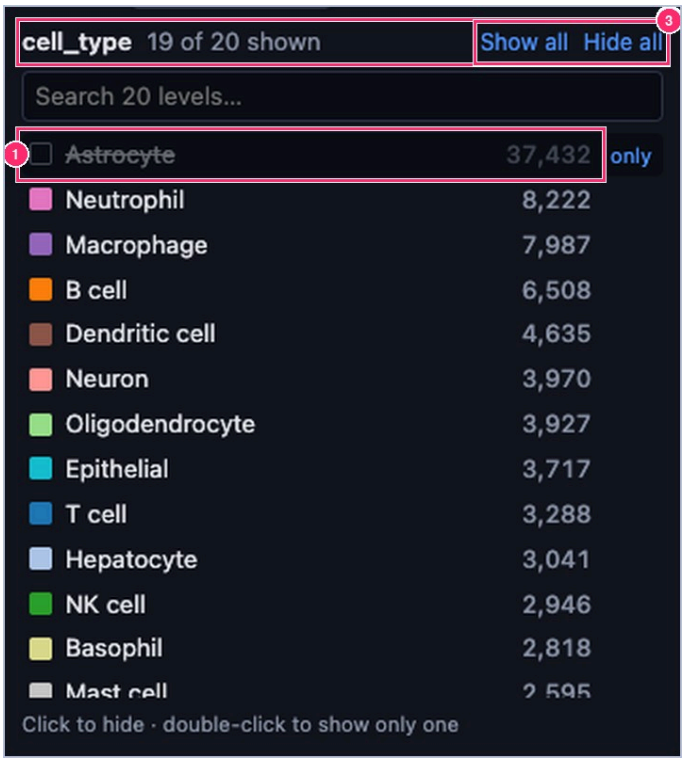


Numeric: a viridis ramp.

1. **Ramp.** A bar with the lowest and highest value. Dark purple is low, yellow is high.

The legend works like a Plotly legend

Do this	Result
Click a row	Hides that level. The row is struck through and the header says, for example, "19 of 20 shown".
Click it again	Shows it again.
Double-click a row (or hover and press **only**)	Shows *only* that level.
Double-click the only visible row	Shows everything again.
Show all / Hide all	Acts on every level of the field at once, not just the rows you can see.
Type in the **search box**	Narrows the list. The box appears when a field has more than 10 levels.



One level hidden.

1. **Hidden level.** Struck through. Its cells vanish from the plot.
2. **Counter.** "19 of 20 shown".
3. **Show all / Hide all.** Bulk buttons.

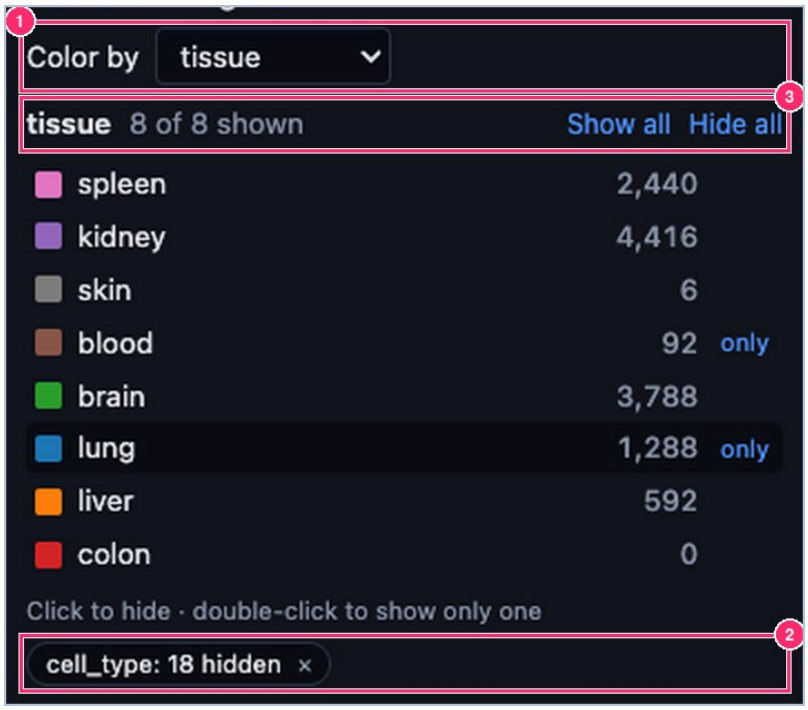


After a double-click.

1. **Counter.** "1 of 20 shown": only one level is on.

Filters stay on when you change the color field

Suppose you keep only two cell types and then switch **Color by** to tissue. The cell-type filter stays in force, and it appears as a small **chip** you can remove. The counts beside each tissue only count the cells you can still see.

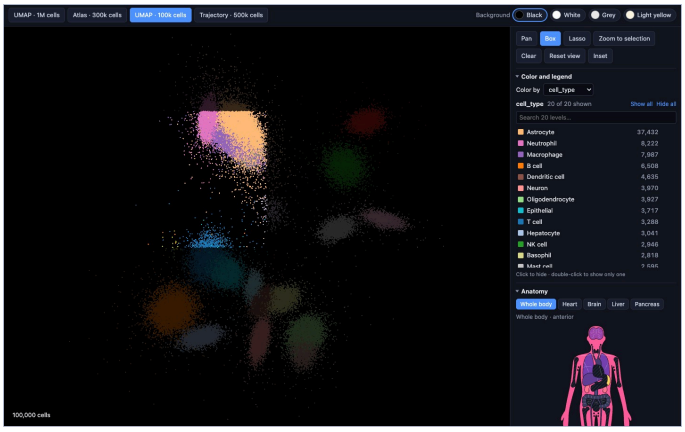


A chip keeps a filter visible while you color by something else.

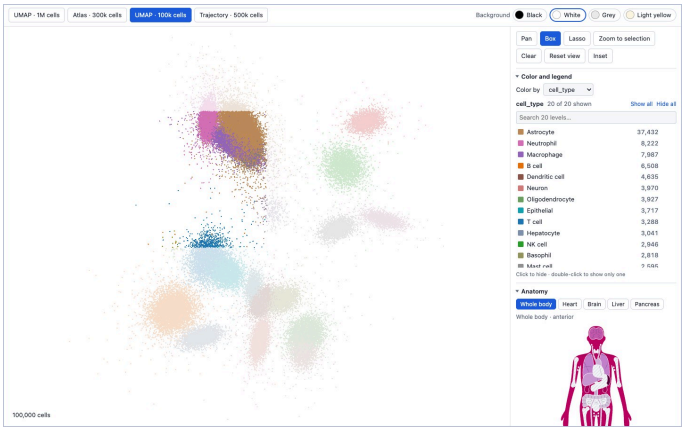
1. **Color by tissue.** The color field was changed after filtering.
2. **Filter chip.** Shows the filter that is still active on *cell_type*. Click its × to clear it.
3. **Counts.** Only visible cells are counted.

Four backgrounds

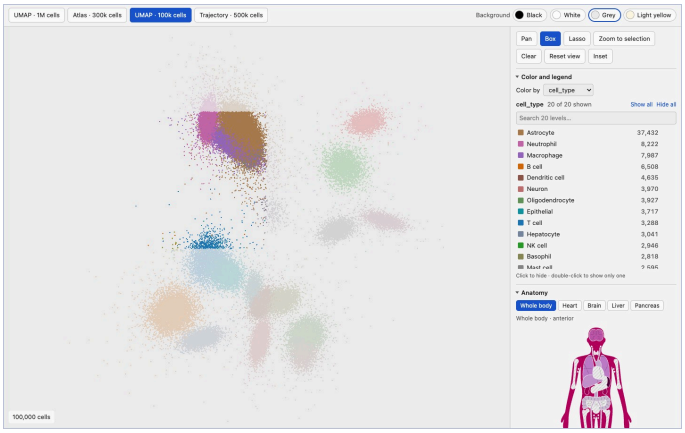
The **Background** picker in the top-right corner switches the whole page between four looks. Your choice is saved in your browser. Add `?theme=white` (or `grey`, `yellow`, `black`) to the address to open a link with a specific background.



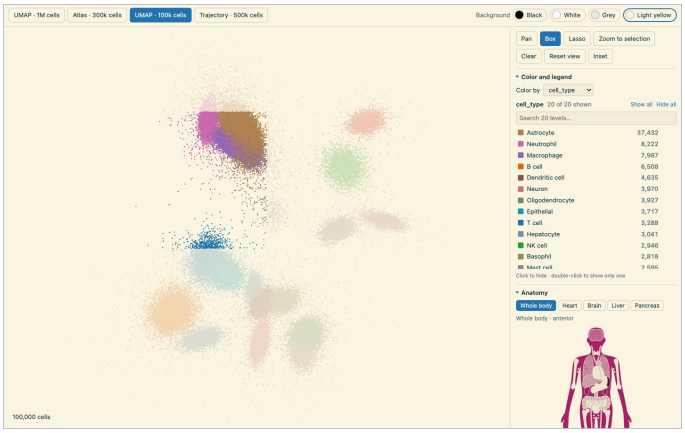
Black.



White.



Grey.



Light yellow.

Why do the cells fade toward the background?

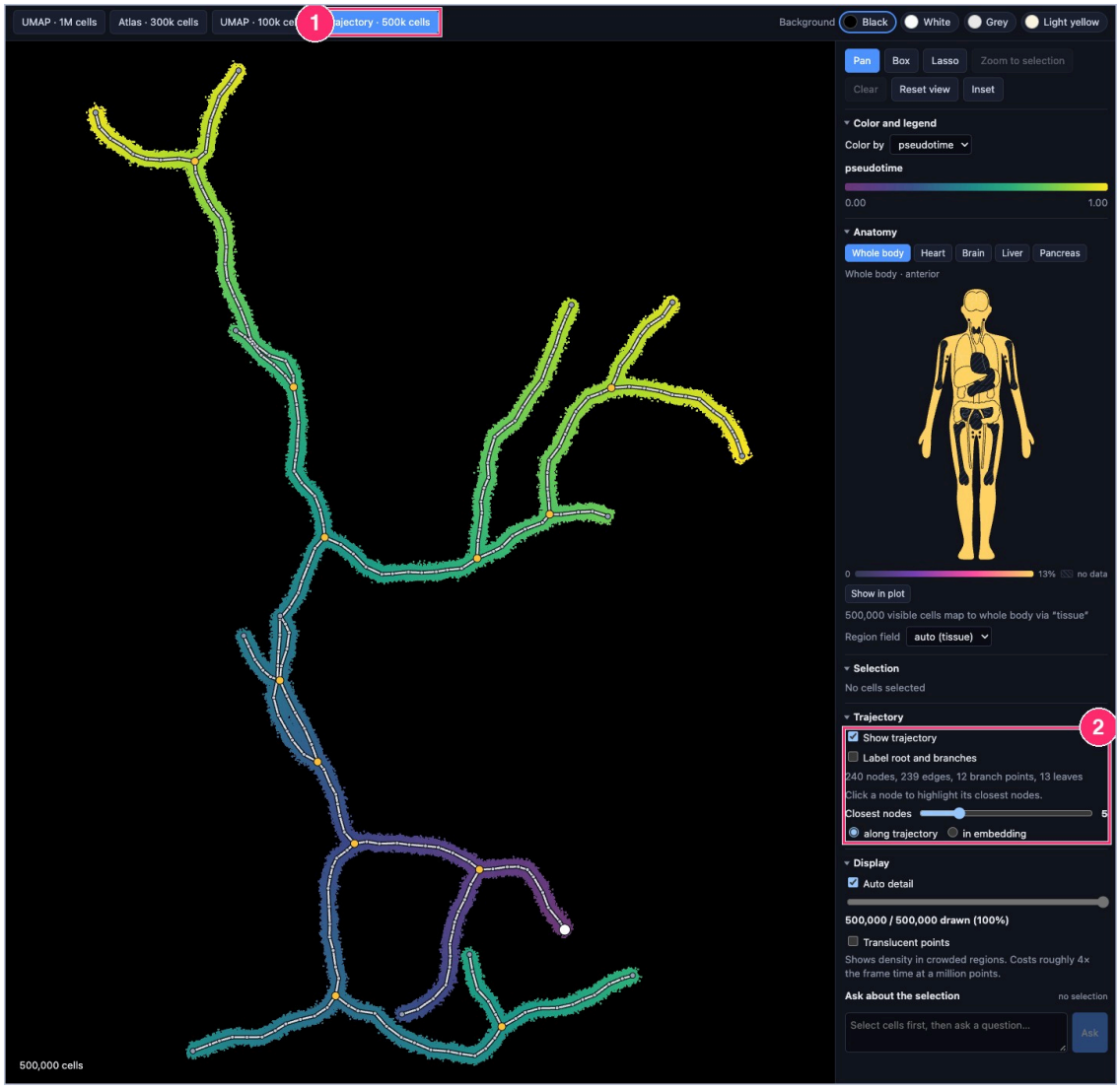
On a black plot, a faded cell is a dark dot. On a white plot, a faded cell would turn *light*. If the app always faded toward black, the cells that should recede would become the darkest, loudest marks on a white plot. So the fading color always follows the background.

Theme	Plot background	Use it for
Black	#000000	Default. Bright colors pop. Best for dark rooms.
White	#ffffff	Papers, slides and printing.
Grey	#ebebeb	Soft contrast. Easy on the eyes for long sessions.
Light yellow	#fdf6e3	A warm "solarized" look.

CHAPTER 7

Trajectories and the principal graph

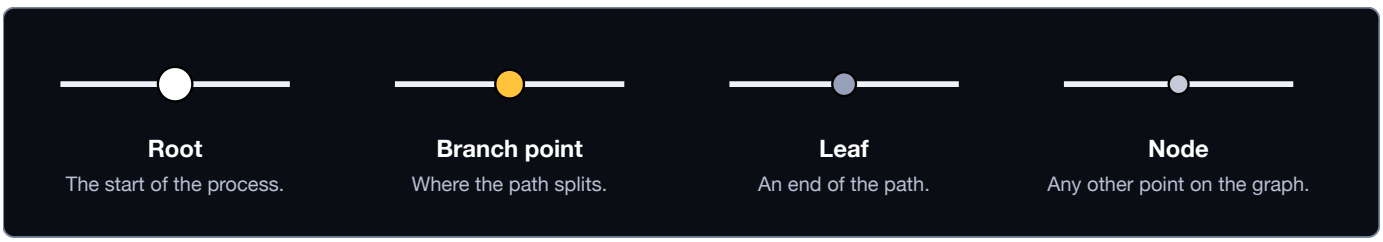
A **trajectory dataset** is a normal embedding with two extras: a number called **pseudotime** for every cell, and a **principal graph**, which is a thin tree that runs through the cloud of cells. Open one with the **Trajectory** button in the dataset switcher.



The trajectory view. The side panel scrolls down to the Trajectory section.

1. **Trajectory button.** Opens the trajectory dataset.
2. **Trajectory controls.** Show or hide the graph, add labels, choose how many neighbors to highlight.

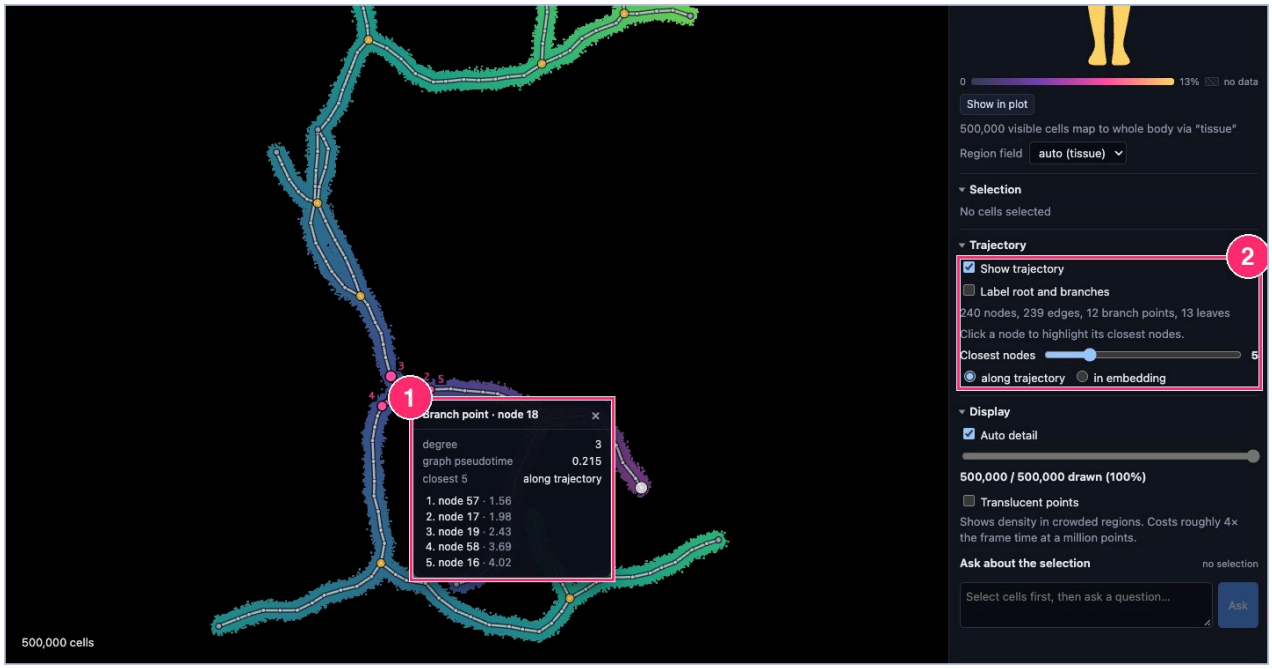
What the markers mean



The four kinds of graph node.

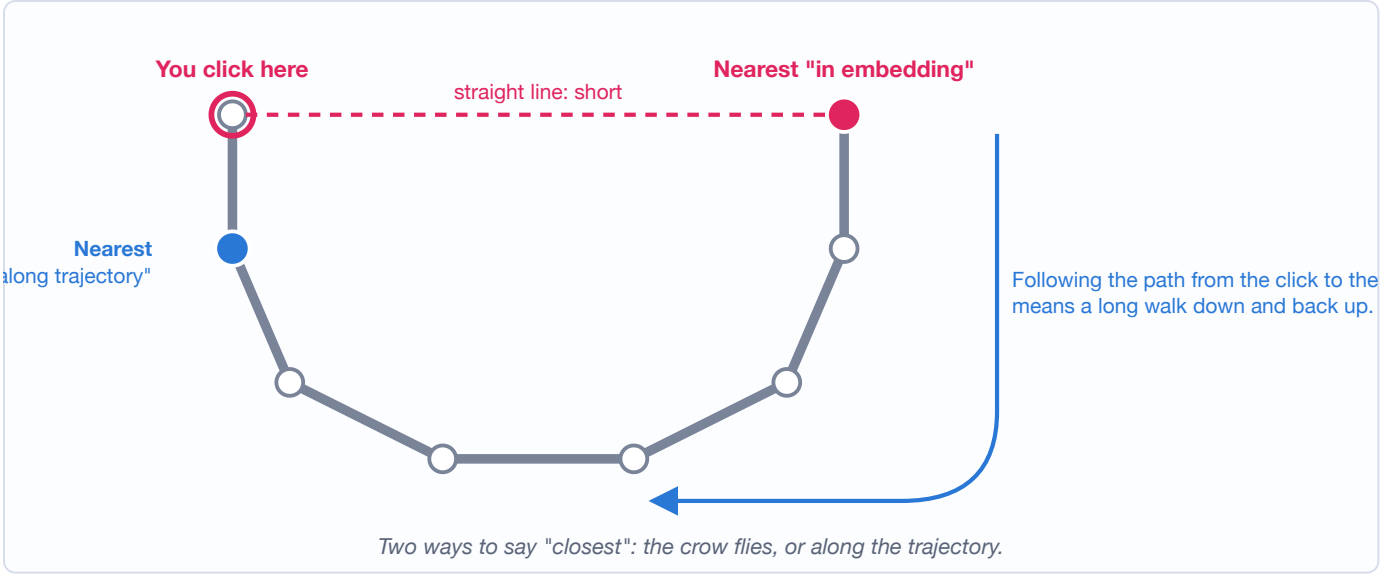
Click a node to find its neighbors

1. Click any node on the white graph. A larger ring marks it.
2. The **closest nodes** light up in pink, numbered 1, 2, 3... by how near they are. A card lists them with their distances.
3. Use the **Closest nodes** slider (1 to 20) to change how many are shown.
4. Choose how distance is measured: **along trajectory** or **in embedding**.
5. Click empty space to clear the highlight.



A branch point selected, with its five closest nodes found along the trajectory.

1. **Node card.** Names the node (root, branch point, leaf or node), its degree (how many edges meet there), its pseudotime, and the numbered neighbors with distances.
2. **Controls. Show trajectory** and **Label root and branches** are switches. The slider sets how many neighbors.



Two ways to say "closest": the crow flies, or along the trajectory.

"Along trajectory" adds up the lengths of the graph edges. "In embedding" measures a straight line on the map. Both answers are useful; pick the one that matches your question.



The same node with ten neighbors measured in the embedding.

1. **Ten neighbors, straight-line distance.** The card now says "in embedding". Connector lines join the neighbors to the clicked node.

> **Tiny nodes are still easy to hit**
>
> Internal nodes are drawn only 2.4 pixels wide, which is hard to click. The app accepts a click within 10 pixels of any node, and within the marker size plus 4 pixels for bigger markers.

Labels

Tick **Label root and branches** to write "root" and branch names next to those nodes. The app places each label so it does not sit on top of another label or on the neighbors you have highlighted.

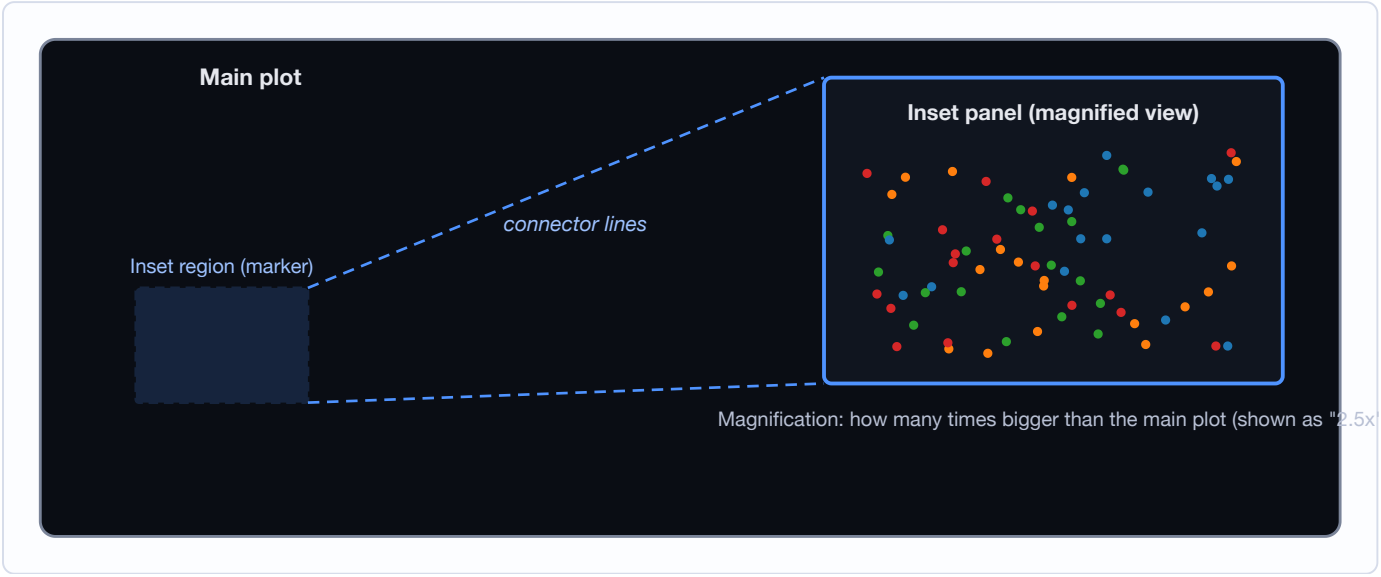


Labels switched on.

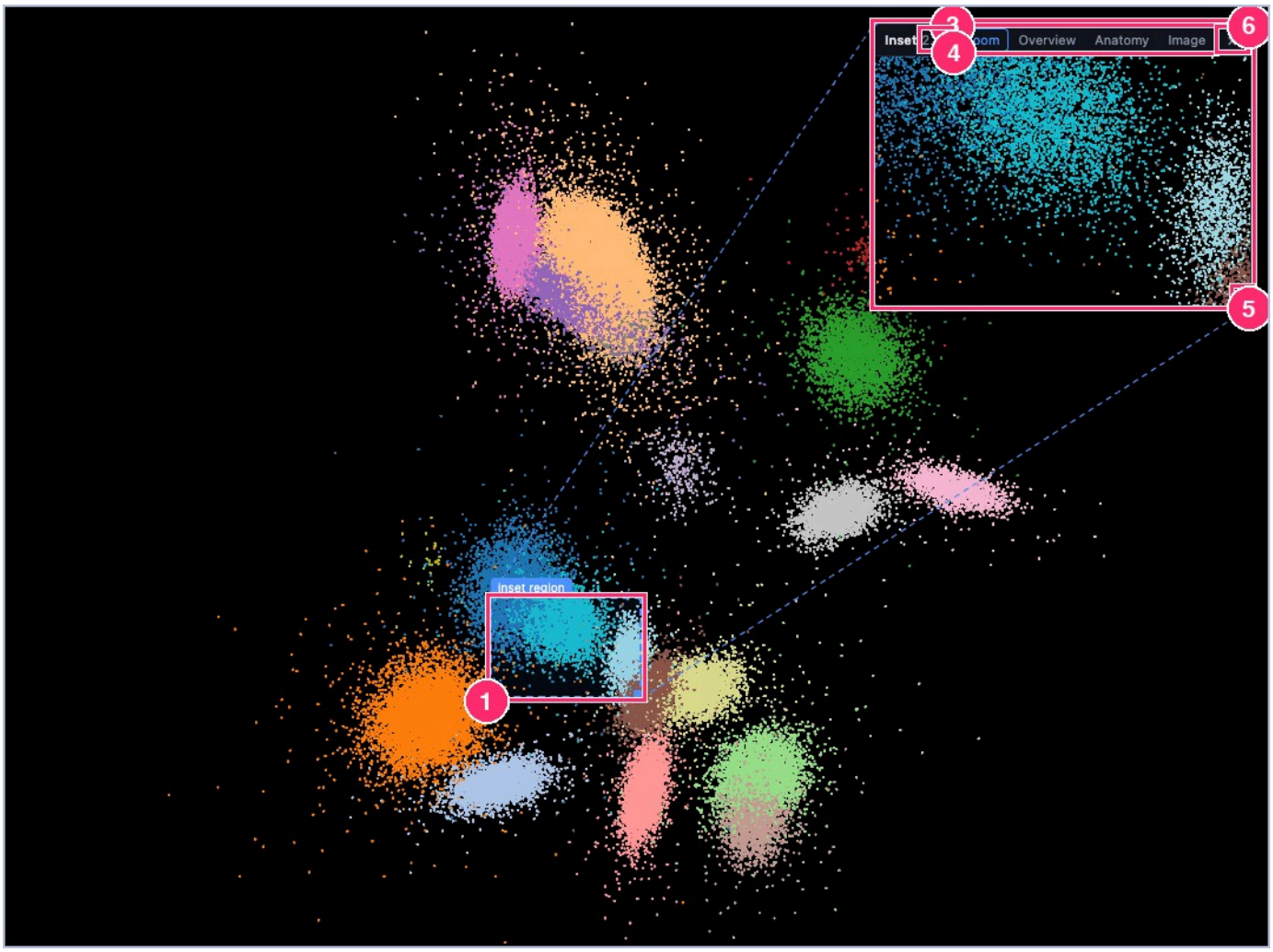
CHAPTER 8

The inset: a second, magnified view

Click **Inset** in the toolbar. A floating panel opens over the plot. It has four modes, chosen with the small buttons in its header.



The inset idea, borrowed from scientific figures: a small box on the big picture, a magnified copy, and two lines joining them.

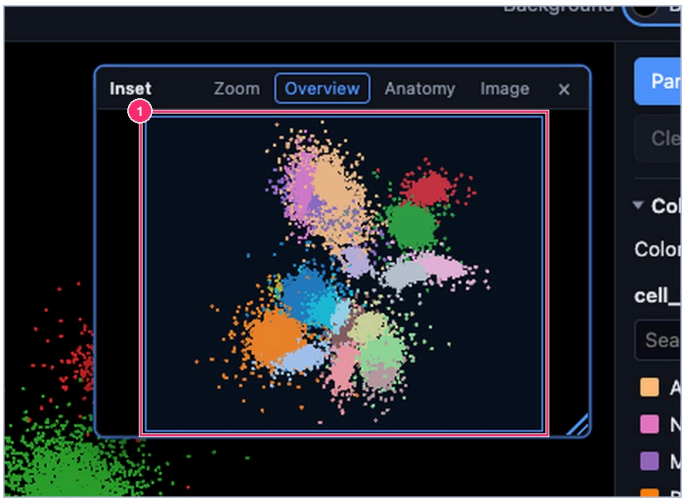


Zoom mode. The panel shows the dashed region magnified, with two connector lines.

1. **Inset region.** The dashed box on the main plot. Drag its label bar to move it. Drag the square corner to resize it.
2. **Inset panel.** Shows the area inside the dashed box, magnified. Drag its header to move the whole panel.
3. **Mode buttons. Zoom**, **Overview**, **Anatomy**, **Image**.
4. **Magnification.** How many times bigger the inset is than the main plot.
5. **Panel resize grip.** Drag to make the panel bigger or smaller (minimum 200 by 150 pixels).
6. **Close.** Closes the panel, the box and the lines.

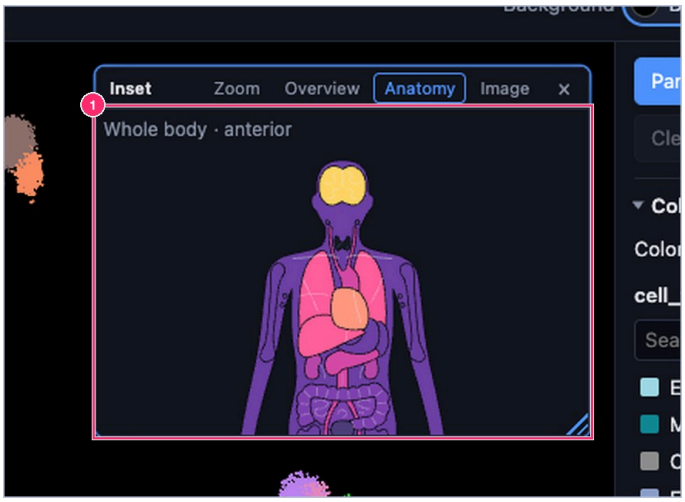
The four modes

Mode	What it shows	Good for
Zoom	The area inside the dashed box, magnified. The inset draws four chunks of cells (up to 1,000,000), so it is as dense as the main view.	Looking at a crowded island without losing your place.
Overview	The whole dataset, with a rectangle showing what the main plot currently shows. Click or drag inside it to move the main view.	Staying oriented while zoomed in far.
Anatomy	The organ drawing (Chapter 9) inside the panel.	Keeping the body picture in view over the plot.
Image	A picture of your own. Click to pick a file, or drop one on the panel. It stays in your browser and is never uploaded.	A tissue photo or a figure next to your cells.



Overview mode.

1. **Viewport outline.** The part of the dataset the main plot shows right now.



Anatomy mode (on the atlas dataset).

1. **Organ figure.** The same figure as in the side panel.

The inset is cheap

The magnified picture is a *second camera* looking at the *same* data already on the graphics card. It is not a second copy. That is why opening the inset hardly changes the frame rate.

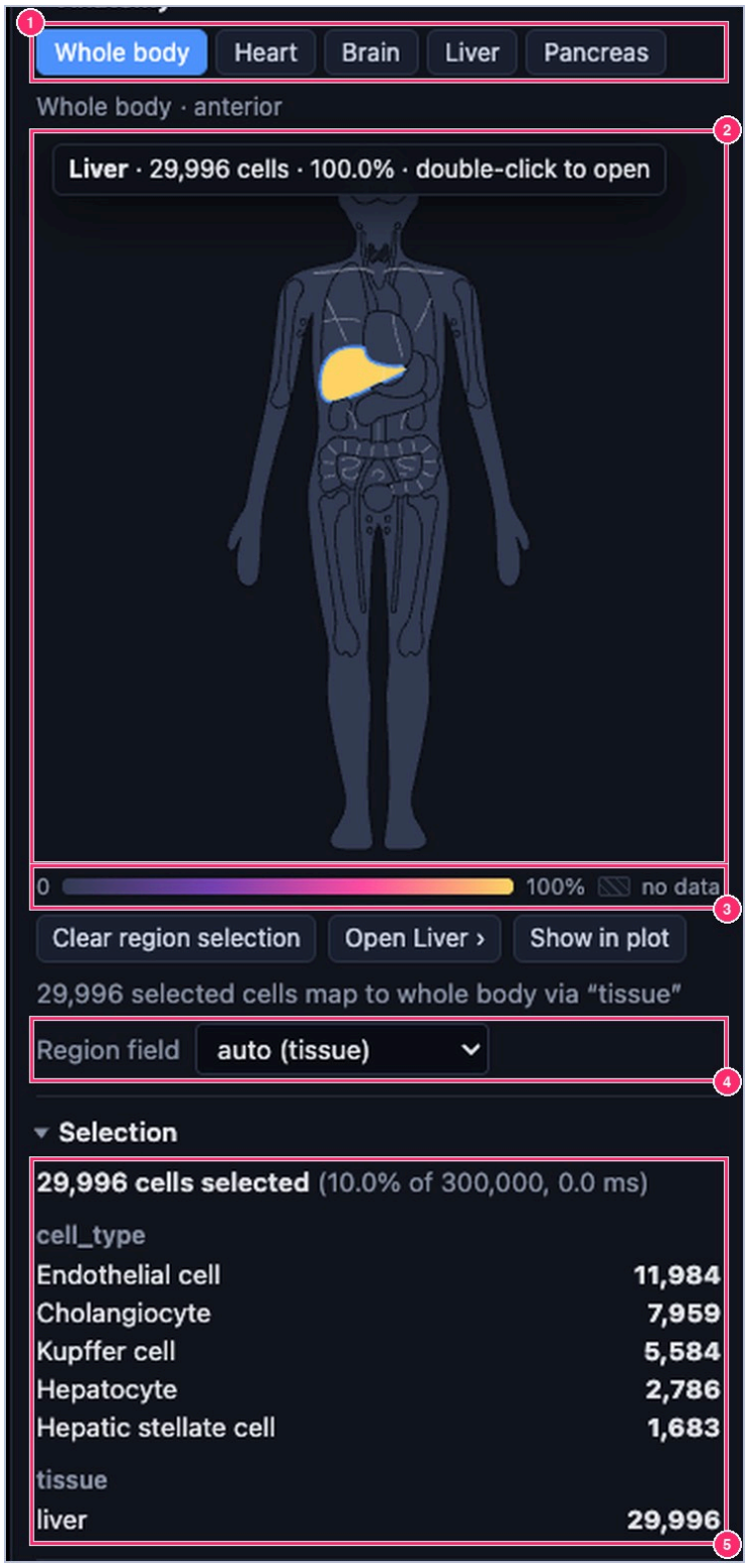
It remembers

The mode and the panel position and size are saved in your browser (key `cellviewer.inset`), so the inset reopens the way you left it.

CHAPTER 9

Anatomy: link cells to the body

If your cells come from different organs, the **Anatomy panel** shows *where*. It draws a body (or one organ) split into named regions. Each region is shaded by the share of cells that come from it. It also works in reverse: **click a region to select its cells.**



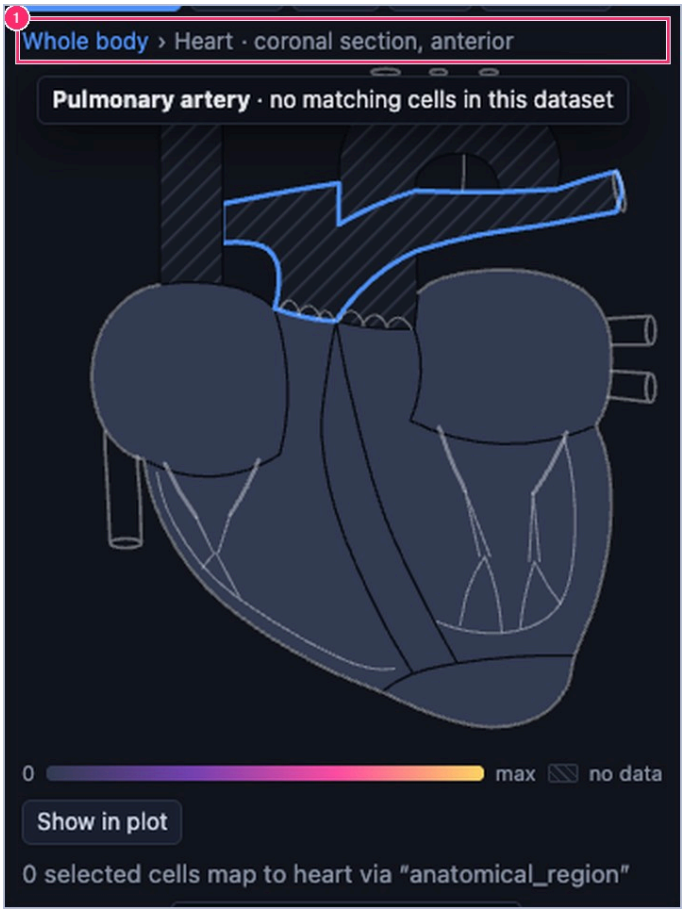
The side panel of the atlas dataset after clicking the liver.

1. **Tabs.** Whole body, Heart, Brain, Liver, Pancreas.
2. **Figure.** Each region is a clickable shape. Darker purple means fewer cells. Pink to yellow means more.
3. **Shading scale.** The bar runs from 0 to the biggest share. The hatched square means "no data".
4. **Region field.** Which annotation field maps to the regions. "auto" lets the app choose.

5 **Selection summary.** After clicking the liver, only liver cells are selected.

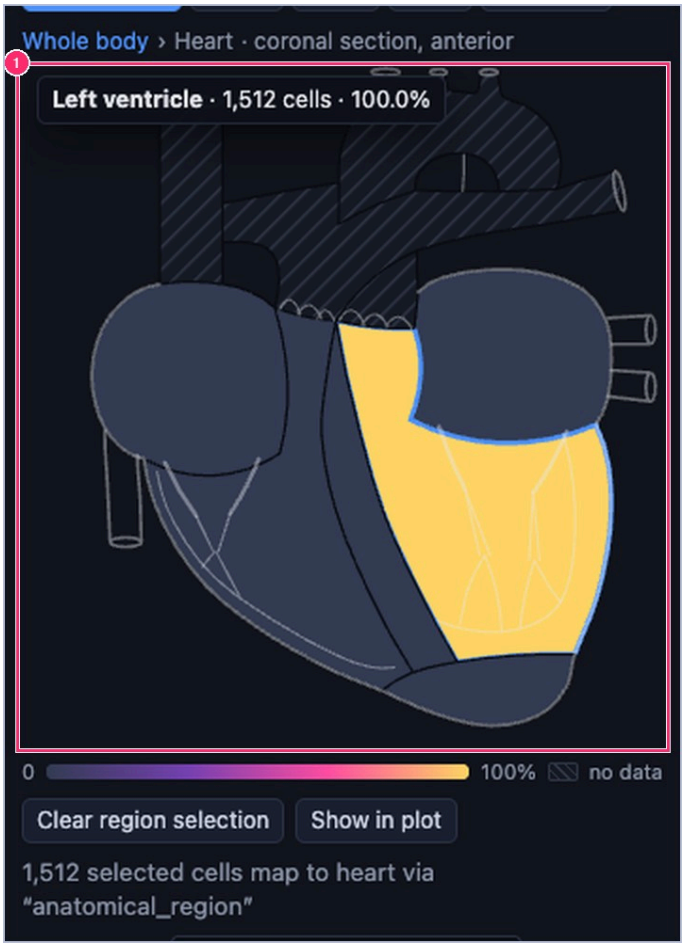
Using it

Do this	Result
Click a region	Selects exactly the cells that belong to it.
`Shift` (or Ctrl, or Cmd) + click another region	Adds that region to the selection, or removes it if it was already in.
Double-click the heart, brain, liver or pancreas on the body	Opens that organ in detail. A breadcrumb ("Whole body › Heart") leads back.
Choose another **Region field**	Re-maps the figure using a different annotation.
Press **Show in plot**	Opens a copy of the figure in the inset over the plot.
Press **Clear region selection**	Drops a selection you made by clicking regions.
Draw a box or lasso on the plot	The figure re-shades to show where *those* cells come from.
Hover over a region	A tooltip names the region and its cell count, or says "no matching cells in this dataset".



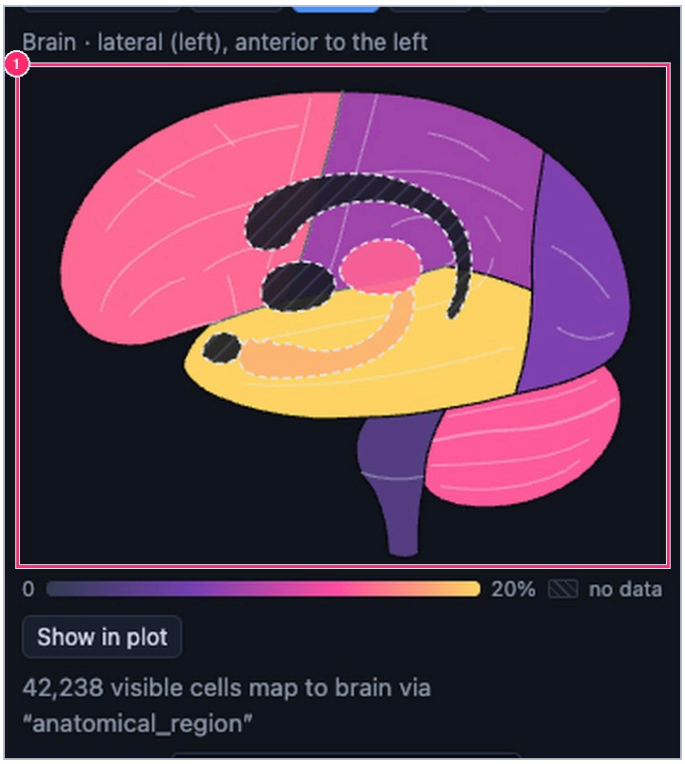
Double-click on the heart opens the heart.

1 **Breadcrumb.** Click "Whole body" to go back.



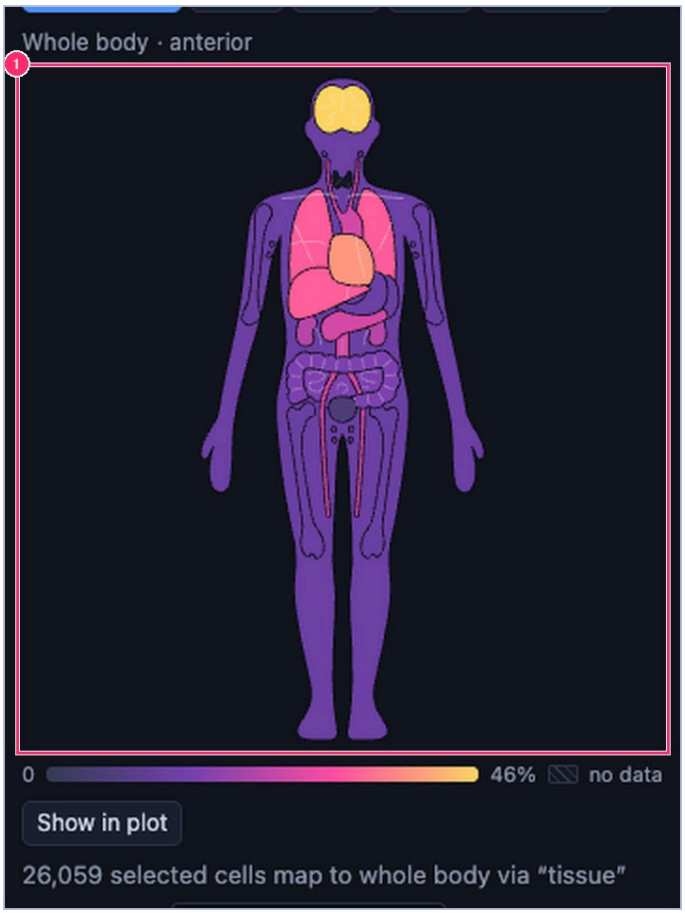
Clicking a heart chamber.

1 **Left ventricle.** Clicked: its cells are selected.



The brain figure.

1 **Brain.** Lobes, plus dashed deep structures such as the hippocampus.



Selection drives the shading.

1 **Re-shaded body.** After a box selection on the plot, regions show where the selected cells come from.

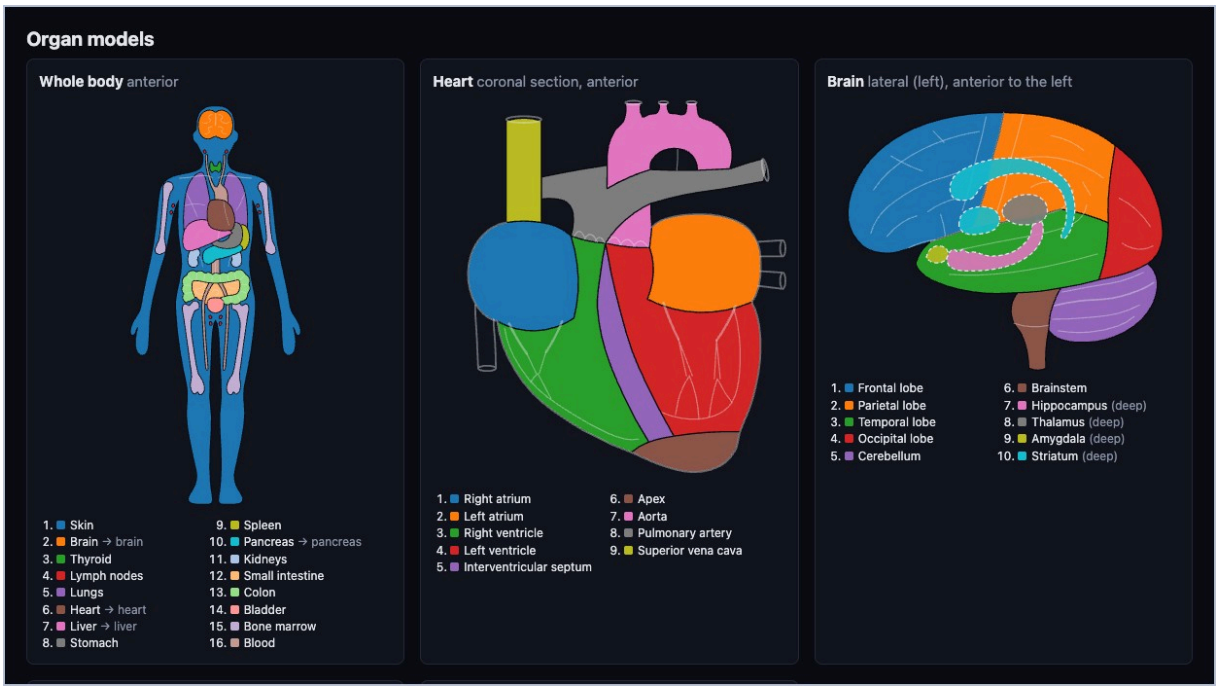
How does the app know which cells belong to a region?

It matches **names**. Every region has a list of names it answers to (its *aliases*). The app looks at the levels of each field in your data, such as `left ventricle` or `Heart: LV`, and finds the field whose levels match the most regions. Matching ignores capital letters, punctuation and accents. It also understands labels with a prefix like `liver: caudate lobe` and simple plurals like `kidneys`.

> **Your data needs matching names**
>
> If a region stays hatched ("no data"), your annotations probably use names the model does not know. You can add names in the organ file (Chapter 29) without touching any other code.

The organ gallery page

Open `http://localhost:5173/anatomy.html` to see every organ model with each region in its own color. It is the quickest way to check a new or edited organ.

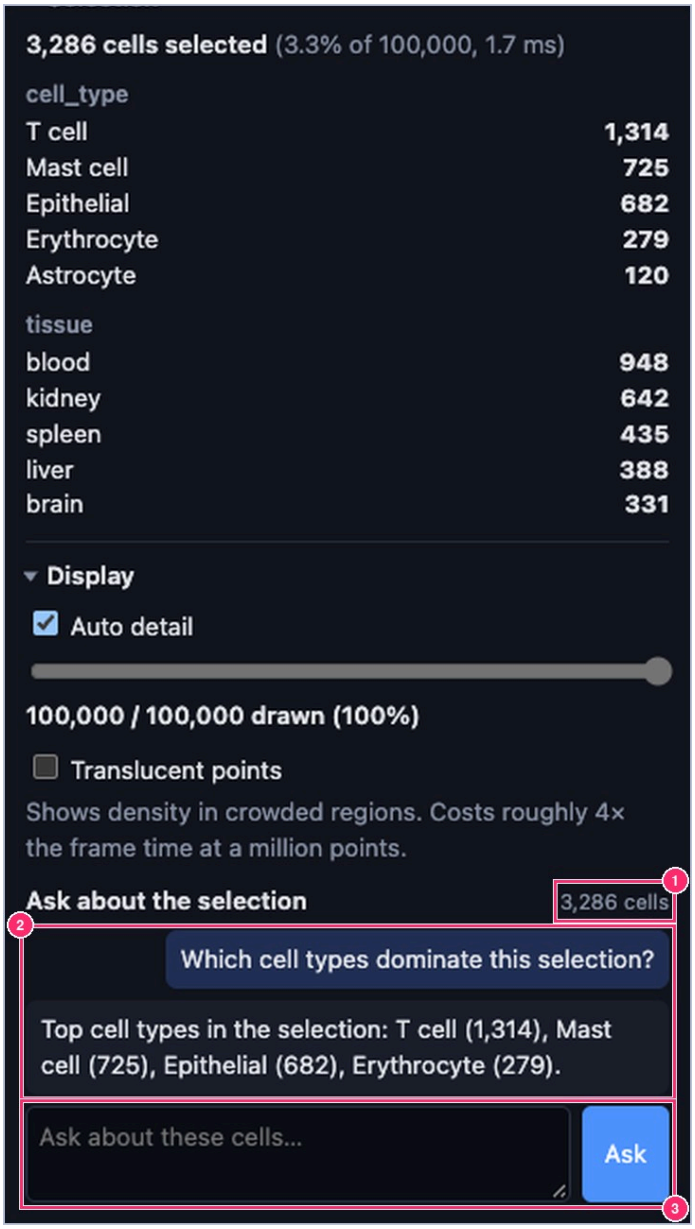


The gallery shows one card per organ.

CHAPTER 10

Chat: ask about the selected cells

The chat panel sits at the bottom of the side panel. Select some cells, type a question such as "Which cell types dominate this selection?", and press **Ask** (or `Enter`; use `Shift`+`Enter` for a new line).



A question and an answer. Here the offline helper answered, so its text starts with a note saying no model is connected.

1. **Cell count badge.** How many cells the question is about. Hover to see which adapter answers.
2. **Conversation.** Your questions on the right (blue), answers on the left.
3. **Question box.** Disabled until you select cells.

Two answerers

	Offline helper (default)	Claude (needs a key)
Needs	Nothing	An Anthropic API key in `.env` on the server
How it answers	Looks up numbers in the selection summary (counts, top cell types, pseudotime range). It says it is not a model.	A real AI model reads the summary and your question and writes an answer.
Where it runs	In your browser	On the server, which calls the API
Good for	Trying the app, working offline	Real questions

What exactly is sent?

> **Never raw rows**
>
> The model never receives the cells themselves. It receives a short **summary**: how many cells, the bounding box and centre, the biggest groups in each field with counts, the minimum, mean, quartiles and maximum of numeric fields, a few example ids, which field is used for color, and which categories the legend has hidden. The summary is limited to 32,000 bytes. That is why asking about 500,000 cells costs about the same as asking about 12.

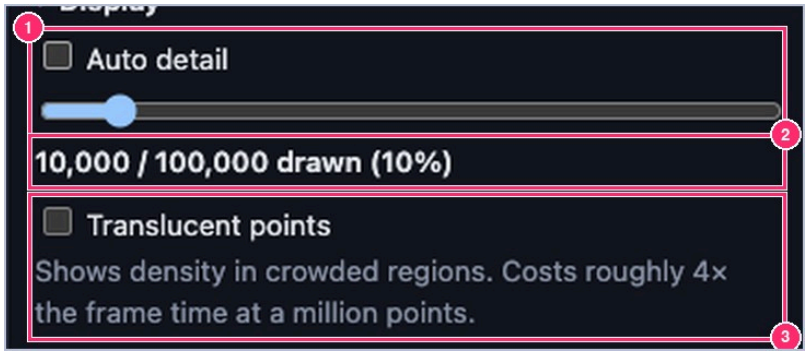
The system prompt tells the model to answer only from that summary and to say so when a question would need gene-level data it does not have. You can read it and change it in `server/chat.py` (Chapter 25).

> **Your key is safe**
>
> The browser only ever talks to `/api/chat` on your own server. The key is read by the server from its environment and is never sent to the page, stored in it, or bundled into it.

CHAPTER 11

Display options and speed

Drawing millions of dots is hard work, even for a GPU. Two controls in the **Display** section let you trade detail against smoothness.



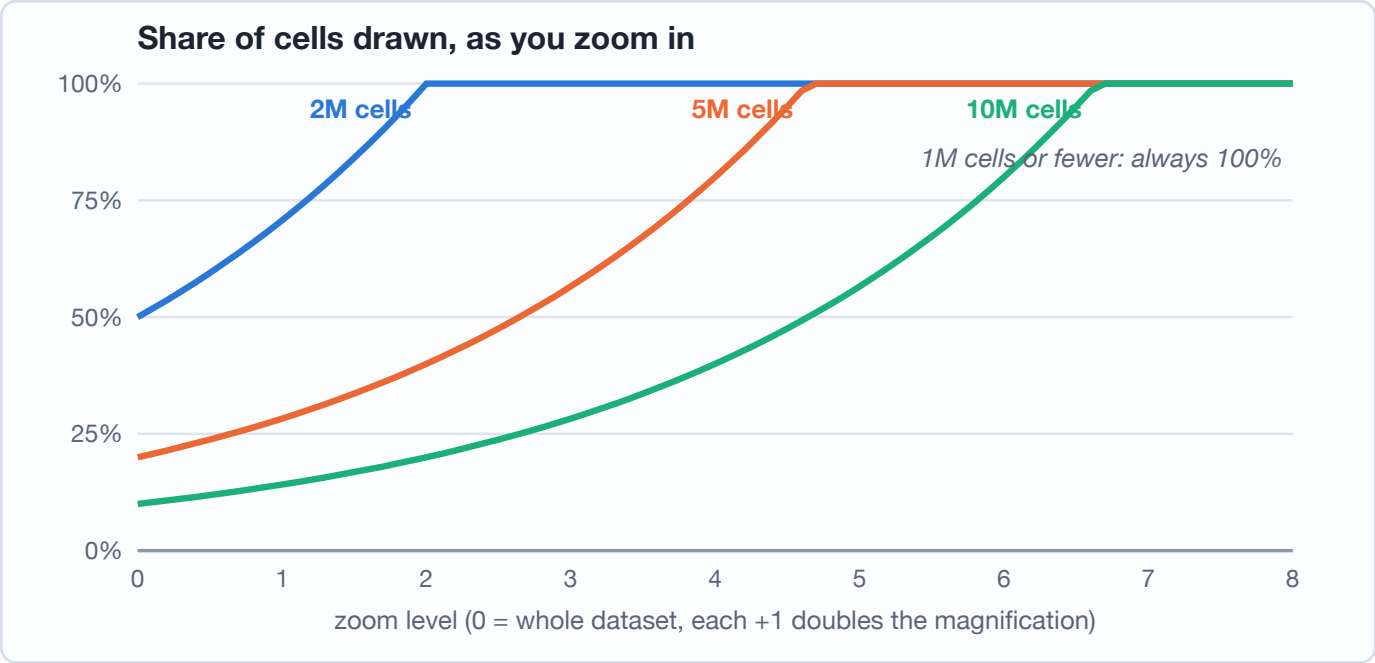
The Display section. Here Auto detail is off and the slider is at 10 percent.

1. **Auto detail.** On: the app chooses how many cells to draw. Off: you set the fraction with the slider.
2. **Readout.** How many cells are drawn out of how many are loaded, with the percentage.
3. **Translucent points.** Draws dots slightly see-through so crowded areas look denser. Costs about four times the frame time at a million points.

How level of detail works

The server saves cells in a **random order** (see Chapter 12). So the first 100,000 cells of a 1,000,000-cell dataset are a fair random sample of all of them. To draw less, the app simply draws only the first part of the list. There is no filtering and no extra memory.

As you zoom in, fewer cells fit on the screen, so the app can afford to draw a bigger share of them. The chart shows the share drawn for four dataset sizes with the default budget of 1,000,000 points.

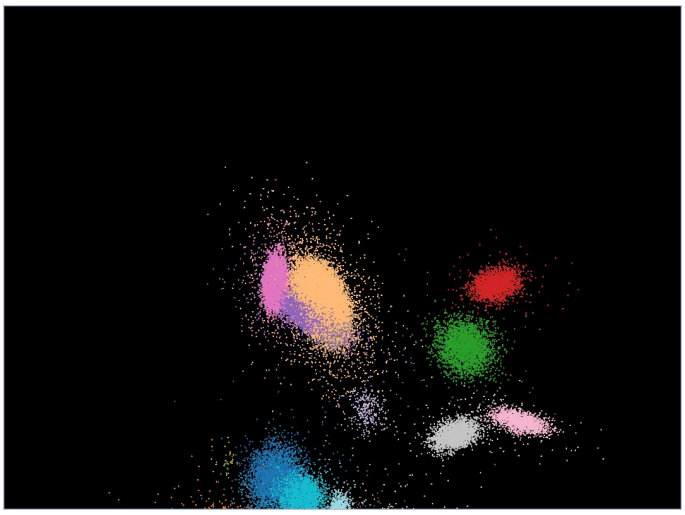


The share grows with the square root of the magnification: `fraction = budget x 2^(zoom/2) / cells`*, capped at 100 percent. The formula lives in* `packages/core/src/render/lod.ts`*.*

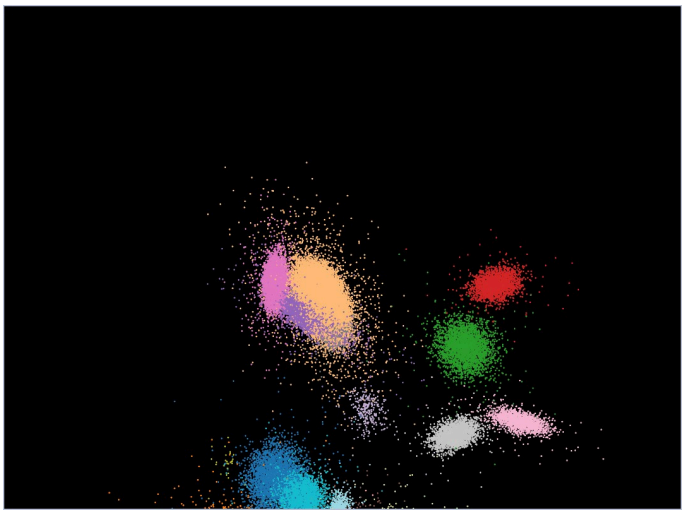
> **Selection and chat always use every cell**
> Level of detail only changes what is *painted*. When you select, count or ask a question, the app uses all the cells that are loaded, not just the drawn sample.

Translucent points

Normally every dot is solid (opaque). That is the fastest way to draw. If you switch on **Translucent points**, dots blend into each other, so a crowded spot looks denser and a sparse spot looks lighter.



Opaque dots (default).



Translucent points on.

> **Why it is off by default**
> Blending is the single most expensive thing the renderer can do. In a test at one million points, turning it off raised the frame rate from 12 to 38 frames per second. See Chapter 32 for the numbers.

PART 3

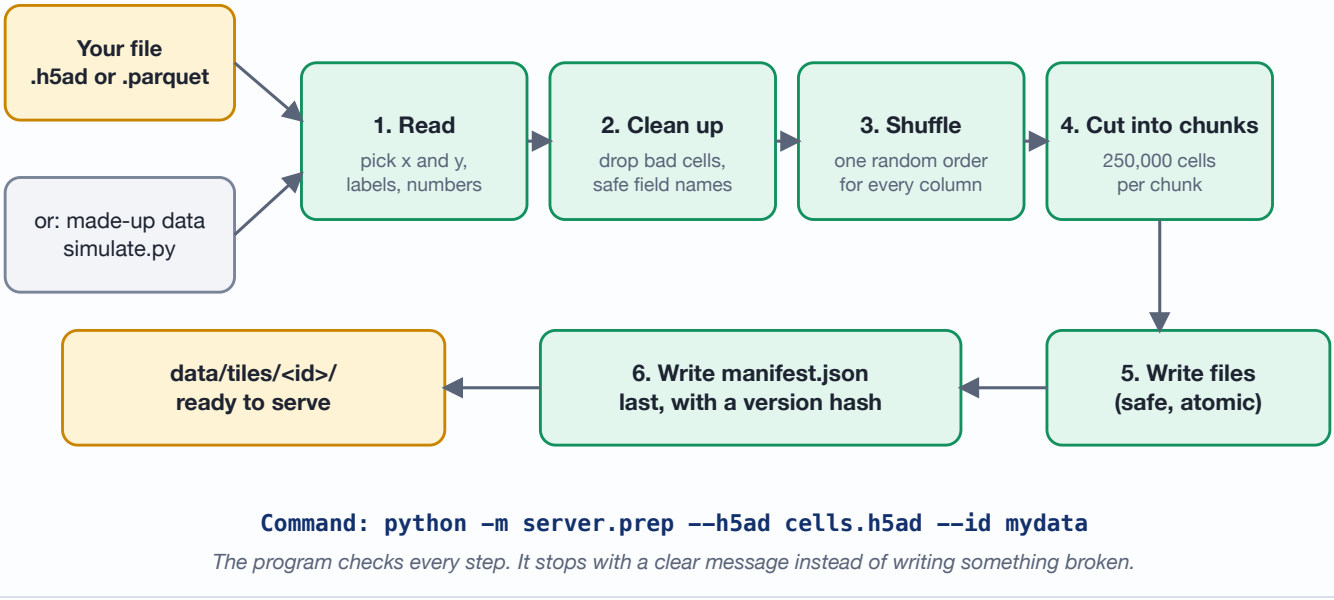
Your own data

How to turn your cells into a dataset, and what a dataset looks like on disk.

CHAPTER 12

Preparing your own data

The viewer does not make the map for you. You bring cells that already have an x and a y (from UMAP, t-SNE or any other method), plus the labels you care about. A small program, **prep**, reads your file and writes it in a format the browser can load very fast.



What the prep program does to your file.

From an .h5ad file (scanpy / AnnData)

```
.venv/bin/python -m server.prep --h5ad path/to/cells.h5ad --id mydata --embedding X_umap
```

Part	Meaning
`--id mydata`	The name of the new dataset. Letters, digits, `_` and `-` only, up to 64 characters.
`--embedding X_umap`	Which key of `adata.obsm` holds the map. It must have exactly two columns. `X_umap` and `X_tsne` are the usual ones. A 50-column `X_pca` is refused with a message listing the keys you do have.
Categorical columns of `adata.obs`	Become color-by fields automatically.
Numeric columns	Become number fields. A column called `pseudotime` is what the trajectory view colors by.
`adata.obs_names`	Kept as the cell names, together with each cell's row number. A clicked cell shows both, so you can find it in your AnnData again.

From a Parquet file

```
.venv/bin/python -m server.prep --parquet path/to/cells.parquet --id mydata
```

The file needs columns called `x` and `y`. A column called `cell_id`, `barcode` or `obs_names` (the first one present, in that order) gives the cell names and is not treated as a field. Name another one with `--id-column`. Every other column is sorted into categorical or numeric by its type.

Choosing columns

```
--obs-fields cell_type tissue donor      # categorical columns to keep
--numeric-fields pseudotime percent_mt     # numeric columns to keep
```

Without these options the program keeps everything it can. A text column with more than 65,535 different values (such as a unique join id) is skipped with a warning, because the app stores each cell's level as a 16-bit number. If you *name* such a column yourself, the program stops with an error instead of quietly storing wrong colors.

What prep fixes for you

Problem in your file	What prep does
Field name is not safe in a web address (`percent.mt`, `cell type`)	Renames it (`percent_mt`, `cell_type`), adds `_2` on a clash, and prints the renames.
A cell has a missing or infinite x or y	Drops that cell and prints how many.
A number field has missing values	Keeps them. Color ranges use only the real values.
Too many levels for 16 bits	Skips the column (auto) or stops with an error (named).
You prepare the same dataset id again	Overwrites it, removes files it no longer needs, and gives it a new `version` so browsers never mix old and new chunks.
The server is reading while prep writes	Every file is written under a temporary name and renamed, so the server sees an old file or a new one, never half of one.

Then open it

The server lists whatever is in `data/tiles/`, so you do not restart anything. Reload the page. A new button appears in the dataset switcher. Or go straight to `http://localhost:5173/?dataset=mydata`.

Trajectory data

To show a principal graph, put a file called `graph.json` in the dataset folder and set `"hasGraph": true` in `manifest.json`.

```
{
  "nodes": [[x, y], [x, y], ...],     # position of every node
  "edges": [[0, 1], [1, 2], ...],     # pairs of node numbers (any order)
  "root": 17,                         # node number of the start
  "branchPoints": [17, 42, 88],
  "leaves": [3, 9, 55]
}
```

Every option of the prep program

Option	Default	Meaning
`--out`	`data/tiles`	Where to write the datasets.
`--chunk-size`	250000	Cells per chunk. Smaller chunks fill in more smoothly on a slow connection. Larger ones mean fewer requests.
`--seed`	0	Seed for the shuffle and for the made-up data. Same seed, same result.
`--sizes`	100000 1000000	Sizes of made-up embeddings to create.
`--traj-size`	500000	Size of the made-up trajectory.
`--atlas-size`	0	Size of the made-up multi-organ atlas. 0 skips it.
`--h5ad`, `--parquet`	none	Read your file instead of making data.
`--id`	none	Name for your dataset.
`--embedding`	`X_umap`	Key of `adata.obsm` (h5ad only).
`--obs-fields`, `--numeric-fields`	all	Columns to keep.
`--id-column`	auto	Which Parquet column holds cell names.

CHAPTER 13

What is inside a dataset folder

A prepared dataset is just a folder of small files. You never need to open them, but knowing what they are makes the rest of the guide easier. The format is **deliberately boring**: raw numbers, laid end to end, with no parsing needed.

```
data/tiles/
└─ sim100k/                          one folder per dataset (the folder name is the dataset id)
   ├─ manifest.json                  describes the dataset (see below)
   ├─ graph.json                     trajectory datasets only: the principal graph
   ├─ xy/
   │   ├─ 000.bin                    x,y,x,y,... as 32-bit decimals, 250,000 cells per file
   │   └─ 001.bin ...
   ├─ codes/
   │   ├─ cell_type/ 000.bin ...     one 16-bit whole number per cell: the level number
   │   ├─ tissue/    000.bin ...
   │   └─ ...
   ├─ num/
   │   └─ pseudotime/ 000.bin ...    one 32-bit decimal per cell
   └─ cells/
       ├─ rows.npy                   each cell's row in your original file
       └─ ids.npy                    each cell's name (barcode), if your file had one
```

File	Holds	Type	Bytes per cell
`xy/NNN.bin`	x and y of each cell	`float32` pairs	8
`codes/<field>/NNN.bin`	The level number of each cell for one field. Level 0 is the first name in the manifest list, and so on.	`uint16`	2 per field
`num/<field>/NNN.bin`	One number per cell for one number field	`float32`	4 per field
`cells/rows.npy`, `ids.npy`	Row number and name. Only the server reads these, one cell at a time.	NumPy files	not sent in bulk
`manifest.json`	The dataset description	JSON	-

Why cut a dataset into chunks?

sim1m (1,000,000 cells) is stored as 4 chunks of up to 250,000 cells

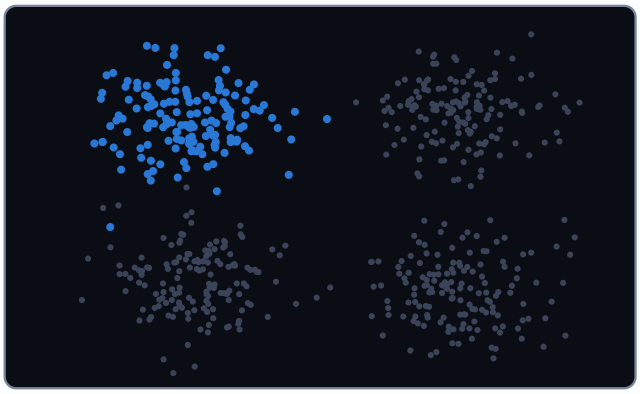
xy/000.bin	xy/001.bin	xy/002.bin	xy/003.bin
codes/cell_type/000.bin	codes/cell_type/001.bin	codes/cell_type/002.bin	codes/cell_type/003.bin
codes/tissue/000.bin	codes/tissue/001.bin	codes/tissue/002.bin	codes/tissue/003.bin
num/pseudotime/000.bin	num/pseudotime/001.bin	num/pseudotime/002.bin	num/pseudotime/003.bin

The browser asks for six files at a time, in order. The plot starts filling from the first chunk.

Every column is cut the same way, so chunk number 2 of every file covers the same 250,000 cells.

- The browser downloads chunks **in parallel** (six at a time) and keeps retrying a chunk that fails.
- A chunk goes to the GPU **once**, when it arrives. Later chunks never make the app re-upload earlier ones.
- Files are named with three digits (`000.bin`), so they sort in order.

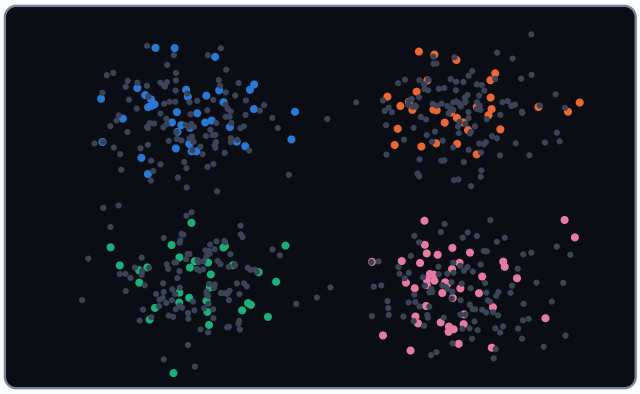
Sorted by group (not shuffled)



The first quarter of the list is all one group.

Bright dots = the first 25 percent of the list

Shuffled (what prep writes)



The first quarter is a fair sample of every group.

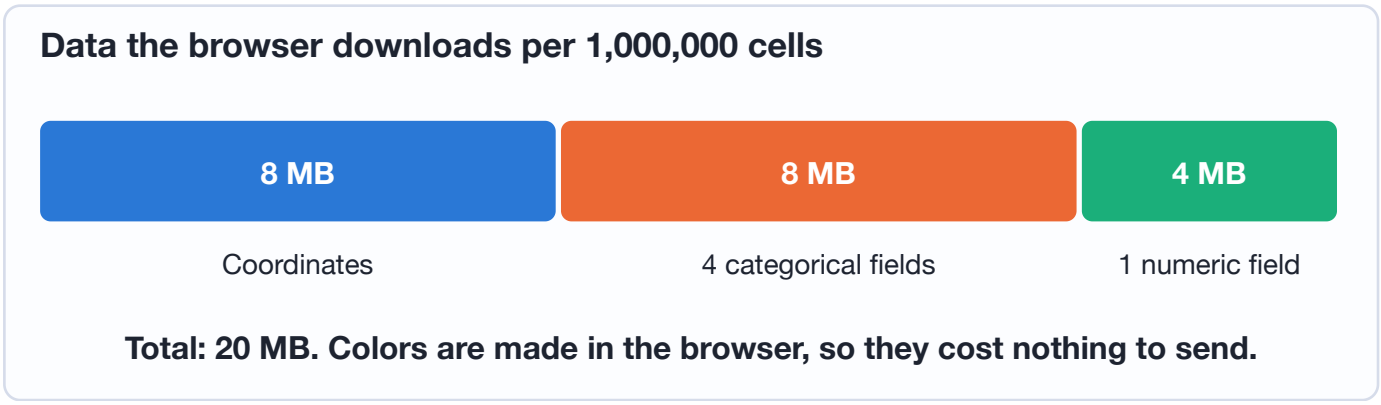
Bright dots = the first 25 percent of the list

Why the shuffle matters. To draw less, the app just draws the first part of the list. That only gives a fair picture if the list is in random order.

The manifest, line by line

```
{
  "datasetId": "sim100k",          <- the folder name
  "n": 100000,                     <- number of cells
  "chunkSize": 250000,             <- cells per chunk
  "chunks": 1,                     <- how many chunk files per column
  "bounds": [-78.3, -74.9, 65.1, 73.0],   <- x min, y min, x max, y max (with a 2% margin)
  "categorical": {                 <- fields made of named groups
    "cell_type": { "levels": ["T cell", "B cell", ...] },
    "tissue":    { "levels": ["lung", "liver", ...] }
  },
  "numeric": {                     <- fields made of numbers
    "pseudotime": { "min": 0.0, "max": 1.0 }
  },
  "hasGraph": false,               <- true if graph.json exists
  "cellIds": true,                 <- true if cells/ids.npy exists
  "version": "4db6daabaf2fe935"    <- changes whenever any byte changes
}
```

How big is a dataset?



Four category fields and one number field. Each extra category field adds 2 MB per million cells. Each extra number field adds 4 MB.

What is the version for?

The `version` is a fingerprint (the first 16 characters of a SHA-256 hash) of every byte in the dataset. The browser adds it to every chunk address (`...?v=4db6daab...`). Because the address changes whenever the data changes, the server can tell the browser to keep each chunk **for a year** without risk. Prepare the same data again and the version stays the same, so caches stay warm.

PART 4

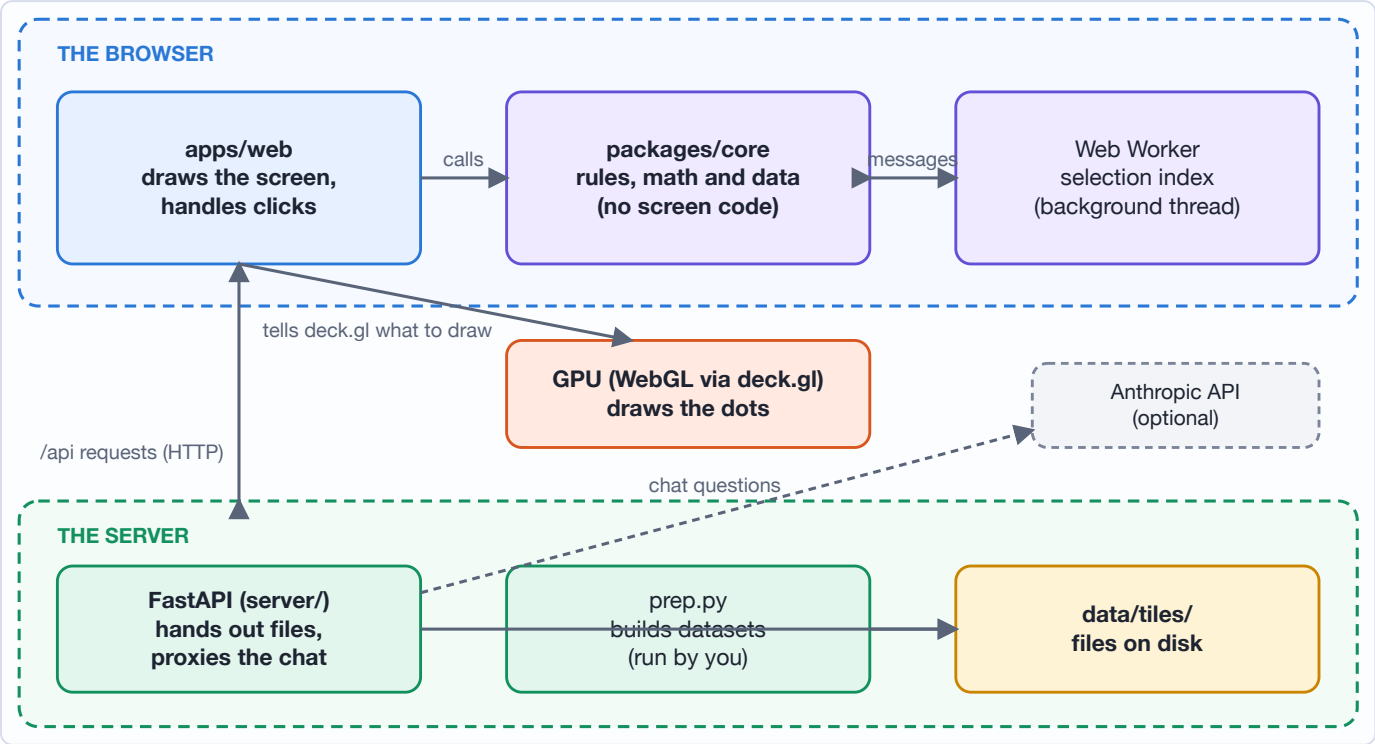
How the code works

The design, in order: from the big picture down to each feature.

CHAPTER 14

The big picture

Cell Viewer has three parts. Knowing which part does what is most of the work of understanding the code.



Everything in one picture. Solid boxes are always there. The dashed box is optional.

Part	Language	Folder	Job in one sentence
web The app	TypeScript	`apps/web`	Builds the page, listens to the mouse, and tells the GPU what to draw.
core The logic	TypeScript	`packages/core`	Holds the rules and math: loading, selecting, filtering, coloring, themes, anatomy. It does not know the screen exists.
server The data server	Python	`server`	Prepares datasets, serves them as files, and forwards chat questions.

Why split web and core?

> **The most important rule in the project**
>
> `packages/core` **may not touch the browser screen or deck.gl.** It only works with numbers and typed arrays. That keeps the hard thinking (which cells are inside this lasso?) easy to read, easy to test in plain Node, and impossible to break by changing a button. `apps/web` is the only place that draws or listens.

The tools used

Tool	What it is	Where you see it
TypeScript	JavaScript with types. The compiler catches mistakes before the app runs.	All `.ts` files. Check with `npm run typecheck`.
Vite	A fast dev server and bundler.	`vite.config.ts`, `npm run dev`, `npm run build`
deck.gl	Draws large amounts of data on the GPU.	`ScatterCanvas.ts`, `graphLayers.ts`
FastAPI + uvicorn	A Python web framework and its server.	`server/main.py`
NumPy	Fast number arrays in Python.	`server/prep.py`, `simulate.py`
pyarrow / anndata	Read Parquet and .h5ad files.	`server/prep.py`

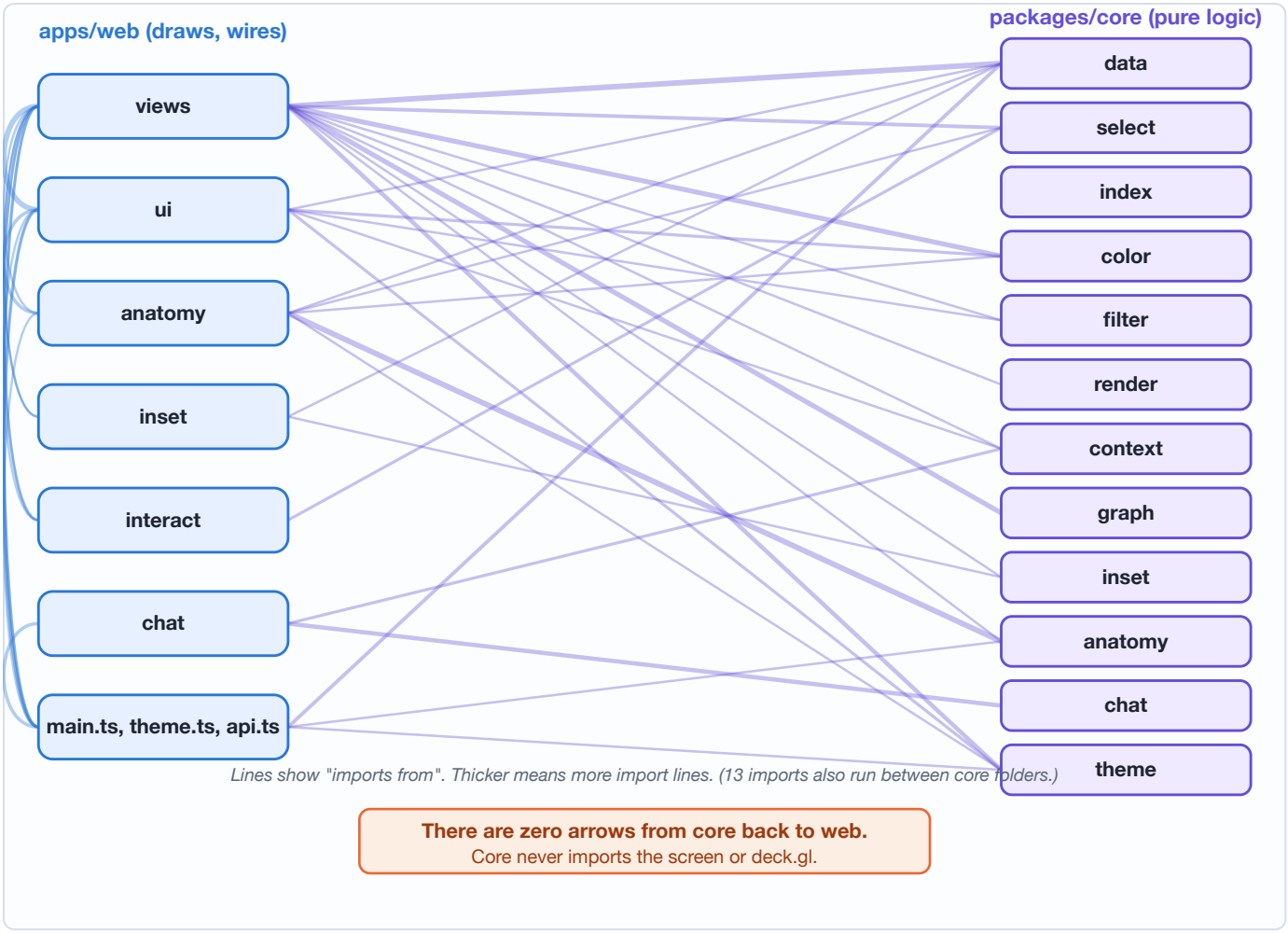
CHAPTER 15

The folder map

```
cell-viewer-master/
├─ apps/web/                        the browser app
│   ├─ index.html, anatomy.html     the two pages (each loads one script)
│   └─ src/
│       ├─ main.ts                  start-up: find datasets, open one
│       ├─ theme.ts, api.ts         theme storage / where the API lives
│       ├─ style.css                all styling, themes via CSS variables
│       ├─ views/                   ScatterCanvas, ScatterView, TrajectoryView, graphLayers
│       ├─ ui/                      legend, toolbar, info card, pickers, controls
│       ├─ anatomy/                 organ panel, figure renderer, gallery page
│       ├─ inset/                   the floating inset panel
│       ├─ interact/                lasso/box drawing, click detection
│       └─ chat/                    chat panel, Claude adapter
├─ packages/core/src/               pure logic, no screen code
│   ├─ data/  index/  select/  color/  filter/  render/  context/
│   ├─ graph/  inset/  anatomy/  chat/  theme/
├─ server/                          Python: main, tiles, chat, prep, simulate, names
├─ data/tiles/                      generated datasets (not saved in git)
├─ docs/                            this guide
├─ vite.config.ts, tsconfig.json, package.json, pyproject.toml, .env
└─ README.md
```

Who uses whom

The picture below was **measured from the code**: every line is a real `import` statement. Notice that all arrows go from the left (web) to the right (core). That is the rule from the last chapter, and it holds.



Which folder uses which, measured from the real import lines.

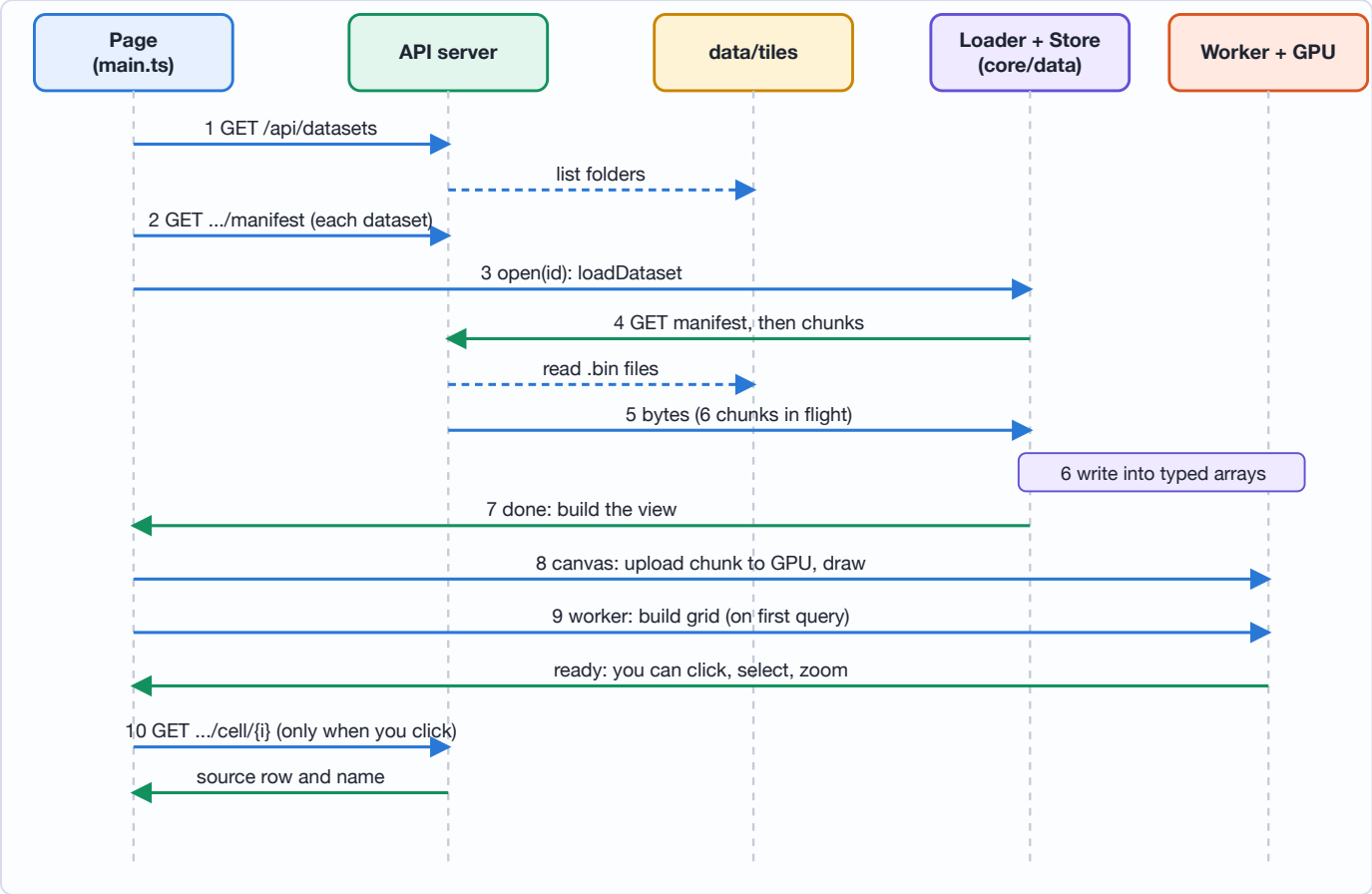
How to find your way

If you want to change...	Start in
How something **looks** (colors, spacing, fonts)	`apps/web/src/style.css`
What a **button or panel** does	The file in `apps/web/src/ui/` with that name
How dots are **drawn**	`apps/web/src/views/ScatterCanvas.ts`
How **selection** works	`packages/core/src/select/`
How **colors** are chosen	`packages/core/src/color/`
What the **themes** are	`packages/core/src/theme/themes.ts` and the top of `style.css`
An **organ drawing**	`packages/core/src/anatomy/models/`
What the **server** serves	`server/tiles.py`
What the **chat** says to the model	`server/chat.py`
How data is **prepared**	`server/prep.py`

CHAPTER 16

Life of a dataset: from click to dots

This chapter follows one dataset from the moment you open the page until dots appear. It touches almost every file in the project, so use it as a map.



What happens between opening the page and seeing dots. Time runs from top to bottom.

The same story in words

1. **Start.** `main.ts` builds the empty page (top bar, stage, status line) and asks the server for the list of datasets (`GET /api/datasets`) and each dataset's manifest. It labels them ("UMAP · 100k cells") from the id and puts them in the dataset switcher. If the server does not answer, it shows the waiting message and retries.

2. **Open.** Clicking a button (or the first load) calls `open(id, kind)`. It cancels any load in progress, destroys the old view, and starts `loadDataset()` from `packages/core/src/data/loader.ts`.

3. **Allocate.** The loader fetches the manifest and creates a `CellStore`: every column is allocated at its **full size** straight away (typed arrays, filled with zeros).

4. **Fetch.** A pool of six workers downloads chunks in order. For each chunk it fetches the coordinate file and one file per field, copies the bytes into the right slot of the right array, and then marks the chunk as loaded. A failed request is retried with growing delays, then once more after a longer pause. A "not found" error is not retried.

5. **Build the view.** When loading finishes, `main.ts` creates a `ScatterView` (or `TrajectoryView`). It builds the side panel, the `ScatterCanvas` (which creates deck.gl), and connects everything.

6. **Draw.** The canvas paints colors into `store.color` and gives deck.gl one layer per chunk. deck.gl uploads the arrays to the GPU once. You see dots.

7. **Wait for you.** The selection worker is not started until the first time you select or click. Then the view sends it the coordinates and it builds the grid index.

> **Where the status line gets its text**
> While loading, the status line says "loading sim100k...". Afterwards it says how many cells are loaded, for example "100,000 cells". If some chunks failed, it says how many cells could not be loaded.

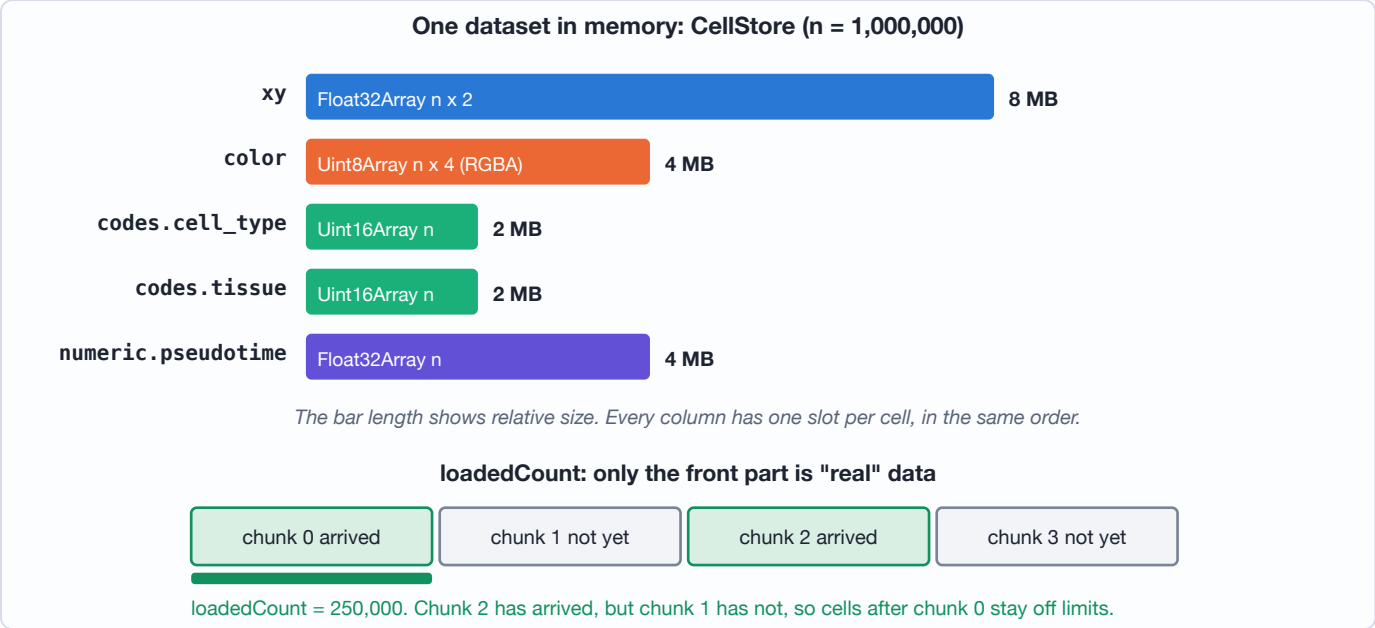
Cancelling

Clicking another dataset while one is loading is safe. The old load gets an `AbortController` signal. Its pending downloads stop, and its store is dropped. A rule of thumb in this codebase: **any async job that can be superseded checks a sequence number or an abort signal before it touches the screen.** You will see this in selection (`selectionSeq`), clicks (`clickSeq`) and node lookups.

CHAPTER 17

Data in memory: the CellStore

Everything about the cells in the browser lives in **one object**: the `CellStore` in `packages/core/src/data/schema.ts`. It is a set of typed arrays side by side, one slot per cell. A cell is not an object. A cell is a **number**: its position in these arrays.



Columns of the CellStore, and why the loaded count only counts a gap-free front part.

Why no cell objects?

Approach	What happens at 1,000,000 cells
An array of 1,000,000 objects like `{x, y, type, tissue...}`	The browser allocates a million little objects and later has to clean them up (garbage collection). That alone costs more time than drawing. It is the main reason many scatter plots stop at 100,000 points.
Typed arrays (what Cell Viewer does)	A few big blocks of memory, allocated once. The GPU can read them directly. Loops over them are very fast.

The columns

Member	Type	Meaning
`xy`	`Float32Array` (n x 2)	x, y, x, y... for each cell.
`codes`	Map: field to `Uint16Array`	The level number of each cell for each categorical field.
`levels`	Map: field to list of names	Turns level numbers back into names: `levels.get("tissue")[3]` is "colon".
`numeric`	Map: field to `Float32Array`	The values of each numeric field.

Member	Type	Meaning
`numericRange`	Map: field to [min, max]	Used for the color ramp.
`color`	`Uint8Array` (n x 4)	Red, green, blue, alpha of each cell. **Rewritten** whenever color-by, the selection, the filter or the theme changes. This is what the GPU reads.
`loadedCount`	number	How many cells at the front are real. See below.
`failedChunks`	list	Chunks that could not be loaded.

Rule 5: only [0, loadedCount) is real

Chunks download in parallel and can finish in any order. If chunk 3 arrives before chunk 1, the array has real numbers at the end and zeros in the middle. Zeros mean "a cell at position (0, 0)". So `loadedCount` only advances when there is **no gap** in front of it. Every loop in the app (drawing, selecting, counting, chat) stops at `loadedCount`. Anything beyond it is unwritten.

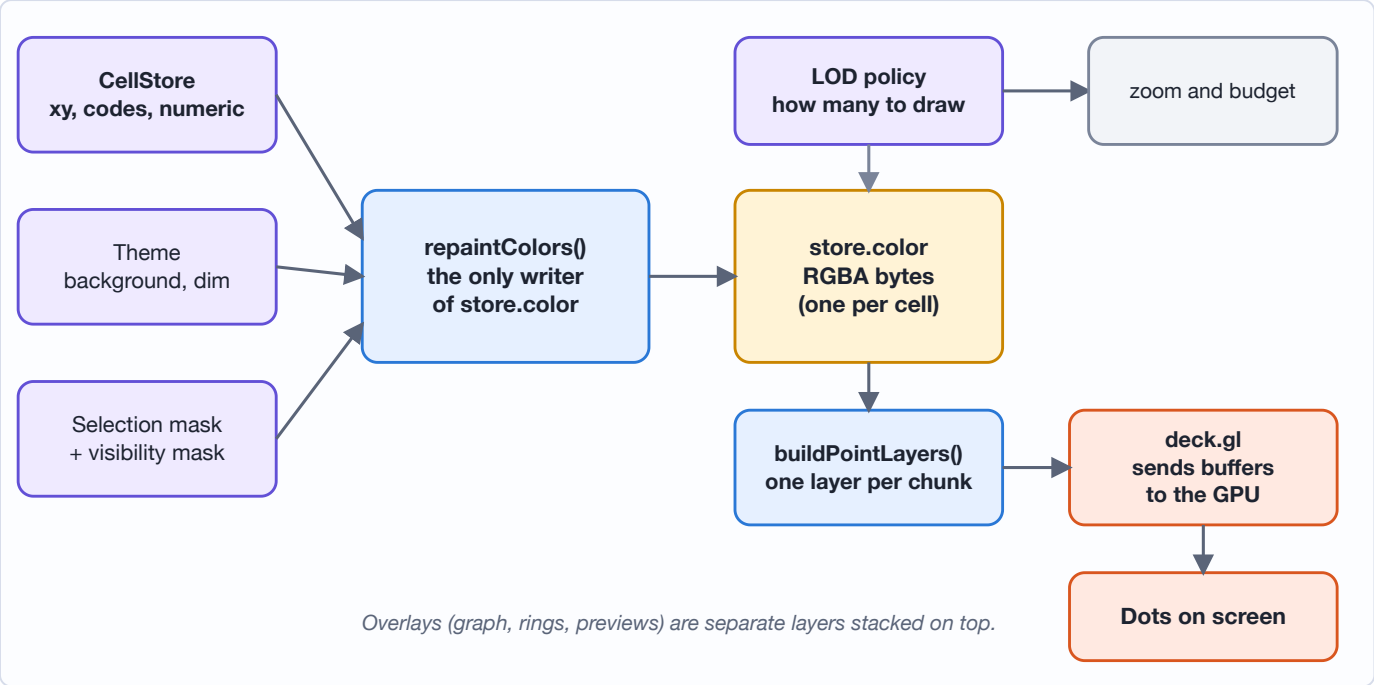
Memory use

Item	Per cell	At 1,000,000 cells
Coordinates (`xy`)	8 bytes	8 MB
Colors (`color`)	4 bytes	4 MB
Each categorical field	2 bytes	2 MB
Each numeric field	4 bytes	4 MB
Selection worker copy of `xy` and the grid index	about 12 bytes	about 12 MB
Measured total in the browser (4 category fields, 1 numeric)	about 54 bytes	**54 MB** heap

CHAPTER 18

Drawing the dots: ScatterCanvas

`apps/web/src/views/ScatterCanvas.ts` owns the GPU canvas. Both the embedding view and the trajectory view are built on it. It decides *what* deck.gl should draw each time something changes.



From data to pixels. Color work is done on the CPU into one buffer, then deck.gl uploads it.

Five ideas that make it fast

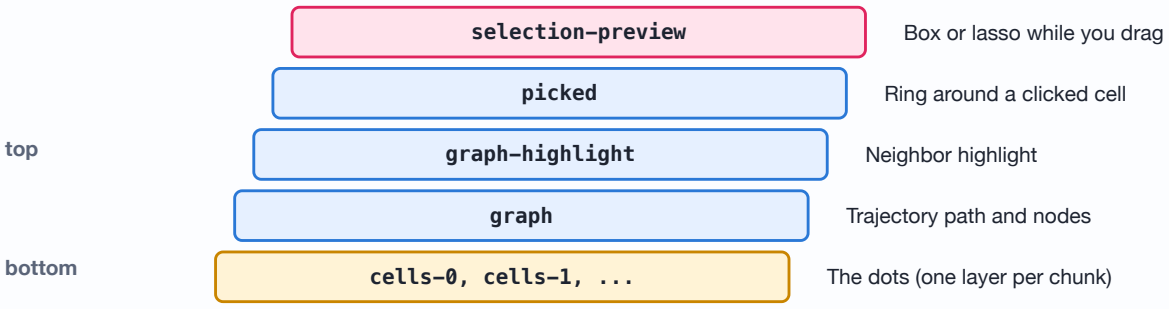
1. **Binary attributes.** The layers get the typed arrays directly (`getPosition` and `getFillColor` point at slices of `store.xy` and `store.color`). deck.gl never calls a function per point.
2. **One layer per chunk.** A new chunk adds a new layer. Old layers are untouched, so their GPU buffers are not re-uploaded.
3. **Re-upload only what changed.** A color change makes a *new attribute object* for colors only. deck.gl sees the new object and re-uploads colors, but not positions. A zoom changes the point radius and rebuilds layers, yet re-uploads nothing.
4. **A cache key.** `buildPointLayers()` remembers a key made of the drawn count, radius, color version and blend mode. If nothing in it changed, it returns the same layer objects. Dragging a lasso fires many events per second, and without this each one would rebuild every layer.
5. **One writer for color.** `repaintColors()` is the only code that writes `store.color`. Color-by, selection dimming, the legend filter and the theme all go through it, so two things can never fight over the buffer.

Hiding cells: culled in the shader

The legend filter hides a cell by giving it **alpha 0**. With blending off, a fully transparent dot would still overwrite whatever is behind it. So a tiny piece of shader code moves any cell with alpha 0 outside the screen, where it produces no pixels at all:

```
class CellLayer extends ScatterplotLayer {
  getShaders() {
    const shaders = super.getShaders();
    return { ...shaders, inject: { ...shaders.inject,
      'vs:#main-end': 'if (instanceFillColors.a == 0.0) { gl_Position = vec4(0.0, 0.0, 2.0, 1.0); }'
} };
  }
}
```

Layers stack in a fixed order



Overlays are separate small layers so they can change without touching the big point layers (list `OVERLAY_ORDER` in `ScatterCanvas.ts`).

The camera

- It is an **orthographic** view: flat, no perspective. A view state is just `target` (the map point at the center) and `zoom` (a power of two: each +1 doubles the size).
- The app keeps the view state itself (**controlled mode**) and gives it to deck.gl on every render. That makes a pan a cheap update instead of a full reset.
- `fitBounds()` flies the camera to show a rectangle. **Reset view** and **Zoom to selection** both use it.
- `screenToWorld()` and `worldToScreen()` convert between screen pixels and map positions. Clicks, the lasso, cards and the inset all use them.
- The y axis points **down** on screen, as in deck.gl's default orthographic view.

If the graphics card is lost

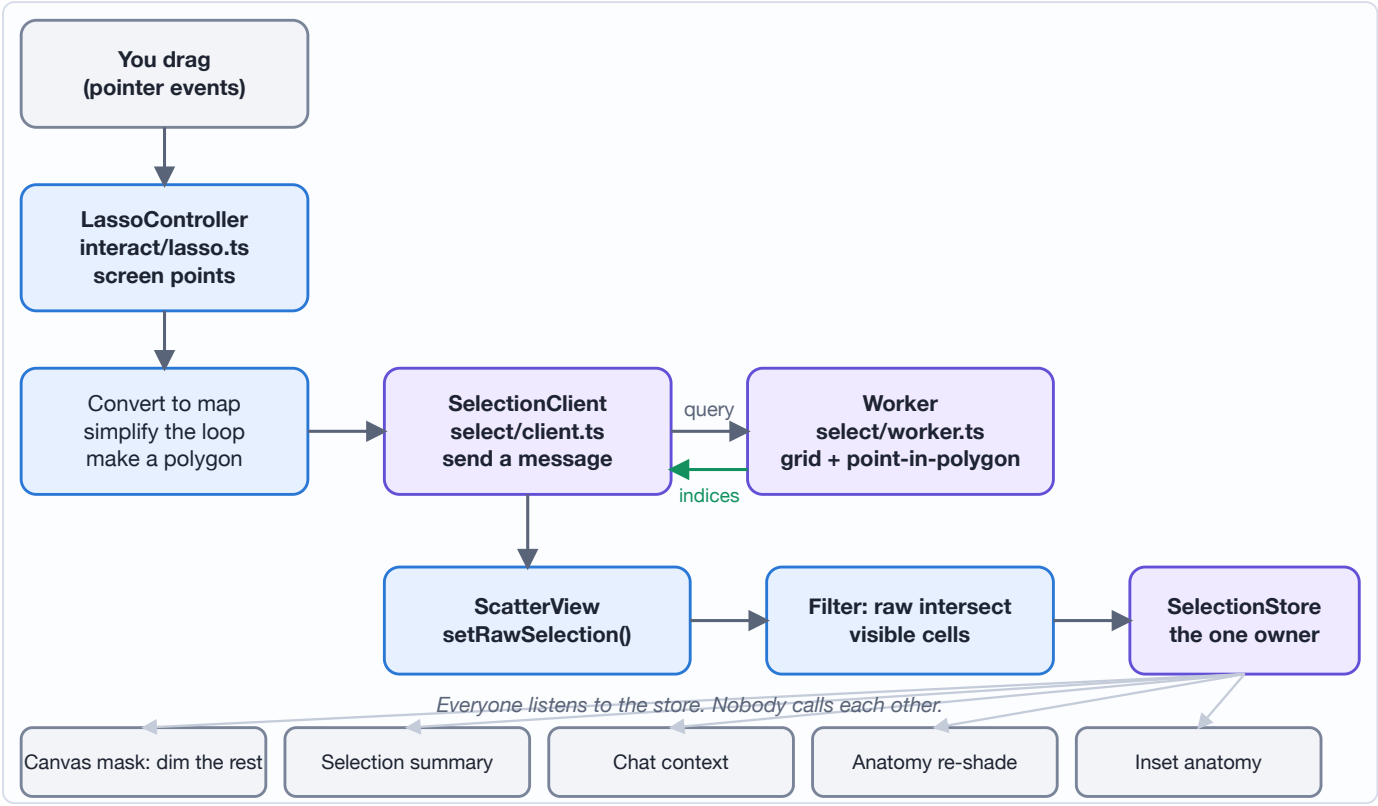
Sometimes the browser throws away the WebGL context (a driver reset, a sleeping laptop). `installContextLossRecovery()` listens for that and simply renders again from the typed arrays the app still holds. Nothing needs to be re-downloaded.

> **Two gotchas worth remembering**
>
> 1) deck.gl's `controller` option, if set on the Deck itself, writes a copy into the first item of the `views` array and that copy sticks. So the controller is set on the *view* instead. 2) After the user scrolls, deck.gl adds `zoomX` and `zoomY` to the view state and they override `zoom`. `withoutAxisZoom()` removes them before every programmatic camera move.

CHAPTER 19

How selection works

Selecting looks simple (draw a loop, cells light up) but it has to stay smooth with millions of cells. Four small pieces cooperate. Each lives in its own file.



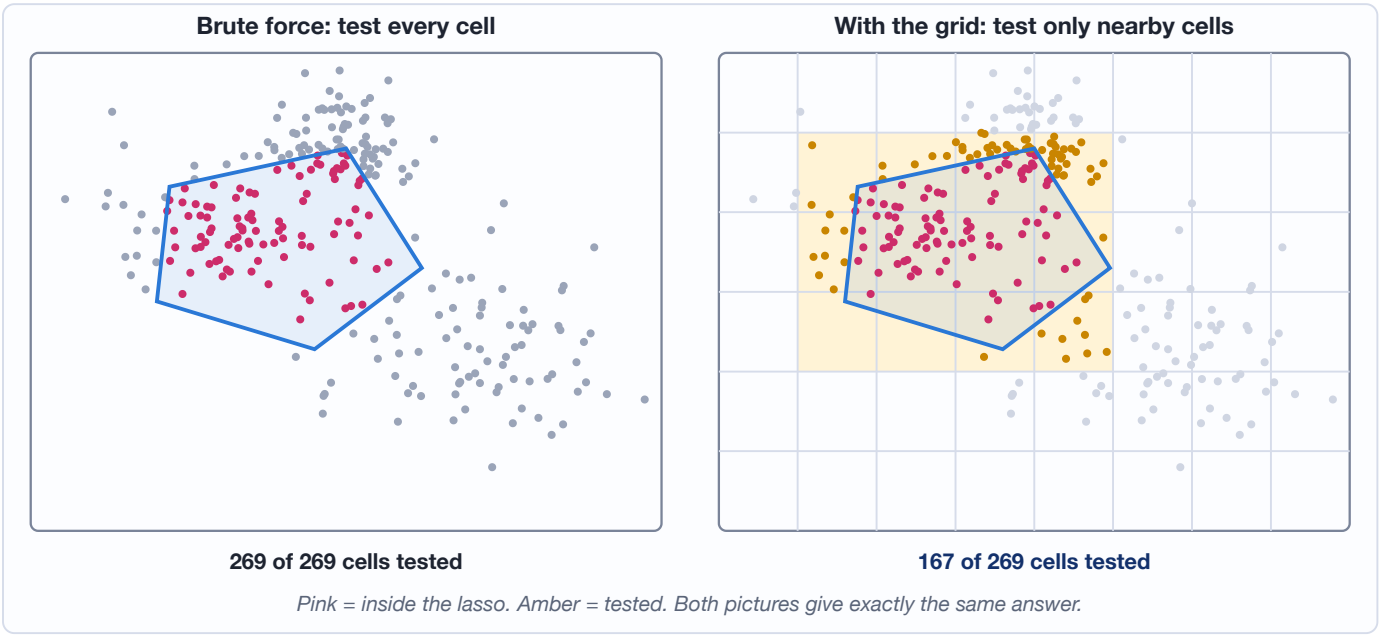
From a mouse drag to a changed screen.

Piece	File	What it does
LassoController	`apps/web/src/interact/lasso.ts`	Listens to the pointer. In Box mode it makes a rectangle, in Lasso mode a loop. Draws the live preview, simplifies the loop (drops vertices that hardly change its shape), turns screen pixels into map positions, and calls `onComplete(shape)`. Escape cancels. A drag shorter than 5 px is ignored.
SelectionClient	`packages/core/src/select/client.ts`	The front desk of the Worker. Sends the coordinates once (and again only if more cells have loaded), then sends each question with an id and hands you a Promise.
Worker	`packages/core/src/select/worker.ts`	Runs in a background thread. Builds the grid, answers *region* and *nearest* questions. Its logic is a plain function (`handleMessage`) so it can also run in Node.
Queries	`packages/core/src/select/query.ts`	`selectPolygon`, `selectRect`, `nearestPoints`: the actual search code.
SelectionStore	`packages/core/src/select/store.ts`	The **only** place the current selection lives. Everyone else subscribes to it.

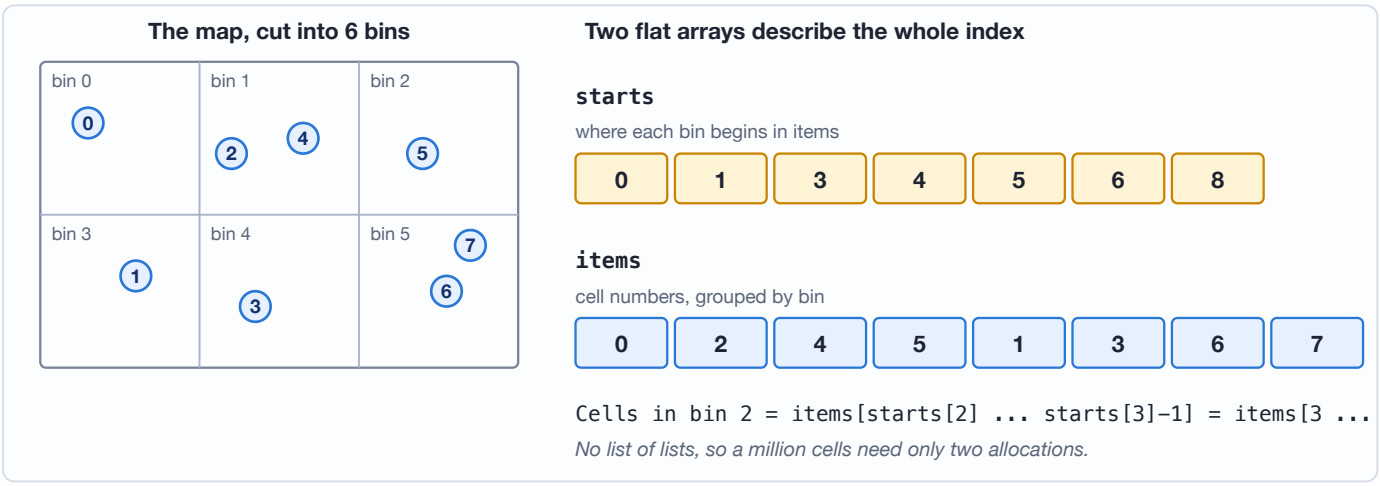
Why a Worker?

JavaScript in a web page normally runs on one thread that also draws the page and handles the mouse. A search over a million cells can take tens of milliseconds, long enough to make panning stutter. A Web Worker is a second thread. The search runs there, and the answer comes back as a message. The list of results is **moved** (not copied) back to the page, which is instant.

The grid makes it fast

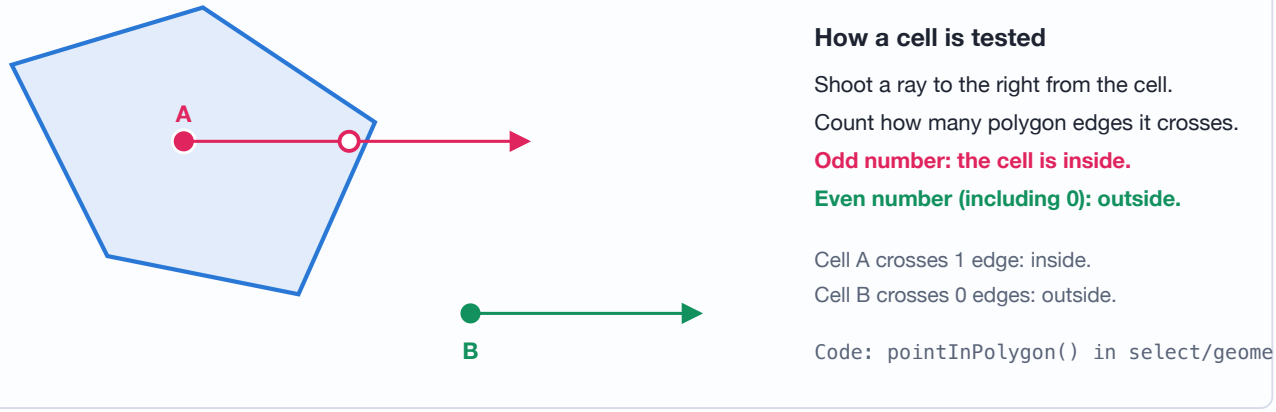


The grid index. The plot is cut into square bins. A lasso only looks at bins that its bounding box touches.



The same layout used in sparse-matrix files, called "compressed sparse row" (CSR).

Without a grid, the worker tests every cell against the polygon. With the grid, it first finds the bins touched by the polygon's bounding box and tests only the cells in those bins. The result must be **exactly** what the slow way would give. The grid only skips cells that cannot be inside.



"Crossing-number" point-in-polygon, the test run for every candidate cell.

Edge cases the code handles	How
A cell with `NaN` or infinite coordinates	Is left out of every bin, so no query can return it, and it cannot corrupt the grid.
All cells in the same spot	The grid's width is never allowed to be zero.
A query asks for more cells than the worker has	It clamps to what it has and rebuilds the grid if the loaded part has grown.
One bad request	It is answered with an error carrying its own id, so only that one promise fails.
A newer selection while an older one is still computing	A sequence number lets the newer one win; the older answer is thrown away.

Clicks use the same index

A click asks the worker for the **nearest** cells to the clicked spot (radius: the dot size plus a few pixels, at least 5). It asks for 32 candidates first. If every candidate is hidden by the legend, it asks again with 8 times more, up to 16,384. The first visible one wins.

The selection store and its listeners

```
selectionStore.subscribe(sel => { /* sel is null or { indices, mask, bbox, shape } */ });
selectionStore.set(indices, "poly");   // "poly", "rect" or "segment" (an anatomy click)
selectionStore.clear();
```

A `Selection` holds the list of selected cell numbers (`indices`), a 0/1 flag per cell (`mask`), the bounding box and how it was made. If one listener crashes, the others still run: a broken chat panel must not break the legend.

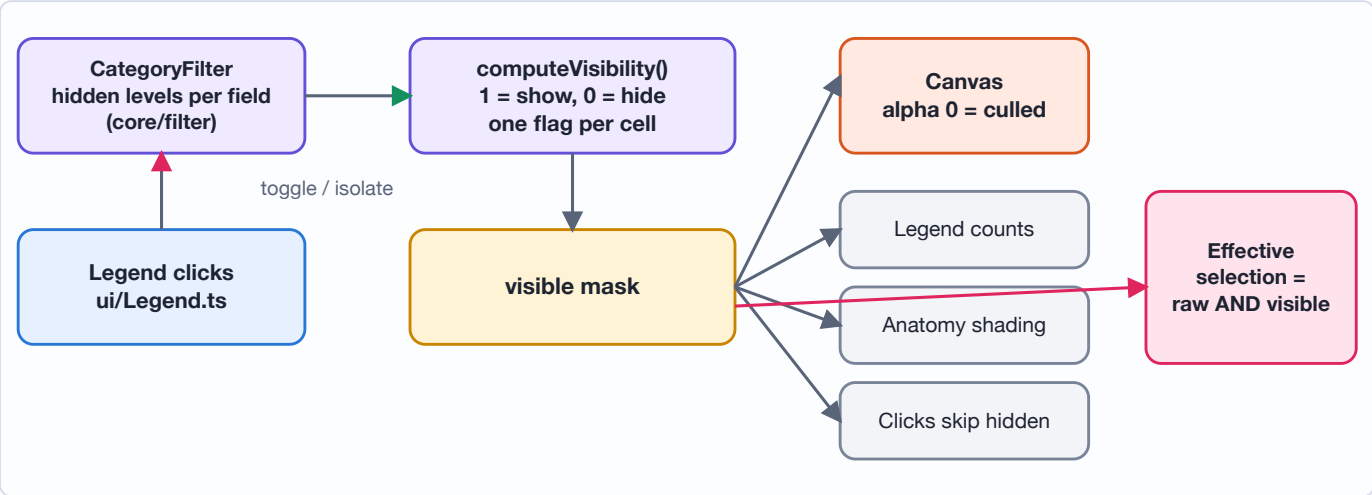
> **Raw selection versus effective selection**
> The worker returns the cells inside your outline, **including cells the legend has hidden**. `ScatterView` keeps that as the *raw* selection. The *effective* selection (what the store holds) is the raw one with hidden cells removed. When you change a filter, the effective selection is recomputed from the raw one, so un-hiding a type brings back its selected cells.

CHAPTER 20

Filtering and color

The legend filter

`CategoryFilter` (`packages/core/src/filter/categories.ts`) remembers **which levels are hidden**, field by field. It has no knowledge of the screen. It offers the same actions as Plotly's legend: `toggle`, `isolate`, `showAll` and `hideAll`, and it tells its listeners when something changed.



One flag per cell is the single source of "is this cell visible?".

- `computeVisibility()` loops once over all loaded cells. A cell is visible only if **none** of its levels is hidden in any filtered field. If nothing is filtered it returns `null`, so every downstream loop can skip the check.
- The result buffer is **reused**, so a legend click allocates nothing, even at ten million cells.
- `hiddenLabels()` tells the chat which categories are hidden, so the model knows what you can see.
- Hiding the picked cell's level closes its details card.

Colors

All color decisions live in `packages/core/src/color/`.

File	Job
`palette.ts`	24 base hues, plus a darker and a lighter cycle for datasets with more than 24 levels (72 colors in all). Two ramps: `VIRIDIS` and `MAGMA`. Every color can be **adjusted for a background** so it stays readable.
`recolor.ts`	Writes the RGBA bytes: `recolorCategorical`, `recolorNumeric`, `recolorConstant`, plus `legendEntries` for the legend rows and counts.
`defaults.ts`	Chooses the first color field when a dataset opens (the first categorical field, or the first numeric one if there are none).

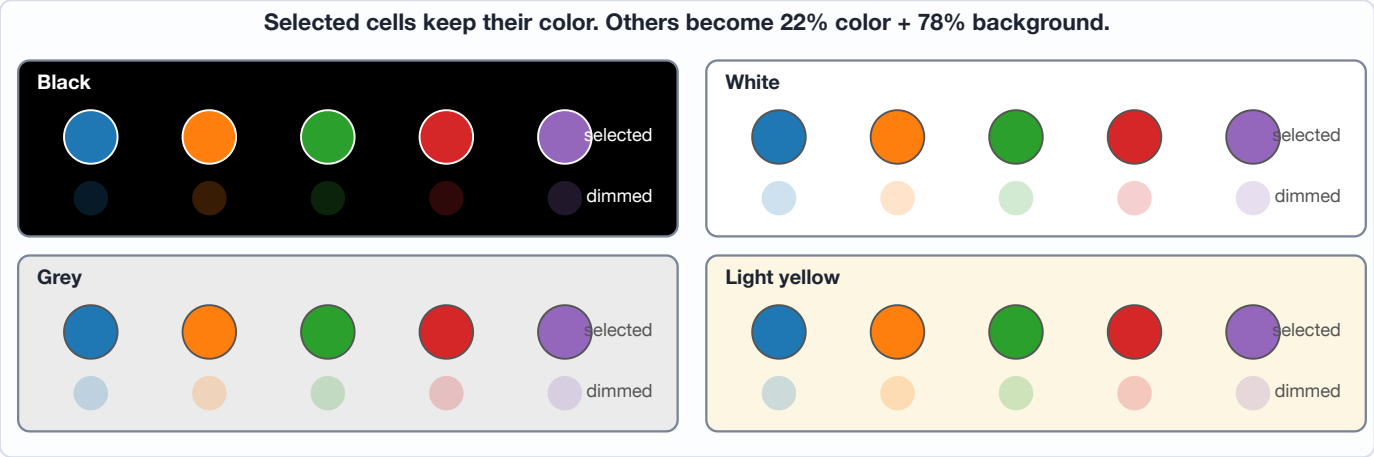
Staying readable on every background

A pale yellow that looks great on black disappears on white. So for each theme the app searches each color and nudges its brightness until its **contrast ratio** against the background is at least 3 to 1 (categorical colors) or 2.5 to 1 (ramps). The result is cached per background, so this is only done 72 times per theme, whatever the number of cells.

Dimming the unselected cells

```
// inside recolorCategorical (simplified)
for (let i = from; i < loadedCount; i++) {
  const base = palette[codes[i] % levels];
  if (mask === null || mask[i]) color[i] = base;                // selected: full color
  else color[i] = base * factor + background * (1 - factor);    // others: toward background
  alpha[i] = visible === null || visible[i] ? 255 : 0;          // hidden: alpha 0
}
```

`factor` is `0.22` in every built-in theme. The colors are computed **from the original codes every time**, never from the current color, so repeated selections never make the dimming pile up.



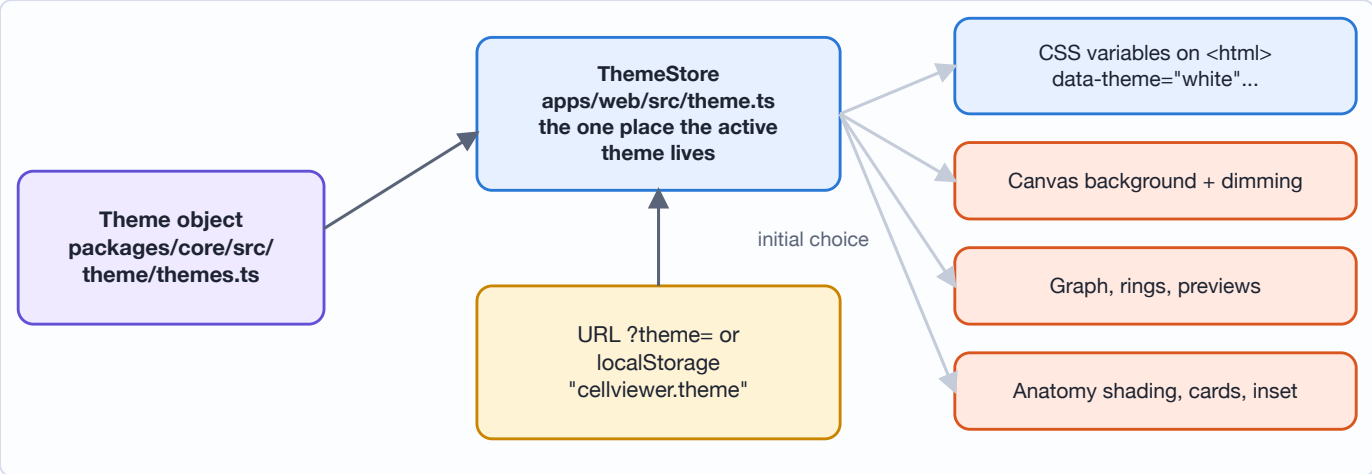
Dimming blends toward the background of the active theme, so unselected cells always recede.

> **Why not just lower the alpha?**
> Transparent dots need **blending**, the slowest thing a GPU can do with millions of overlapping points. Mixing the color toward the background looks the same on a plain background and keeps every dot fully opaque.

CHAPTER 21

Themes

A theme is more than a background color. The dimming, the graph lines, the highlights, the rings and the organ shading all have to stay readable on it. So a theme is a whole **bundle of colors**.



Changing the theme in one place changes every part of the app that listens.

Two halves that must match

Half	Where	Used for
Colors for the **GPU**	`packages/core/src/theme/themes.ts`	As RGB numbers [r, g, b]: the plot background, the dim factor, the graph colors, ring and highlight colors, the heat ramp.
Colors for the **page**	`apps/web/src/style.css`	As CSS variables (`--bg-panel` , `--text` ...), one block per theme: `:root[data-theme="white"] { ... }` .

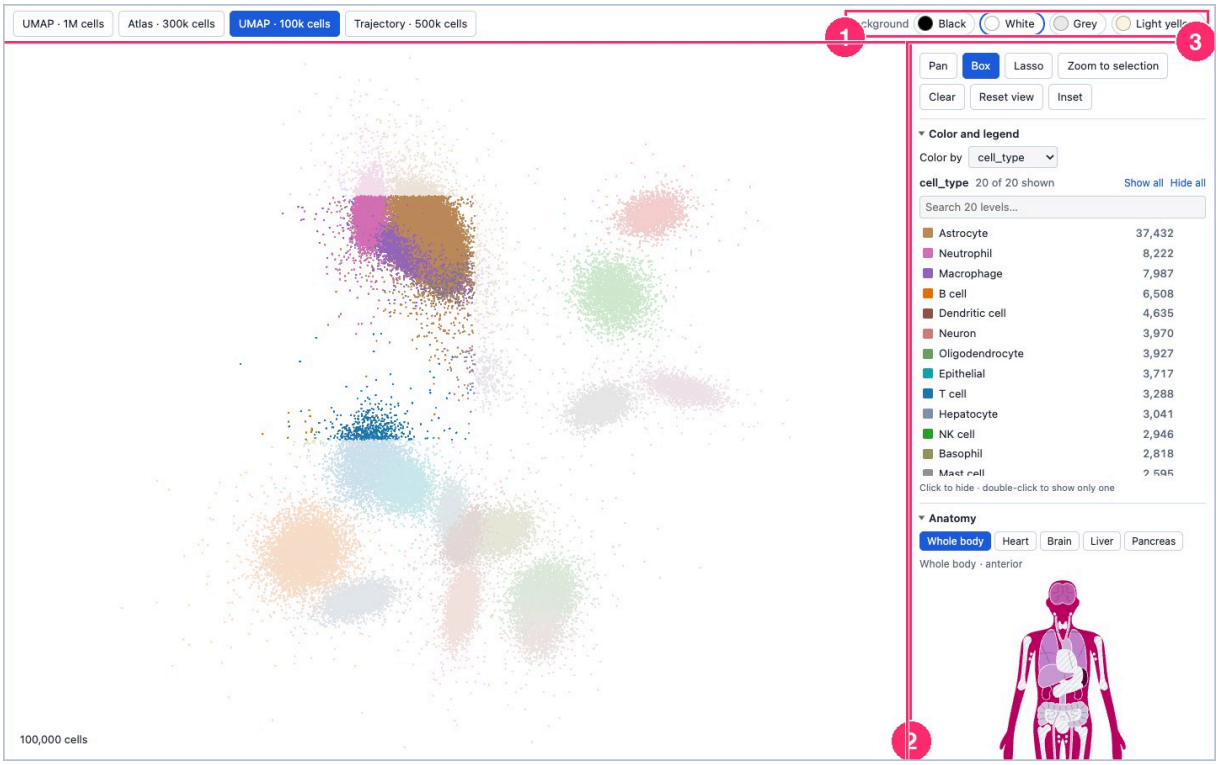
> **Rule 10**
>
> Colors that depend on the background must come from the active theme: CSS variables in the page and `Theme` RGB in deck.gl layers. Never write a color like `"#fff"` straight into a component.

What a Theme contains

Field	Meaning
`id` , `label` , `hint` , `mode`	Name shown on the picker button, its tooltip, and whether it is a dark or light theme.
`background`	The plot color. Unselected cells fade toward it.
`dimFactor`	How much of its own color a dimmed cell keeps (0.22).
`graph`	Colors of the trajectory: path, halo, node, root, branch, leaf.

Field	Meaning
`highlight`, `accent`, `ring`	Neighbor highlight, selection outline, ring around a clicked cell.
`panel`	Side panel color. The anatomy figure is drawn on it.
`heat`	Four color stops for anatomy shading, from "few cells" to "most cells".
`pointDefault`	Dot color for datasets with no field to color by.
`selectionPreview`	Outline and fill of an in-progress box or lasso.

The module also holds helpers to measure **contrast** (`contrastRatio` , `relativeLuminance`), the graph transparency constants and `toHex()` .



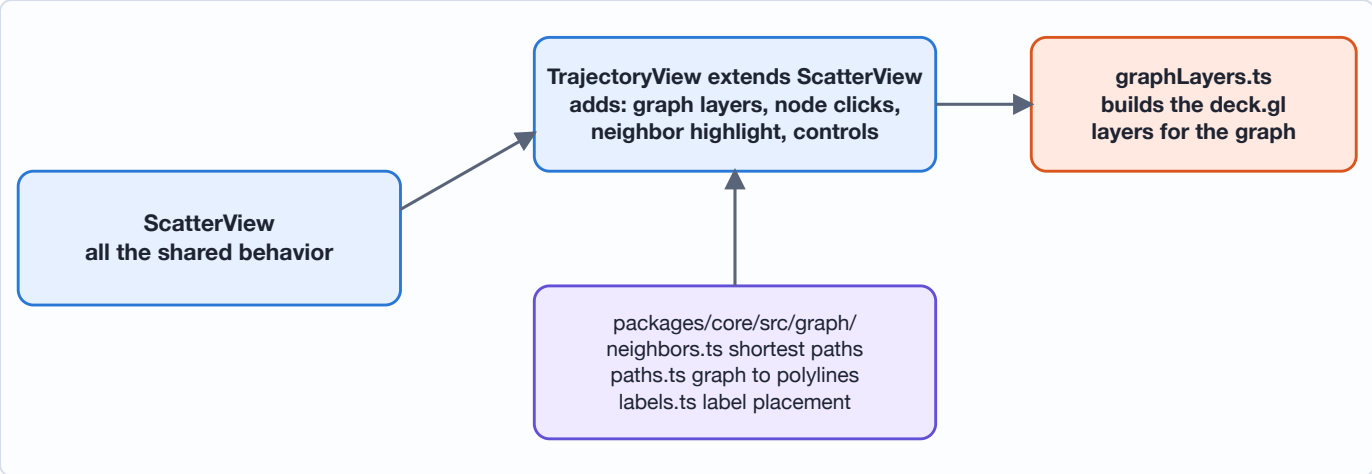
The white theme with a selection.

1. **Picker.** Reads its buttons from the list of themes. Add a theme and a button appears by itself.
2. **Plot background.** `theme.background`
3. **Panel.** `--bg-panel` (page) and `theme.panel` (anatomy figure)

CHAPTER 22

How the trajectory view works

`TrajectoryView` is a **subclass** of `ScatterView`. It inherits selection, zoom, the summary, the legend and the chat, and adds the graph and node clicks. That is why the trajectory view needed very little new code.



Where the trajectory code sits.

What happens when you click a node

1. `handleClick()` in `TrajectoryView` turns the click into a map position and asks `nearestNode()` for a graph node within the hit radius (at least 10 pixels). If there is one, it is selected. Otherwise the click falls through to the normal "pick a cell" code in `ScatterView`.
2. `closestNodes()` in `neighbors.ts` finds the nearest *k* nodes. With the **graph** metric it first runs `shortestPaths()` (Dijkstra's algorithm) from the clicked node, so distance means "sum of edge lengths along the path". With the **euclidean** metric it is a plain straight-line distance.
3. `shortestPathEdges()` works out which edges lie on the paths to the neighbors, so they can be drawn.
4. `buildGraphHighlightLayers()` draws the pink rings, numbers and connector lines. `placeLabels()` in `labels.ts` places the rank numbers so they avoid each other.
5. A node card opens with the neighbors listed.

The graph file

The graph is small (hundreds of nodes) so it is drawn plainly: a halo under a path, then nodes on top. `graphPaths()` merges the edge list into long polylines (it follows a run of nodes until it reaches a branch point or a leaf), and `graphNodeMarkers()` decides each node's kind. Every node is drawn, because every node can be clicked. All colors come from the theme.

Constant (in TrajectoryView.ts)	Value	Meaning
`NODE_HIT_PX`	12	How far from a node (in pixels) the search for a clicked node looks.

Constant (in TrajectoryView.ts)	Value	Meaning
`NODE_MIN_HIT_PX`	10	Every node accepts a click within at least this many pixels.
`NODE_MARKER_PX`	root 7, branch 5, leaf 3.5, node 2.4	Drawn marker sizes.
`k` (default)	5	Neighbors shown, changeable from 1 to 20 with the slider.

CHAPTER 23

How the inset works

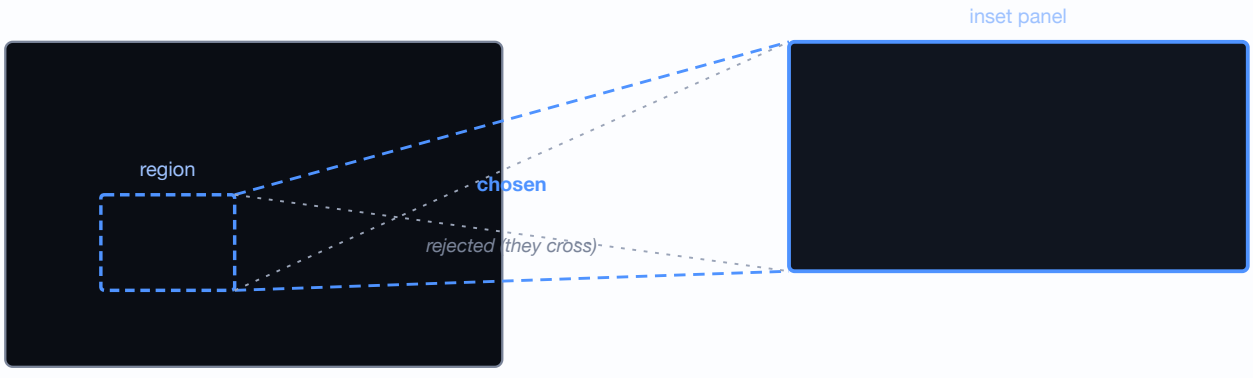
The inset has two parts: **maths** in core and **wiring** in the app.

Part	File	Does
Maths	`packages/core/src/inset/geometry.ts`	`visibleBounds` (what rectangle of the map a camera shows), `regionAround`, `magnification`, `clampRect` (keep a box inside the canvas), `rectsIntersect`, `connectorLines` (the two lines).
Wiring	`apps/web/src/inset/InsetController.ts`	Builds the frame, the four mode buttons, the dashed marker and the SVG lines. Handles dragging and resizing. Saves mode and frame to `localStorage`.
Drawing	`apps/web/src/views/ScatterCanvas.ts`	Adds a **second deck.gl view** for the inset rectangle (`setInset()`) and chooses which layers appear in it.

The second camera

The inset is a second *view* inside the same deck.gl instance. Same WebGL context, same data on the GPU, a different rectangle on the canvas and a different camera. The layer filter says which layers appear where: the main view shows everything except layers whose id starts with `inset:`. The inset shows the first `chunks` point layers (4 in Zoom mode, 1 in Overview), the graph, and its own `inset:` layers.

The two connector lines



The code picks the two outer tangents, so the lines never cut through either rectangle.

`connectorLines(region, inset)` *chooses which two of the four possible lines to draw.*

Keeping the box locked to the data

The marker, the lines and the panel are ordinary page elements placed over the canvas. Whenever the main camera moves, `onViewChanged()` re-projects the region's map corners to screen pixels with `worldToScreen()` and moves the elements. So they stay glued to the data while you pan and zoom.

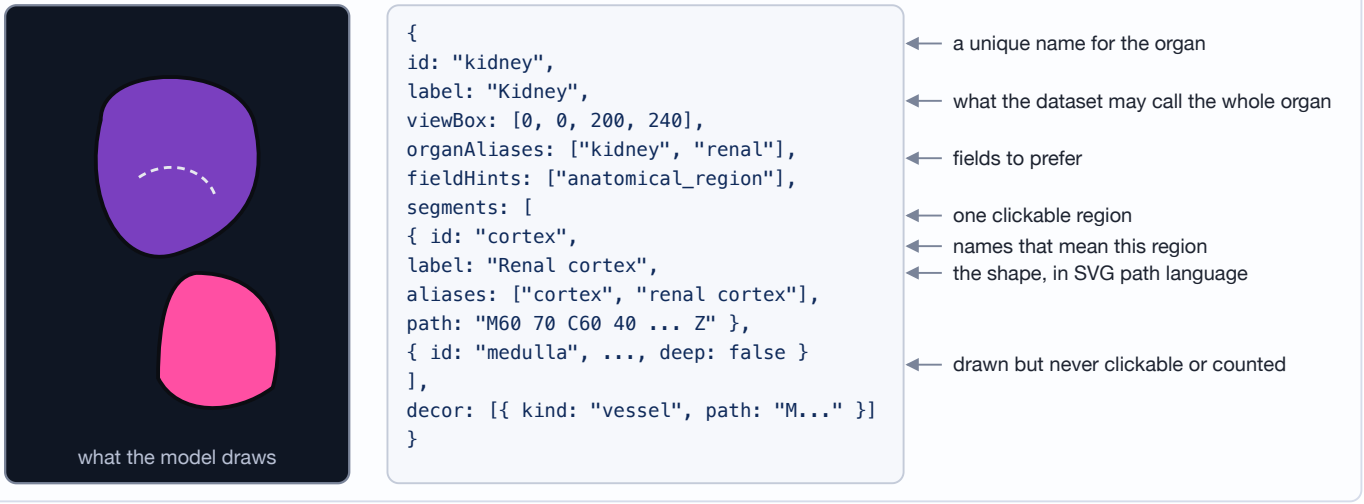
In **Overview** mode the process runs backwards: clicking at a spot in the panel is converted to a map position (`insetScreenToWorld`), and the main camera is moved there with `panTo()`.

Constant	Value	Meaning
`ZOOM_CHUNKS`	4	Chunks drawn in Zoom mode (up to 1,000,000 cells).
`MIN_W`, `MIN_H`	200, 150	Smallest panel size in pixels.
`HEADER_PX`	27	Height of the panel header.
`STORAGE_KEY`	`cellviewer.inset`	Where mode and frame are saved.

CHAPTER 24

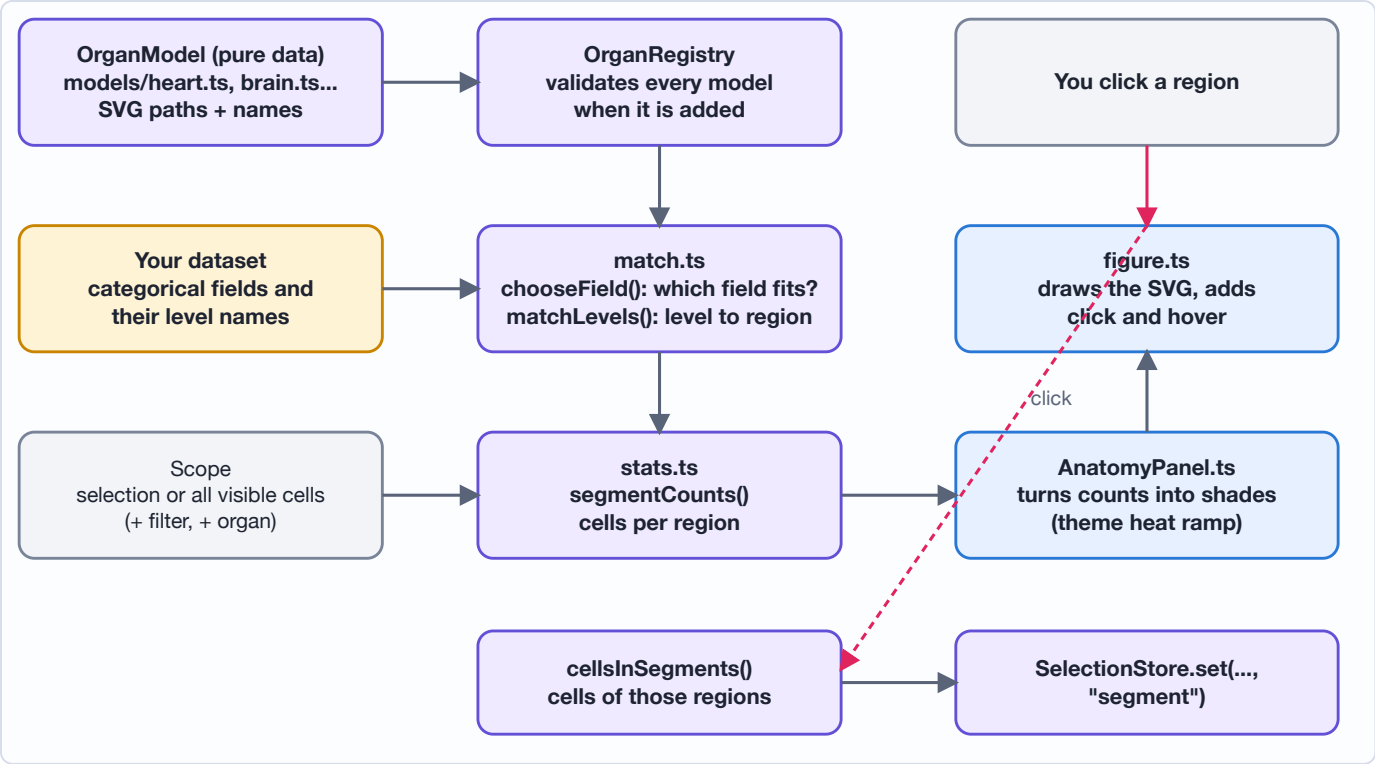
How the anatomy view works

The anatomy feature is a good example of the project's style. **Organs are data.** Drawing one is writing a file, not writing logic. Everything else (matching, counting, shading, clicking) is shared code that works for any organ.



An organ is a small data object. The picture and the matching names live together.

File	Job
`core/anatomy/types.ts`	The shapes: `OrganModel`, `OrganSegment`, `OrganDecor`.
`core/anatomy/models/*.ts`	The five built-in organs: body, heart, brain, liver, pancreas. `index.ts` lists them in `BUILTIN_MODELS`.
`core/anatomy/registry.ts`	`validateModel()` and `OrganRegistry`. A bad model fails with a clear message: no segments, duplicate ids, a name used by two regions, broken path data, or a path outside the box.
`core/anatomy/path.ts`	A small SVG path reader. It gives `pathBounds()` so a model can be checked against its box.
`core/anatomy/match.ts`	Turns dataset labels into regions: `normalizeLabel`, `matchLevels`, `chooseField`, `organScope`.
`core/anatomy/stats.ts`	`segmentCounts` (cells per region) and `cellsInSegments` (cells for a click).
`web/anatomy/figure.ts`	Draws one model as SVG and reports clicks and hovers. Colors come from CSS variables.
`web/anatomy/AnatomyPanel.ts`	Tabs, breadcrumb, shading, tooltip, region-field menu. Connects to the selection.
`web/anatomy/gallery.ts`	The `anatomy.html` page.



Anatomy in both directions. Cells shade the figure (top path). A click on the figure selects cells (pink path).

Matching names to regions

A dataset says `"Heart: Left Ventricle"` or `"left_ventricle"`. The model says `left ventricle`. The matcher makes them meet:

Step	Example
Normalize: lower-case, remove accents and punctuation, collapse spaces	"Left_Ventricle" becomes "left ventricle"
Split qualifiers (`:` `|` `/` or " - ")	"Heart: Left Ventricle" gives qualifier "heart" and part "left ventricle"
Singular as a fallback	"kidneys" is also tried as "kidney"
Look up in an index of every alias of every region	"left ventricle" is an alias of region `left_ventricle`
Check the organ	The last part alone is trusted only if the qualifier names this organ (or there is none). So "lung: medulla" never lights the brain's medulla.

`chooseField()` then picks the dataset field that matches the **most different regions** (ties go to the one whose name is in the model's `fieldHints`). That is why the atlas dataset picks `tissue` for the whole body and `anatomical_region` for the heart.

Scope: which cells are being counted?

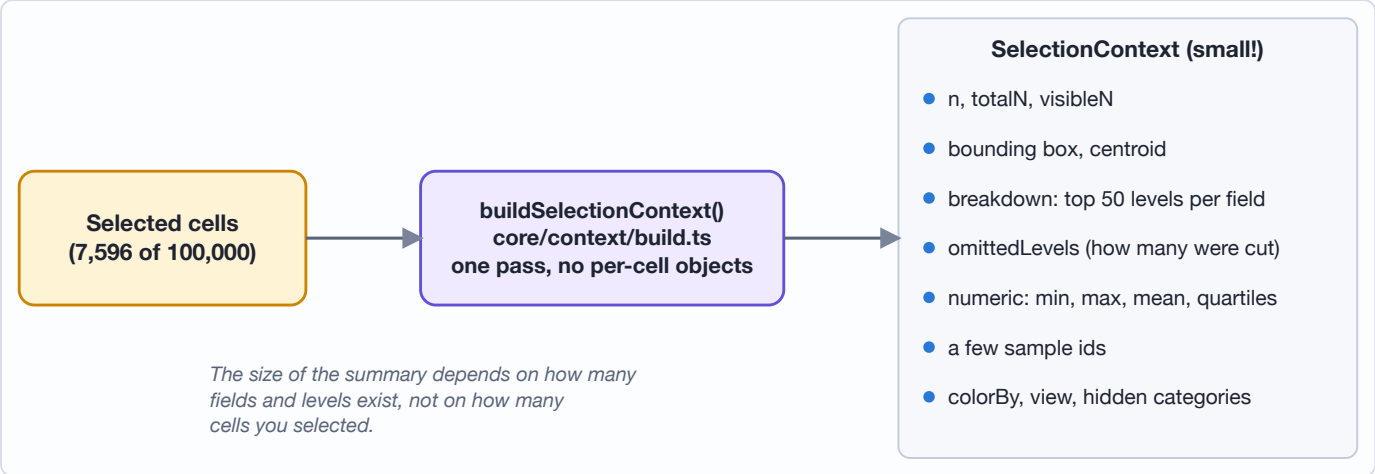
Shading is computed for the cells "in scope": the selection if there is one, otherwise all visible cells (after the legend filter). When an organ is open, scope is also limited to that organ's cells (found with `organScope()`, for example all cells whose tissue is "heart"), so a vague label like "medulla" cannot light up the wrong organ.

While chunks are still arriving, the view re-shades at most every quarter of a second, so the figure can lag a little. Once the last chunk lands it is always exact.

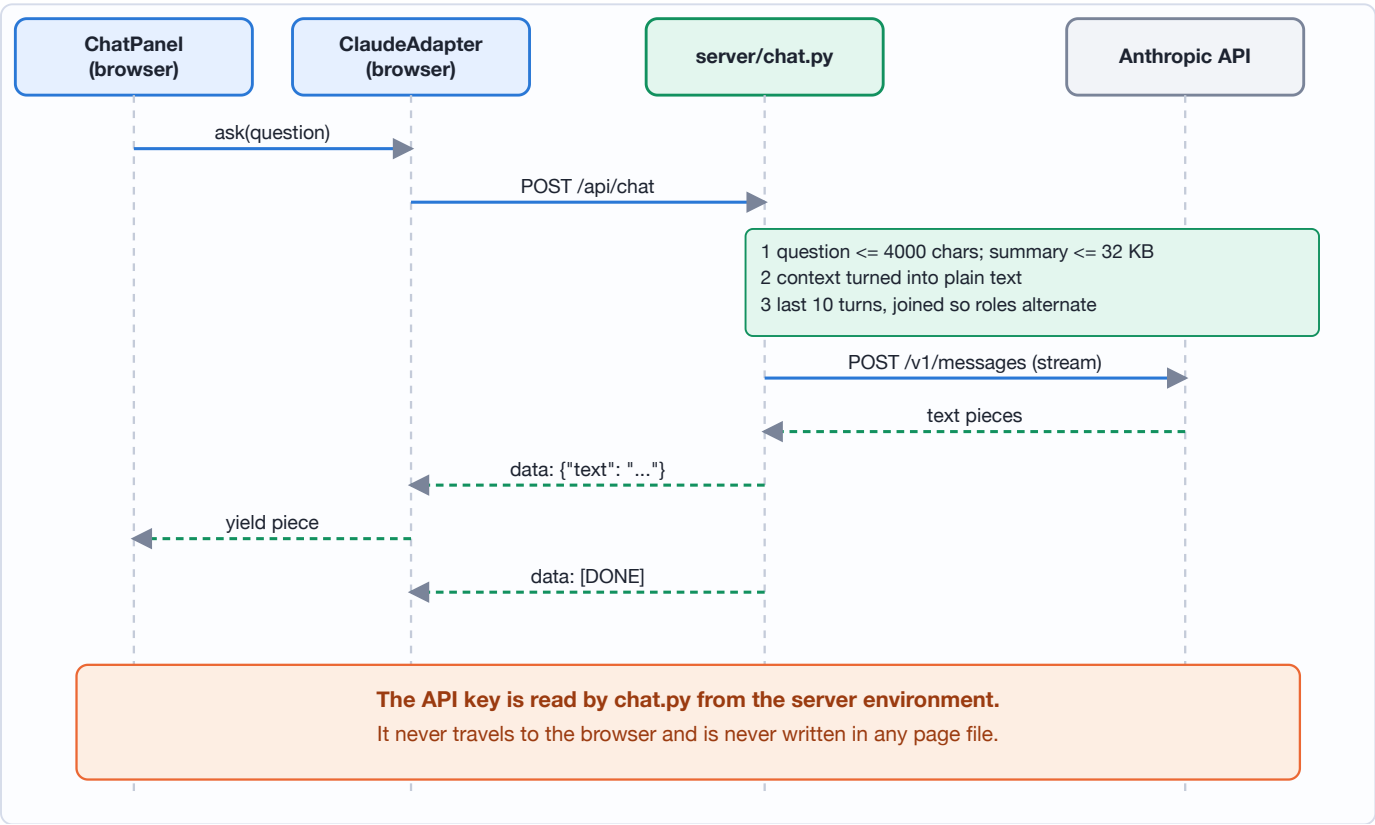
CHAPTER 25

How the chat works

The chat is built so that the part that talks to the AI is **swappable** and the part that holds the secret key is **never in the browser**.



What the chat receives instead of cells.



One chat question, from your keyboard to the model and back. The answer arrives as a stream, a few words at a time.

The pieces

File	Job
`core/context/build.ts`	`buildSelectionContext()`: makes the summary. Quartiles are computed exactly like NumPy's default, so the numbers match a notebook.
`core/chat/adapter.ts`	The `ChatAdapter` interface: `send(context, question, history, signal)` returns text pieces. The panel talks only to this.
`core/chat/mock.ts`	`MockAdapter` (offline helper) and `renderContextAsText()` (summary to plain text).
`web/chat/claude.ts`	`ClaudeAdapter`: calls `POST /api/chat` and reads the stream. `isConfigured()` asks `/api/chat/status` once at start-up to choose the adapter.
`web/chat/ChatPanel.ts`	The messages, the box and the Ask button. Handles Enter, cancel, errors, and disables itself when nothing is selected.
`server/chat.py`	The route, the system prompt, size limits, and the streaming call to the model.

Limits and constants (server/chat.py)

Name	Value	Meaning
`MAX_CONTEXT_BYTES`	32,000	A larger summary is refused with HTTP 413.
`HISTORY_TURNS`	10	Only the last ten messages are sent.
`MAX_HISTORY_CHARS`	60,000	Older turns are dropped to fit.
`MAX_TURN_CHARS`	20,000	One very long turn is cut.
`question` length	1 to 4,000	Checked by the request model.
`max_tokens`	1,024	Longest answer.
`DEFAULT_MODEL`	`claude-sonnet-5`	Used unless `CHAT_MODEL` is set.

Two copies of one function, on purpose

The summary is turned into text twice: by `renderContextAsText()` in TypeScript (for the offline helper) and by `render_context_as_text()` in Python (for the real model). They must produce the same words, otherwise the offline helper would stop predicting what the real model sees. **If you change one, change the other.** The Python version is written to never crash on a strange summary: missing or odd values print as "n/a" and the rest still renders.

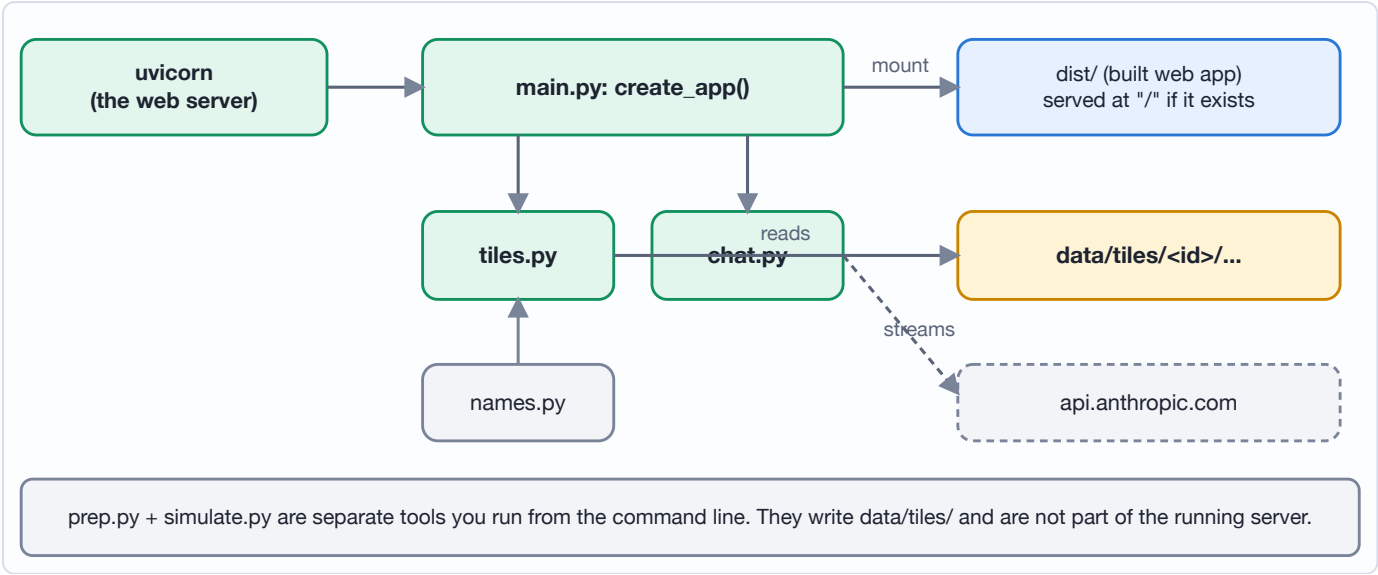
Errors

- If the model call fails before it starts, the route returns an HTTP error (503 if no key, 413 if too large, 422 if the summary is unreadable).
- If the model fails **after** it started answering, the server sends an `{"error": ...}` frame, so a cut-off answer never looks complete.
- The panel shows errors in a red message and keeps the box usable.

CHAPTER 26

The server

The server is a small **FastAPI** application. In normal use it does three things: lists and serves dataset files, forwards chat questions, and (in production) serves the built web page.



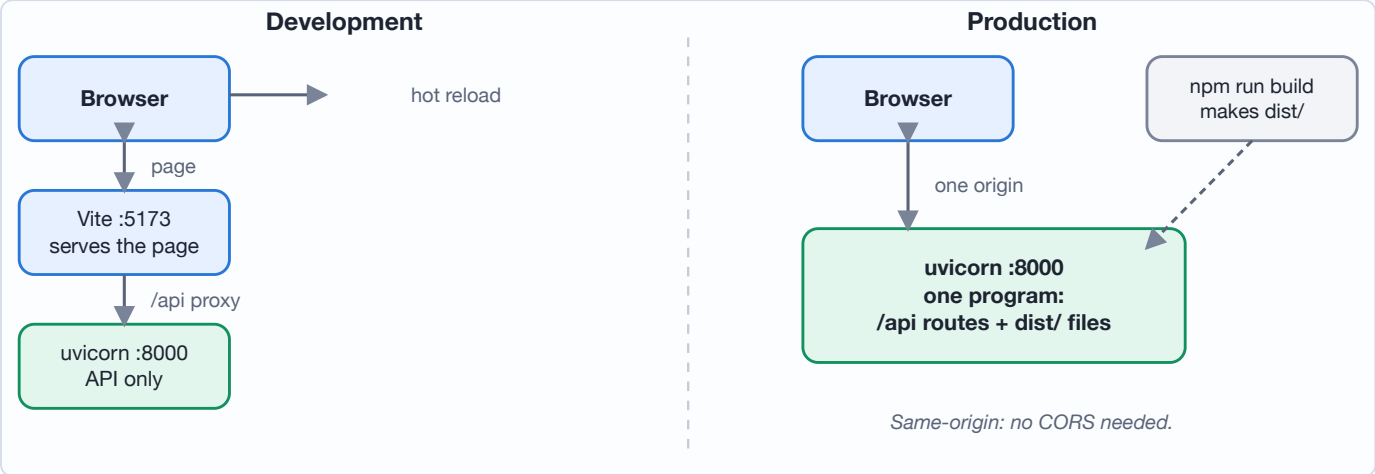
The running server in one picture.

File	What is in it
`server/main.py`	`create_app(tiles_dir, dist_dir)` makes the app. It sets **CORS** (allows any `localhost` port, plus origins listed in `CORS_ORIGINS`), adds the two routers, and mounts `dist/` last so it never hides `/api`.
`server/tiles.py`	The dataset routes. Validates names, memory-maps the per-cell files, sends chunks with a one-year `immutable` cache header.
`server/chat.py`	The chat route and everything that shapes the question (see Chapter 25).
`server/names.py`	`SAFE_NAME`, `is_safe()`, `slug()`: the one rule for dataset ids and field names, shared by prep and the routes.
`server/prep.py`	The prep tool (command line). Reading, cleaning, shuffling, chunking, writing, pruning.
`server/simulate.py`	Makes the demo data: `simulate_embedding`, `simulate_atlas`, `simulate_trajectory` (grows a random tree).
`server/requirements.txt`	Python libraries to install.

Safety rules in the server

Risk	What the code does
A web address that tries to read other files (`../../etc/passwd`)	Dataset ids and field names are checked with `SAFE_NAME.fullmatch()` (letters, digits, `_`, `-`, up to 64) *before* they touch a file path. A bad name gets HTTP 400.

Risk	What the code does
A trailing newline sneaking past a regular expression	The pattern uses `\A...\Z` and the code uses `fullmatch`, not `match`.
The API key leaking	It is read from the environment only inside the chat route. No route ever returns it.
A page showing stale cell names after a dataset is rebuilt	The cell route needs the manifest `version` and answers 409 if it no longer matches.
Half-written files being served	prep writes each file under a temporary name and renames it.



In development two programs share the work. In production one program does everything.

One origin is simpler

In production the page and the API come from the same address, so the browser never needs CORS permission. The CORS rule in `main.py` is only a fallback for a page opened on a different origin (set `VITE_API` and `CORS_ORIGINS` for that).

PART 5

Reference and recipes

Look things up, change things safely, and fix problems.

CHAPTER 27

File-by-file reference

Every file you may need to open. The number is the length of the file in lines, as of this guide. Small files are on purpose: each file has one job.

Root

File	Lines	What it is for
`package.json`	24	Names the project, lists the JavaScript libraries and the commands (`dev` , `build` , `preview` , `typecheck`).
`package-lock.json`	1427	The exact versions installed. Do not edit by hand.
`tsconfig.json`	16	TypeScript settings: strict checks, and the `@core` shortcut for `packages/core/src` .
`vite.config.ts`	25	Vite settings: the `/api` proxy, the `@core` shortcut, and the two pages built (index and anatomy).
`pyproject.toml`	5	Names the Python side and the Python version it needs.
`.env`	4	Server settings (API key, model, tiles folder, CORS). Never commit it.
`.gitignore`	19	Files git must skip: `node_modules` , `.venv` , `dist` , `data/tiles` , `.env` .
`README.md`	580	Short project guide.

apps/web (the browser app)

File	Lines	What it is for
`apps/web/index.html`	9	The main page: an empty `div` and a script tag.
`apps/web/anatomy.html`	9	The organ gallery page.
`apps/web/src/main.ts`	189	Start-up. Finds datasets, builds the top bar, opens datasets, picks the chat adapter, shows start-up errors.
`apps/web/src/api.ts`	4	Where the API lives (`VITE_API` or the same origin) and the hint printed when the server is down.
`apps/web/src/theme.ts`	69	`ThemeStore` : the active theme, saving it, applying CSS variables.
`apps/web/src/style.css`	621	All styling. One block of CSS variables per theme.

File	Lines	What it is for
`apps/web/src/views/ScatterCanvas.ts`	521	The GPU canvas: layers, colors, camera, level of detail, inset view, context-loss recovery.
`apps/web/src/views/ScatterView.ts`	481	The embedding view: builds the whole side panel and connects canvas, selection, filter, legend, cards, anatomy, inset, chat.
`apps/web/src/views/TrajectoryView.ts`	257	Extends ScatterView with graph display, node clicks, neighbor highlight and controls.
`apps/web/src/views/graphLayers.ts`	244	Builds the deck.gl layers for the graph and for the neighbor highlight.
`apps/web/src/ui/ViewSwitcher.ts`	30	The dataset buttons.
`apps/web/src/ui/ThemePicker.ts`	33	The four background buttons.
`apps/web/src/ui/Toolbar.ts`	62	Pan, Box, Lasso, Zoom to selection, Clear, Reset view, Inset.
`apps/web/src/ui/Legend.ts`	226	The legend rows, search box, chips, bulk buttons, double-click logic.
`apps/web/src/ui/InfoCard.ts`	90	The pinned card for a clicked cell or node.
`apps/web/src/ui/SelectionSummary.ts`	42	The counts and breakdown of the selection.
`apps/web/src/ui/LodControl.ts`	36	Auto detail checkbox, slider and readout.
`apps/web/src/ui/RenderControl.ts`	27	The Translucent points checkbox.
`apps/web/src/ui/Section.ts`	30	Foldable side-panel sections that remember their state.
`apps/web/src/ui/escape.ts`	9	`esc()` : makes text safe to place inside HTML.
`apps/web/src/interact/lasso.ts`	200	`LassoController` : box and lasso drawing, preview, cancel.
`apps/web/src/interact/click.ts`	49	`ClickDetector` : tells a click from the start of a drag.
`apps/web/src/anatomy/AnatomyPanel.ts`	420	The anatomy side panel and inset content.
`apps/web/src/anatomy/figure.ts`	114	Draws an organ model as SVG.
`apps/web/src/anatomy/gallery.ts`	39	The gallery page.
`apps/web/src/inset/InsetController.ts`	447	The floating inset: frame, modes, marker, connector lines.
`apps/web/src/chat/ChatPanel.ts`	113	The chat user interface.
`apps/web/src/chat/claude.ts`	58	`ClaudeAdapter` : talks to `/api/chat` .
`apps/web/src/chat/adapter.ts`	2	Re-exports the adapter types and the mock.

packages/core/src (the logic)

File	Lines	What it is for
`packages/core/src/data/manifest.ts`	34	The `Manifest` and `PrincipalGraph` types, and manifest checking.
`packages/core/src/data/schema.ts`	82	`CellStore`: the typed arrays and `loadedCount`.
`packages/core/src/data/loader.ts`	149	Downloads a dataset: parallel chunks, retries, cancel.
`packages/core/src/data/cellInfo.ts`	33	Asks the server for one cell's name and source row.
`packages/core/src/index/grid.ts`	89	`UniformGrid` and `buildGrid()`: the CSR index.
`packages/core/src/select/geometry.ts`	97	Point-in-polygon, polygon bounds, rectangle normalizing, path simplifying.
`packages/core/src/select/query.ts`	142	Region and nearest-cell search, with or without the grid.
`packages/core/src/select/worker.ts`	116	The worker message handler and its entry point.
`packages/core/src/select/client.ts`	83	The page-side handle of the worker.
`packages/core/src/select/store.ts`	73	`SelectionStore`.
`packages/core/src/color/palette.ts`	142	Categorical colors, ramps, and contrast adjustment.
`packages/core/src/color/recolor.ts`	170	Writes the color buffer; builds legend entries.
`packages/core/src/color/defaults.ts`	15	Chooses the first color-by field.
`packages/core/src/filter/categories.ts`	167	`CategoryFilter`, `computeVisibility` and helpers.
`packages/core/src/render/lod.ts`	25	`lodPolicy()`: how many cells to draw and how big.
`packages/core/src/theme/themes.ts`	161	The four themes, contrast math and graph transparency constants.
`packages/core/src/context/build.ts`	127	Builds the selection summary for the chat.
`packages/core/src/chat/adapter.ts`	19	The `ChatAdapter` interface.
`packages/core/src/chat/mock.ts`	133	The offline helper and the summary-to-text function.
`packages/core/src/graph/neighbors.ts`	203	Graph index, Dijkstra, closest nodes, nearest node.
`packages/core/src/graph/paths.ts`	67	Edges to polylines; node markers.
`packages/core/src/graph/labels.ts`	95	Label placement without overlaps.
`packages/core/src/inset/geometry.ts`	125	Inset maths: bounds, magnification, connectors.
`packages/core/src/anatomy/types.ts`	28	Organ model types.
`packages/core/src/anatomy/registry.ts`	75	Model validation and the registry.
`packages/core/src/anatomy/match.ts`	154	Matching dataset labels to regions.

File	Lines	What it is for
`packages/core/src/anatomy/stats.ts`	82	Counting cells per region and finding cells of regions.
`packages/core/src/anatomy/path.ts`	179	SVG path reading and bounds.
`packages/core/src/anatomy/models/body.ts`	44	The whole-body figure.
`packages/core/src/anatomy/models/heart.ts`	42	The heart.
`packages/core/src/anatomy/models/brain.ts`	43	The brain.
`packages/core/src/anatomy/models/liver.ts`	27	The liver.
`packages/core/src/anatomy/models/pancreas.ts`	29	The pancreas.
`packages/core/src/anatomy/models/index.ts`	25	`BUILTIN_MODELS` and `createDefaultRegistry()`.

server (Python)

File	Lines	What it is for
`server/main.py`	42	App factory, CORS, serving `dist/`.
`server/tiles.py`	120	Dataset, chunk, graph and cell routes.
`server/chat.py`	310	Chat route, prompt, limits, streaming.
`server/names.py`	39	The safe-name rule.
`server/prep.py`	558	The prep tool.
`server/simulate.py`	364	Demo data generators.
`server/requirements.txt`	8	Python libraries.

CHAPTER 28

The HTTP API

The browser and the server talk through a handful of web addresses. All start with `/api`. You can try the simple ones in a browser address bar while the server runs, for example `http://localhost:8000/api/datasets`.

Request	Answer	Used by
`GET /api/datasets`	`{"datasets": ["atlas300k", "sim100k", ...]}`	`main.ts` at start-up
`GET /api/dataset/{id}/manifest`	The dataset description (JSON)	`loader.ts`, `main.ts`
`GET /api/dataset/{id}/graph`	The principal graph (JSON). 404 if none.	`loader.ts` (`fetchGraph`)
`GET /api/dataset/{id}/chunk/xy/{n}?v=`	Raw bytes: float32 x,y pairs	`loader.ts`
`GET /api/dataset/{id}/chunk/codes/{field}/{n}?v=`	Raw bytes: uint16 level numbers	`loader.ts`
`GET /api/dataset/{id}/chunk/num/{field}/{n}?v=`	Raw bytes: float32 values	`loader.ts`
`GET /api/dataset/{id}/cell/{index}?v=`	`{"index", "row", "id"}`. 409 if the version changed.	`cellInfo.ts`, on click
`GET /api/chat/status`	`{"configured": true/false, "model": "..."}`	`main.ts` to choose the adapter
`POST /api/chat`	A stream of `data: {"text": "..."}` lines, then `data: [DONE]`	`claude.ts`

The chat request body

```
{
  "question": "Which cell types dominate?",
  "context": { ...the selection summary... },
  "history": [ {"role": "user", "text": "..."}, {"role": "assistant", "text": "..."} ]
}
```

Status codes you may meet

Code	Meaning
400	A dataset id or field name has characters that are not allowed.
404	No such dataset, chunk, graph or cell.

Code	Meaning
409	The dataset was rebuilt since the page loaded. Reload the page.
413	The selection summary is bigger than 32,000 bytes.
422	The summary could not be read.
502	The model service returned an error.
503	Chat is not configured (no API key).

CHAPTER 29

Recipes: how to change things

Each recipe says **what to open**, **what to change** and **how to check**. After any change to TypeScript, run `npm run typecheck`. With `npm run dev` running, the page reloads by itself when you save.

1. Change the default background

Open `packages/core/src/theme/themes.ts` and change `DEFAULT_THEME_ID` from `'black'` to `'white'`, `'grey'` or `'yellow'`. A visitor's saved choice and `?theme=` still win over it.

2. Add a fifth theme

1. In `themes.ts`, add the new id to the `ThemeId` type, for example `'black' | 'white' | 'grey' | 'yellow' | 'sepia'`.
2. Copy one object in the `THEMES` list and change every value: `background`, `panel`, `heat`, graph colors and so on. Keep contrast in mind (`contrastRatio()` is in the same file).
3. In `apps/web/src/style.css`, copy one `:root[data-theme='white'] { ... }` block, rename the id to `sepia` and change the variables.
4. That is all. The picker builds its buttons from the list, so a fifth button appears by itself.

3. Draw more or fewer cells at once

Open `apps/web/src/views/ScatterCanvas.ts` and change `DEFAULT_POINT_BUDGET` (1,000,000). Lower it for slower computers. The detail slider in the app overrides it per session.

4. Make unselected cells fade more or less

In `themes.ts`, change `dimFactor` in a theme (0.22). Zero makes unselected cells vanish into the background. One means no fading.

5. Change the colors used for categories

Open `packages/core/src/color/palette.ts` and edit `BASE_CATEGORICAL`, a flat list of red, green, blue triplets (24 colors). Keep the length a multiple of 3. The contrast adjustment still runs for every theme.

6. Add a new color ramp for numbers

1. In `palette.ts`, copy how `VIRIDIS` is made with `buildRamp([...])` and add your own list of colors.
2. Add it to the `RAMPS` table: `{ viridis: VIRIDIS, magma: MAGMA, mine: MINE }`.
3. Choose it with `canvas.setColorBy(field, 'numeric', 'mine')`. There is no menu for ramps yet. A good first exercise is to add one to `ScatterView.buildColorPicker()`.

7. Add a new field to color or filter by

No code change. Put the column in your file and run prep again. Every categorical column becomes a field with a legend. Every numeric column becomes a ramp.

8. Make an organ recognize more names

Open the organ file in `packages/core/src/anatomy/models/`, find the region, and add the new spelling to its `aliases` list. Matching ignores capitals and punctuation, so `"Left Ventricle"` and `left_ventricle` need only one entry.

9. Add a new organ

1. Copy `models/liver.ts` to `models/kidney.ts`. Change the id, label, aliases, and draw your own `path` values (any SVG editor can export them).
2. In `models/index.ts`, import it and add it to `BUILTIN_MODELS`.
3. To open it from the body, add `drillDown: 'kidney'` to the body model's kidney segment.
4. Open `/anatomy.html` to check every region. A mistake is reported by name in the browser console.

10. Change what the chat model is told

Open `server/chat.py`. `SYSTEM_PROMPT` holds the instructions. `CHAT_MODEL` in `.env` chooses the model. `max_tokens` (1024) limits the answer length. Restart the server after a change.

11. Add a new API route

1. In `server/tiles.py`, add a function under the others:

```
@router.get("/dataset/{dataset_id}/hello")
def hello(request: Request, dataset_id: str) -> JSONResponse:
    root = _dataset_root(request, dataset_id)   # checks the name and that it exists
    return JSONResponse({"dataset": dataset_id, "folder": root.name})
```

2. Restart uvicorn (or start it with `--reload`). Try `http://localhost:8000/api/dataset/sim100k/hello`.
3. From the web app, call it with `fetch(`${API}/api/dataset/${id}/hello`)`. `API` comes from `apps/web/src/api.ts`.
4. **Always** pass a dataset id or field name through `_dataset_root` or `_safe` before using it in a file path.

12. Add a button to the toolbar

1. In `apps/web/src/ui/Toolbar.ts`, add a `<button data-act="mine">` to the HTML and a handler name to the `handlers` argument.
2. Add the click listener the same way as the other buttons.

3. In `ScatterView.ts`, where `createToolbar(...)` is called, pass the new handler.

13. Change how easy it is to click a cell or a node

`clickRadiusWorld()` in `ScatterView.ts` sets the reach of a cell click (the dot size plus 3 pixels, at least 5). `NODE_HIT_PX` and `NODE_MIN_HIT_PX` in `TrajectoryView.ts` do the same for graph nodes.

14. Change the legend

In `apps/web/src/ui/Legend.ts`: `SEARCH_THRESHOLD` (10) is the number of levels above which the search box appears. `DOUBLE_CLICK_MS` (300) is how fast two clicks must be to count as a double-click.

15. Change the chunk size

Run prep with `--chunk-size 100000`. The browser reads the size from the manifest, so no code change is needed. Remember the Zoom inset draws four chunks, so a smaller chunk size means fewer cells in the inset (`ZOOM_CHUNKS` in `InsetController.ts`).

16. Change the ports

Web: `server.port` in `vite.config.ts`. API: `--port` when you start uvicorn, and `API_TARGET=http://localhost:9000 npm run dev` so the proxy follows.

17. Change the names on the dataset buttons

In `apps/web/src/main.ts`, the `describe()` function turns `sim100k` into "UMAP · 100k cells". Datasets with other names (such as `mydata`) simply use their id. Edit `describe()` to label them differently.

18. Fold or open a side-panel section by default

In `ScatterView.ts`, `createSection(this.side, 'anatomy', 'Anatomy')` takes a fourth argument `open` (default `true`). The user's own choice, saved in the browser, still wins.

> **A safe working habit**
> Change one thing at a time. After each change: save, look at the page, run `npm run typecheck`. If something breaks, undo just that change. The project is not in git yet, so consider running `git init` and committing before you start experimenting.

CHAPTER 30

Rules the code follows

These ten rules explain many design choices. If you follow them, your changes will fit in. If you must break one, understand the reason first.

Rule	What it means in practice	Why
1. No per-cell objects	Every column is a typed array. Hot loops are plain `for` loops that create nothing.	A million small objects cost more time to create and clean up than the entire drawing. Typed arrays are also what the GPU wants.
2. Core is free of the screen and deck.gl	`packages/core` imports no DOM and no deck.gl, so it runs in plain Node.	It keeps the hard logic simple, testable and safe from interface changes. `apps/web` only wires and draws.
3. Points are opaque; blending is off	Dimming mixes color toward the background. Hidden cells get alpha 0 and are culled in the vertex shader. The only blending is the optional "Translucent points" switch, off by default.	Blending a million overlapping points can cost three to four times the frame rate.
4. Point layers are not pickable	Clicking a cell asks the selection worker for the nearest cells. Hover does nothing.	A GPU picking pass would re-draw every point on every pointer move, and hover details distracted users.
5. Only [0, loadedCount) is real data	Every loop that reads cells stops at `store.loadedCount`.	Beyond it the arrays are zeros that look like cells at the origin.
6. Spatial queries equal brute force exactly	The grid index must return the same cells as testing all of them.	An index that silently drops a cell is worse than no index.
7. Selection lives only in SelectionStore	The raw outline is intersected with the legend filter before it reaches the store. Everyone else subscribes.	One owner means no two parts of the app can disagree about what is selected.
8. The API key never reaches the browser	It lives in the server's environment, read only in `server/chat.py`.	Anything in the browser can be read by the user, and by anyone else.
9. Dataset text goes through esc()	Any text from a dataset (cell types, names) is passed through `esc()` before it is put into HTML.	A cell type called `<img onerror=...>` must show as text, not run as code.
10. Background-dependent colors come from the theme	CSS variables in the page, `Theme` RGB in deck.gl layers. The four themes are black, white, grey and light yellow.	A color that is fine on black can vanish on white.

Writing style used in the code

- One idea per file, and small files. A new panel is a function `createX(root, ...)` that returns a handle.
- Anything async that can be replaced by a newer request checks a sequence number or abort signal before touching the screen.
- Comments are short. They give a file's purpose, a heading, or a warning about something non-obvious.

CHAPTER 31

When something goes wrong

What you see	Why	What to do
"Waiting for the data server"	The Python server is not running, or is on another port.	Start it (Chapter 3, step 5). The page retries by itself.
The dataset buttons are missing	There are no datasets in `data/tiles`.	Run the prep command, then reload.
"No dataset named ..." in the status line	A typo in `?dataset=`.	Use one of the names listed in the message.
A blank black canvas	WebGL 2 is off or unsupported, or the graphics driver reset.	Try another browser, enable hardware acceleration, reload.
Chat answers start with "[mock adapter: ...]"	No `ANTHROPIC_API_KEY` on the server.	Add it to `.env`, start uvicorn with `--env-file .env`.
Chat says "chat is not configured"	Same cause, reached through `/api/chat`.	Same fix.
"dataset changed since the page loaded; reload"	You prepared the dataset again while the page was open.	Reload the page.
Panning feels slow with millions of cells	The GPU cannot draw that many dots at full detail.	Leave **Auto detail** on, or lower the slider. Keep Translucent off.
The anatomy figure is all hatched	No field in your data matches the region names.	Pick another **Region field**, or add names to the organ (recipe 8).
`npm run build` fails on types	A TypeScript error.	Read the file and line in the message. `npm run typecheck` shows it faster.
Port 5173 is busy	Another Vite is running.	Vite moves to 5174 and prints it. Use that address.
Prep says a column has too many values	More than 65,535 different values.	Drop it or bin it, or leave it out with `--obs-fields`.
Prep says `--embedding` is not two-dimensional	The `obsm` key has more than two columns.	Use `X_umap` or `X_tsne`, or slice two columns first.

Where to look for clues

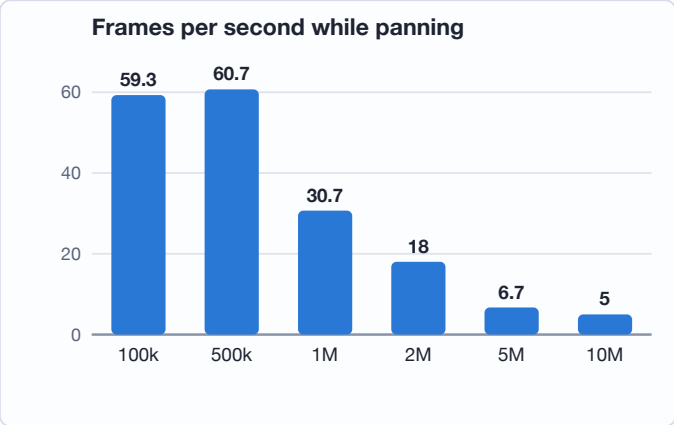
- **Browser console** (`F12`, then Console): errors from the page, including "organ model ... skipped".
- **The terminal running uvicorn**: errors from the server.
- **The terminal running Vite**: errors in the TypeScript files you just saved.
- **Network tab** in the browser tools: shows each `/api/...` request and its status code.

CHAPTER 32

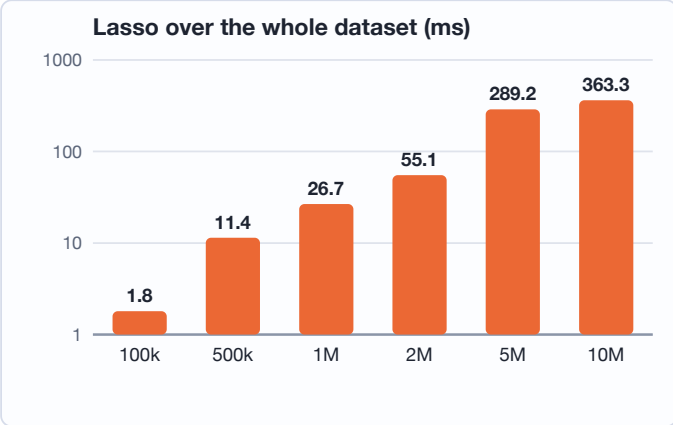
Performance: how fast is it?

All numbers come from a measured sweep on an Apple M4 laptop (Chrome-based browser, real GPU, canvas about 1440 by 420 pixels, a three-second scripted pan). Your numbers will differ with your hardware, but the **shape** of the curves is what matters.

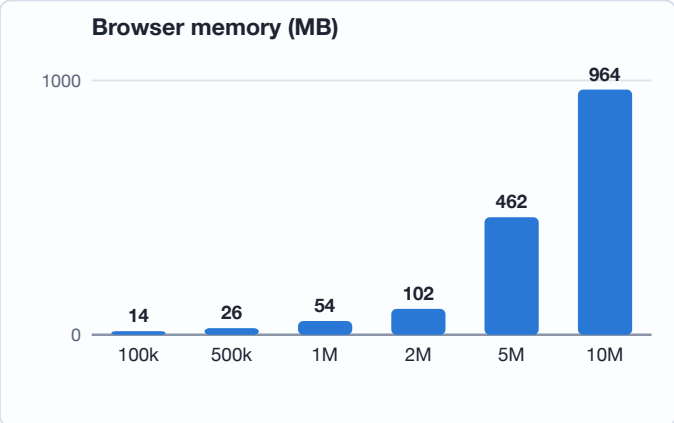
Cells	Load (ms)	Frames per second	Lasso (ms)	Memory (MB)
100,000	21	59.3	1.8	14
500,000	17	60.7	11.4	26
1,000,000	44	30.7	26.7	54
2,000,000	63	18.0	55.1	102
5,000,000	146	6.7	289.2	462
10,000,000	628	5.0	363.3	964



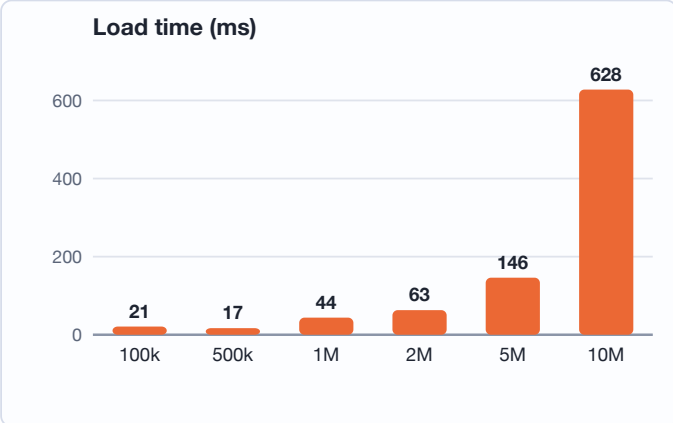
Smooth up to 500k. The default draw budget of 1M keeps panning usable above that.



Log scale. Selection stays interactive far past the point where drawing slows.



Linear: roughly 96 bytes per cell in practice.



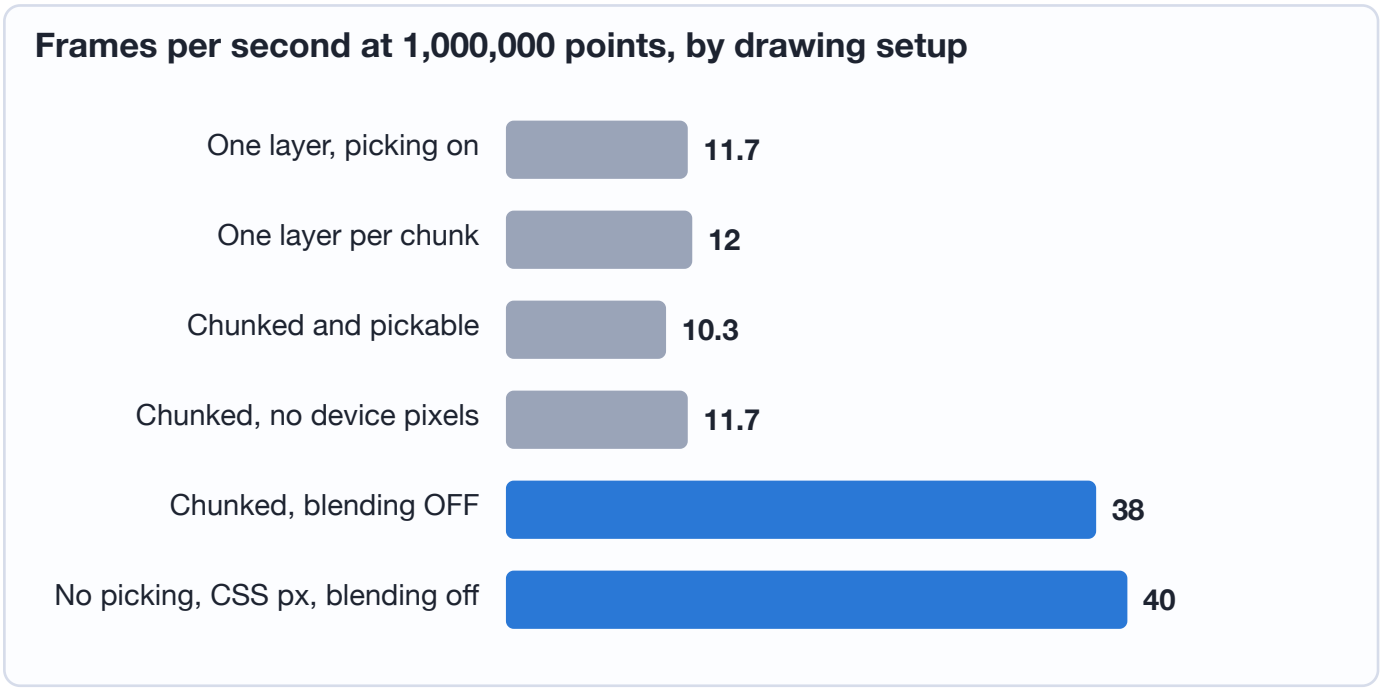
From localhost. The server is not the limit.

Rendering is the wall

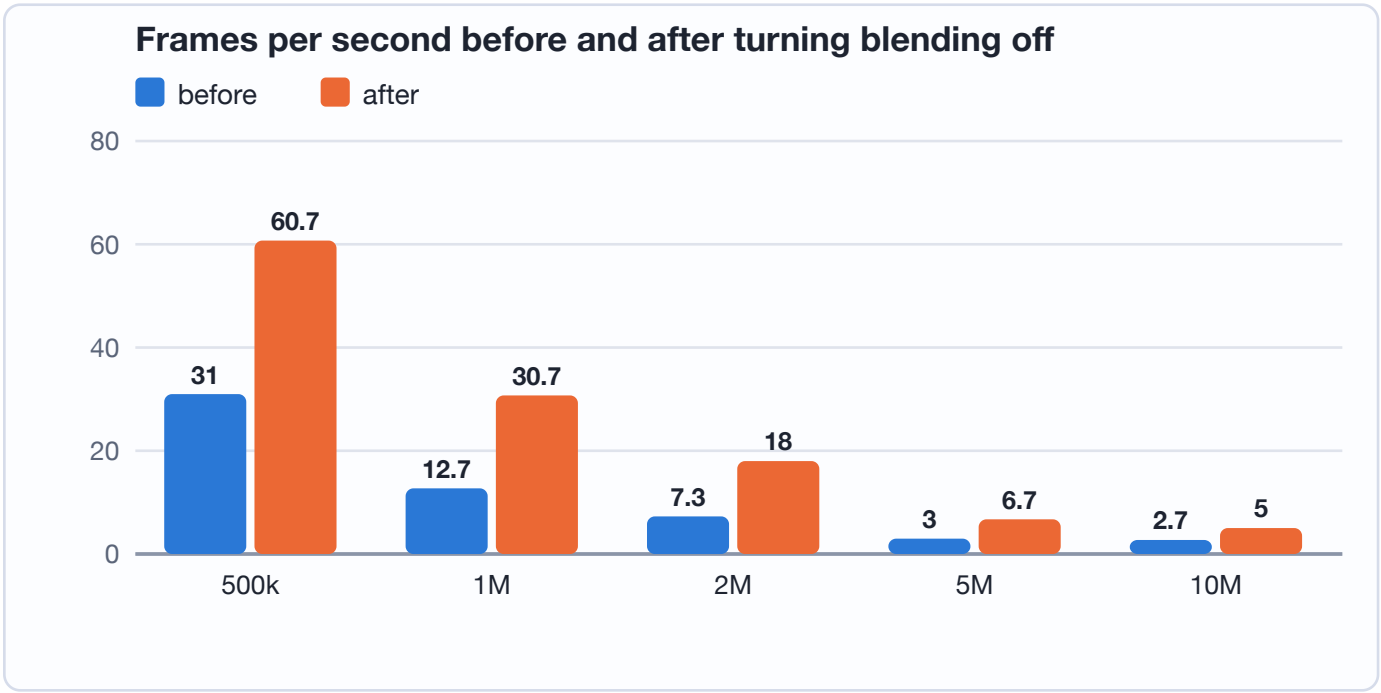
Ten million cells arrive and decode in under a second. The slow part is drawing them, and it starts to bite at about two million dots. That is why the app draws at most about a million at a time by default.

The discovery that shaped the design

At one million points the first version ran at about 12 frames per second, far less than expected. The cause was **alpha blending**. Apple's GPUs draw in small tiles and skip hidden pixels cheaply, but blending a million overlapping dots defeats that trick. Turning blending off tripled the speed.



Chunking and pixel ratio cost nothing. Blending cost more than three times the frame rate.



This is why points are opaque, why selection fades colors toward the background instead of lowering alpha, and why "Translucent points" is an optional switch that starts off. It is also why a click does not use GPU picking.

Smaller facts from the same tests

- Point **count** matters far more than point **size**. At ten million points, making dots 5 times larger (25 times the area) cost only 1.8 times the frame time. So the app limits how many points it draws instead of shrinking them.
- Building the grid index takes 3 to 119 ms across the whole range.
- Headless browsers without a GPU draw on the CPU and are about a hundred times slower. Never judge speed from one.

Dataset size	What to expect
Up to 500,000	Full refresh rate. Nothing to think about.
1,000,000	A steady 30 fps at full detail. Every cell selectable.
2,000,000	18 fps at full detail. Auto detail draws a million and restores 30.
5,000,000	Auto detail all the time. A lasso takes about a quarter of a second.
10,000,000	Loads in under a second and selects in about 360 ms. Needs about 1 GB of memory.

CHAPTER 33

Cheat sheet and glossary

Mouse and keyboard

Action	Result
Drag (Pan mode)	Move the plot
Scroll / pinch	Zoom around the pointer
Click a dot	Open the details card
Click a graph node (trajectory)	Highlight closest nodes
Drag (Box or Lasso mode)	Select cells
`Esc`	Cancel a drag in progress, or close the card
Click / double-click a legend row	Hide / show only this
Click / `Shift`+click an anatomy region	Select its cells / add another region
Double-click an organ on the body	Open it
`Enter` in the chat box	Send. `Shift`+`Enter` makes a new line

Commands

Command	What it does
`npm install`	Download the web libraries
`npm run dev`	Start the web app for development (port 5173)
`npm run typecheck`	Check every TypeScript file for mistakes
`npm run build`	Typecheck, then make the production folder `dist/`
`npm run preview`	Serve `dist/` locally to try it
`.venv/bin/python -m server.prep ...`	Make datasets (Chapter 12)
`.venv/bin/python -m uvicorn server.main:app --port 8000 --env-file .env`	Start the data server

Names the browser stores for you

Key (localStorage)	Holds
`cellviewer.theme`	The chosen background
`cellviewer.sections`	Which side-panel sections are folded

Key (localStorage)	Holds
`cellviewer.inset`	Inset mode and frame position and size

Environment variables

Name	Where	Meaning
`ANTHROPIC_API_KEY`	server	Turns on the real chat model
`CHAT_MODEL`	server	Which model answers
`TILES_DIR`	server	Folder of datasets (default `data/tiles`)
`CORS_ORIGINS`	server	Extra allowed page origins
`API_TARGET`	Vite	Where the dev proxy sends `/api`
`VITE_API`	Vite build	Use a different API address from the page

Glossary

Term	Meaning
Alias	A name a dataset might use for an organ region.
Alpha	How see-through a color is. 255 is solid, 0 is invisible.
Binary attribute	Data given to the GPU as raw typed arrays rather than JavaScript objects.
Branch point	A node of the principal graph where the path splits.
Chunk	One piece (250,000 cells) of a dataset column.
Contrast ratio	How different two colors are in brightness. 3 to 1 or more is easy to read.
CSR	Compressed sparse row: a flat way to store a list of lists as two arrays.
Culling	Not drawing something because it cannot be seen.
Dim factor	How much color an unselected cell keeps.
Embedding	A flat map of cells (UMAP, t-SNE).
Effective selection	The raw selection minus cells hidden by the legend.
Geodesic distance	Distance measured along the graph, not in a straight line.
Heap	The memory a page uses for its data.
Hot loop	A loop that runs for every cell, so it must be written with great care.
Leaf	An end node of the principal graph.
Level	One value of a categorical field.
LOD	Level of detail: drawing only part of the data when there is too much.

Term	Meaning
Manifest	The JSON file that describes a dataset.
Orthographic	A flat camera with no perspective.
Prefix	The first part of a list. Because data is shuffled, any prefix is a random sample.
Principal graph	The thin tree over a trajectory.
Pseudotime	A number from 0 to 1 for how far along the process a cell is.
Raw selection	The cells inside your outline before the legend filter is applied.
Shader	A small program that runs on the GPU.
Tile	A dataset file (same idea as a chunk).
Typed array	A fixed-size list of numbers of one type, very fast.
Viewport / view	A rectangle of the canvas with its own camera.
Web Worker	A background thread in the browser.